

2026 삼성전자 DS Associate Architect 과정

Software Architecture 개론

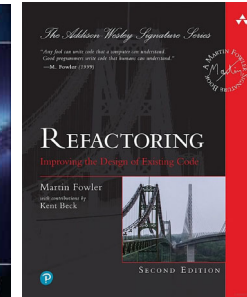
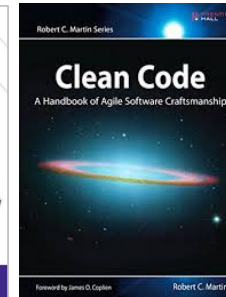
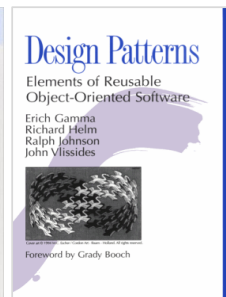
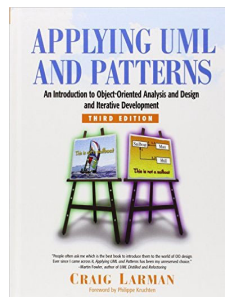
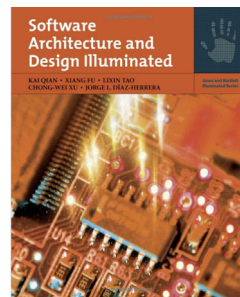
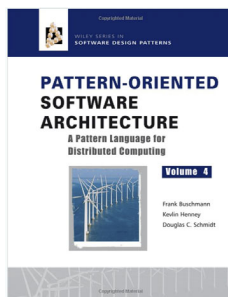
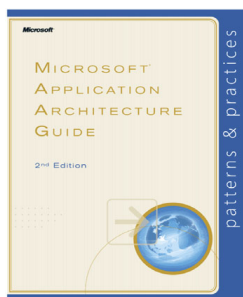
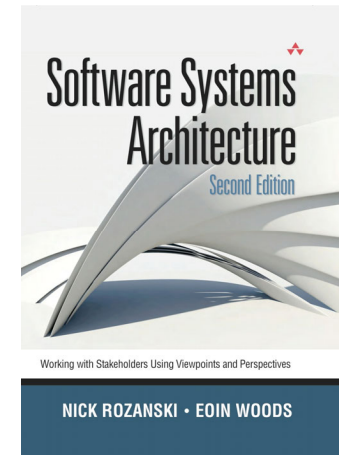
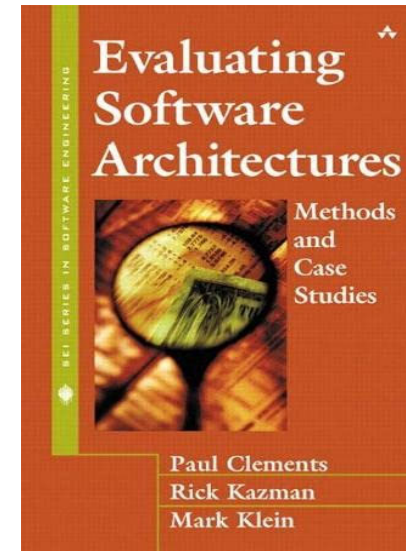
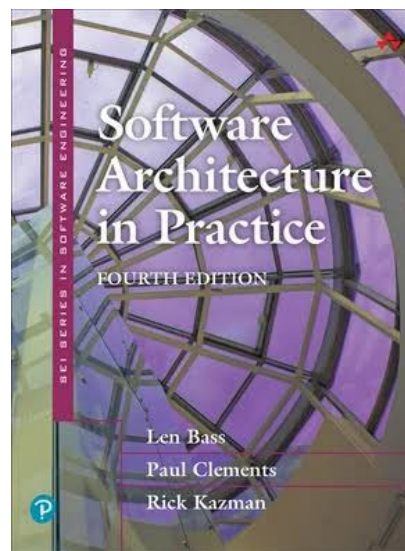
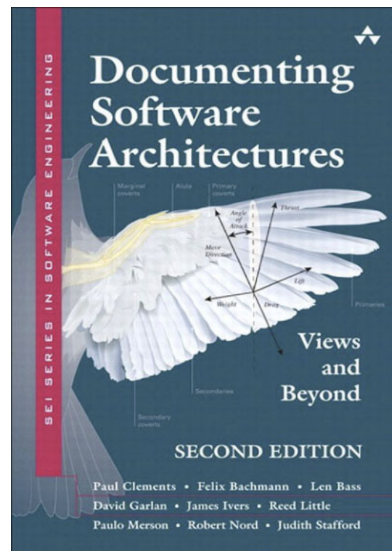
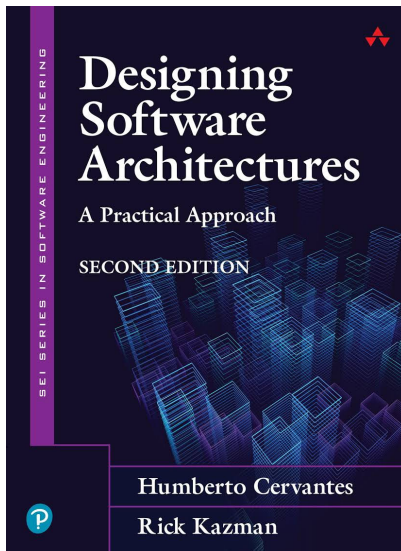
건국대학교 컴퓨터공학부
유준범

jbyoo@konkuk.ac.kr
<http://dslab.konkuk.ac.kr>

Index

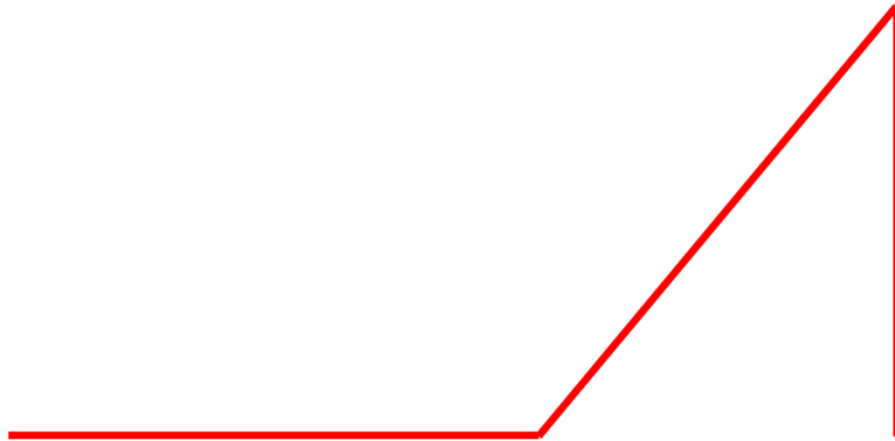
- **Software Architecture in a Nutshell**
- **Software Architecture Design**
- **Software Architecture Design in Our Architecture Description**
 1. Project Overview
 2. System Overview
 3. Architectural Drivers
 4. Architecture Design
 5. Summary
- **Techniques and Principles**
 - Design Concepts: Reference Architecture, Deployment Patterns, Architecture Styles/Patterns, Tactics
 - ISO/IEC 25023:2016 Product Quality Measures

Text and References



Software Architecture in a Nutshell

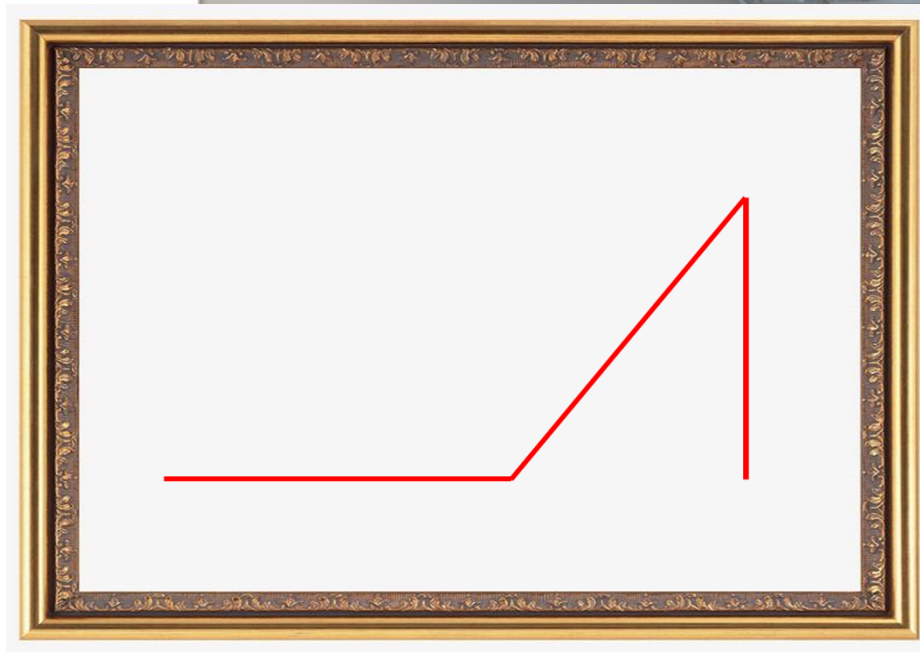
What is This?



This is



What is This? This is



작품명: 엄마 구두

작 가: 유오민 (3세)

설 명: 엄마의 7cm 하이힐을 보고
인상 깊었던 특징을 표현



What is Architecture Design?



Software (System)



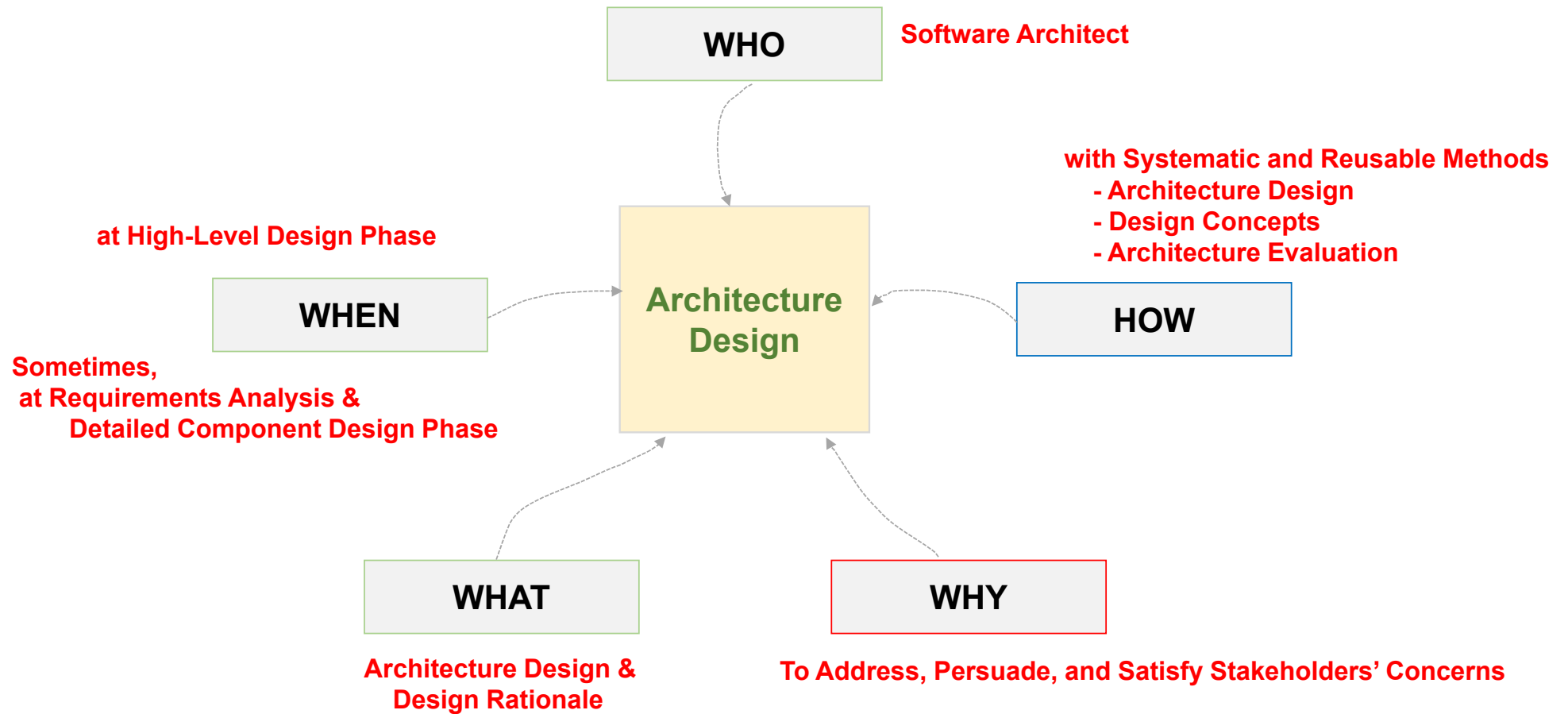
Architecture Design

+

작품명: 엄마 구두
작 가: 유O민 (3세)
설 명: 엄마의 7cm 하이힐을 보고
인상 깊었던 특징을 표현

Architecture Description (AD)

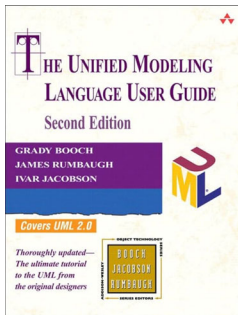
The 4W1H of Architecture Design



4W1H - WHO

- **Software Architect**

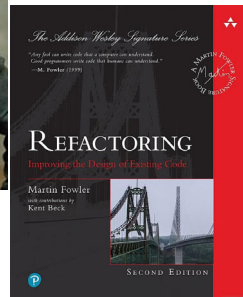
- A software architect is much more than “just” a software designer.
 - Has a long list of duties, skills, and knowledge that are necessary to be successful, including:
Leadership, Communication, Negotiation, Technical skills, Project skills, Analytical skills



Grady Booch



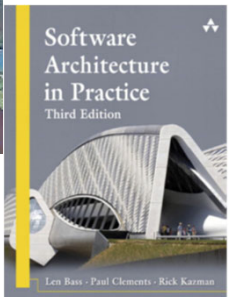
Martin Fowler



Robert C. Martin



Rick Kazman



4W1H - WHEN

- At the **early stage** of **Software Design**

- Sometimes, but very often, **before** and **after** it

High-Level Design
or
Architecture Design

Requirements
Analysis

- **Requirements Analysis**

Requirements Engineering
- Requirement Elicitation/Analysis, Specification, and Validation

SRS

Design

- **Detailed Component Design**

Low-Level Design
or
Component Design

SDS

Implementation

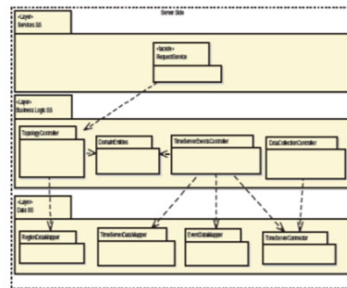
System Test

The Waterfall (Software Development Process) Model

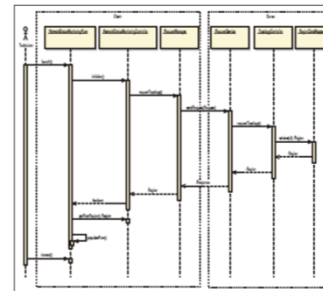
4W1H - WHAT

- **Proposing, Evaluating** and **Explaining** YOUR Software Architecture Design with **Multiple Views**

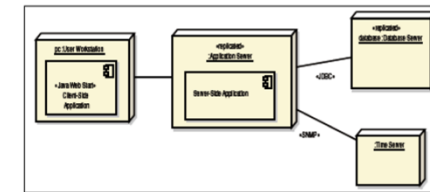
- Structure View
- Behavioral View
- Deployment View
- Other Many Views



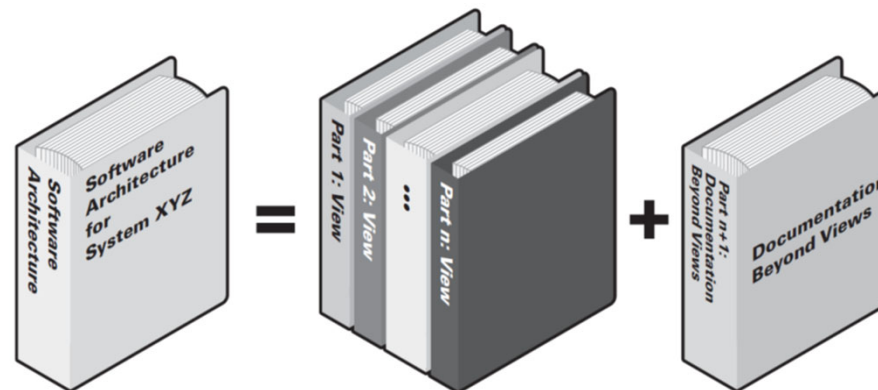
UML Component Diagram



UML Sequence Diagram



UML Deployment Diagram



CMU SEI's Architecture Documentation

4W1H - HOW

- Using **Systematic and Structured methods** for designing and evaluating software architecture
 - **ADD 3.0** (Attribute Driven Design 3.0)
 - **ATAM** (Architecture Trade-off Analysis Method)

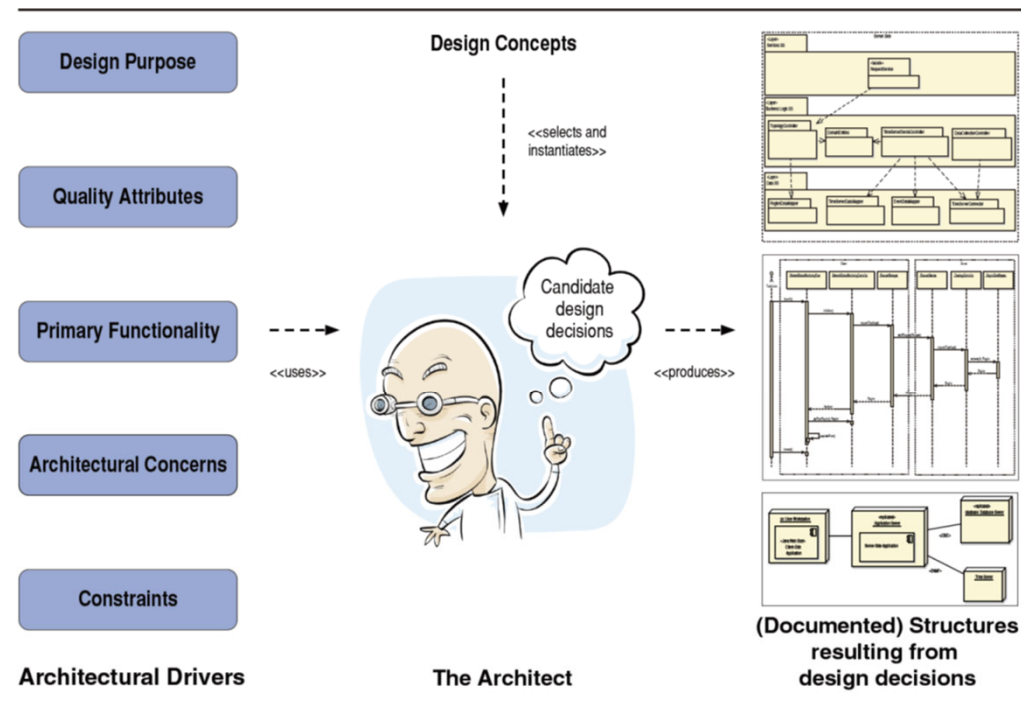
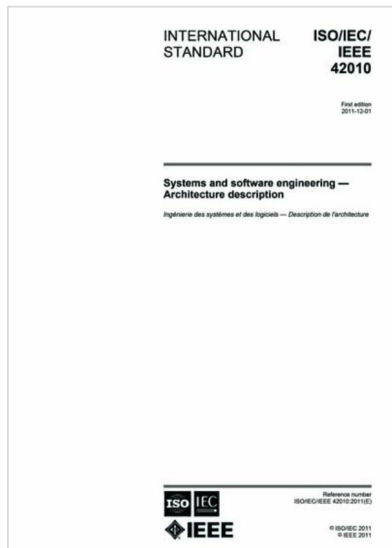


FIGURE 2.1 Overview of the architecture design activity
(Architect image © Brett Lamb | Dreamstime.com)

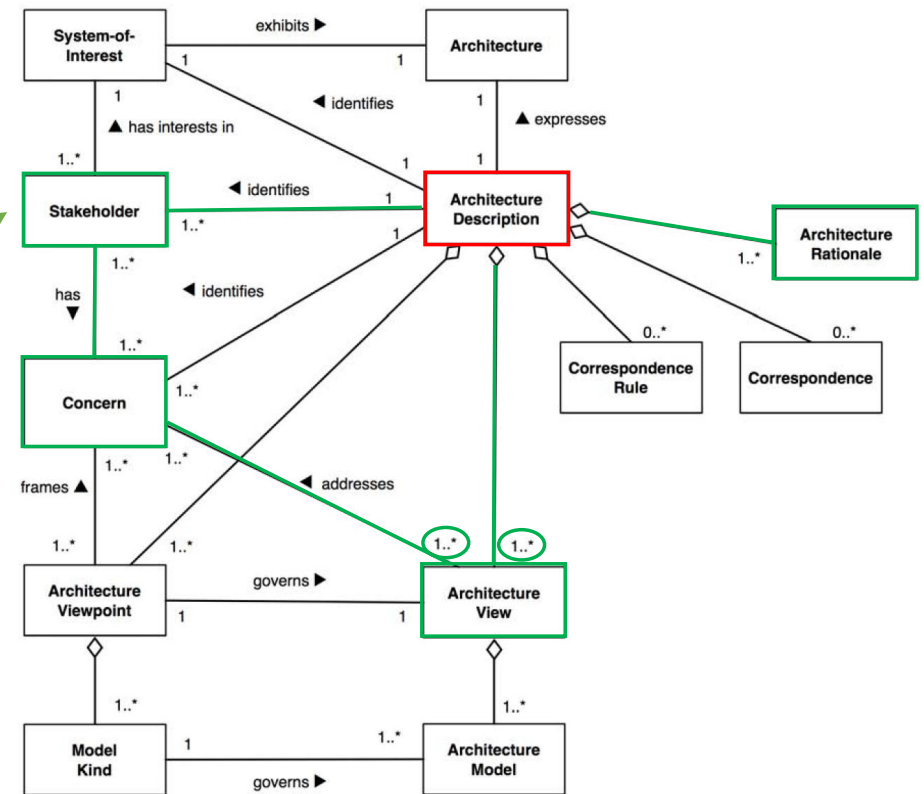
4W1H - WHY

- AD (Architecture Description) should demonstrate How an architecture meets the needs of the system's diverse stakeholders.
- AD is not just a technical report, but a document like a research paper to convince stakeholders.



Stakeholders

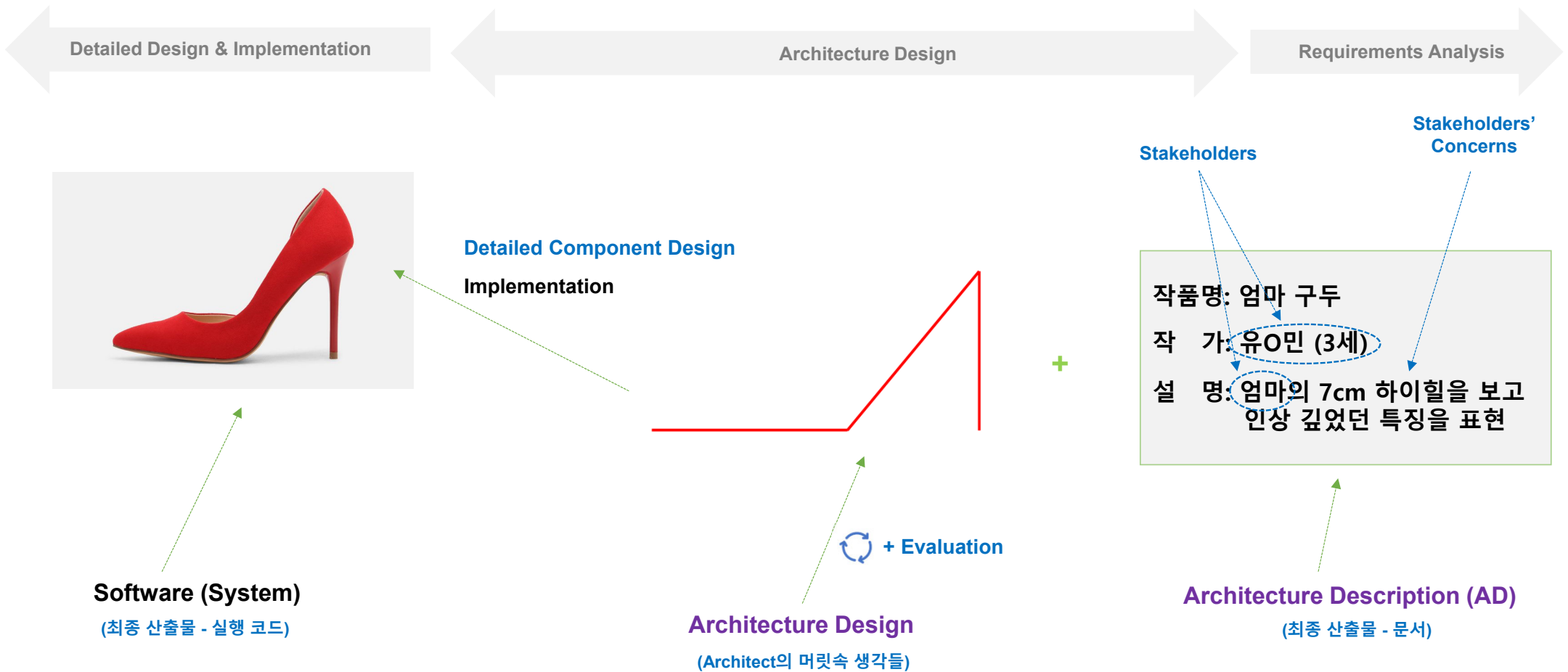
- 이해관계자 / 이해당사자
- 주주
- 표준 및 규제기관
- 고객 (Customer)
- 고객사 (Client)
- 소비자 (Consumer)
- 개발자
- 마케팅 담당자



ISO/IEC/IEEE 42010:2011

"Systems and Software Engineering - Architecture Description"

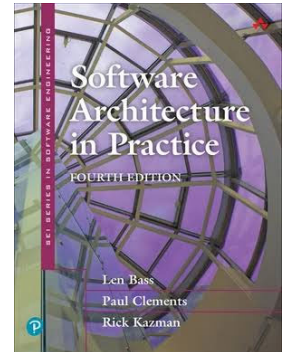
What is This? (Revisited)



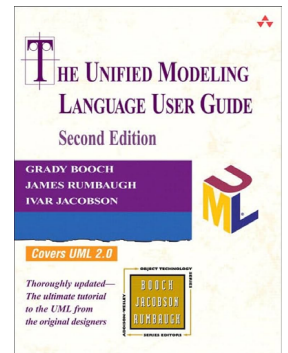
Software Architecture Design

Software Architecture

*“The **software architecture** of a system is **the set of structures** needed to reason about the system, which comprise software elements, relations among them, and properties of both. ”*



*“A **software architecture** is the set of significant decisions about the organization of a software system, the selection of the **structural** elements and their **interfaces** by which the system is composed, together with their **behavior** as specified in the collaborations among those elements, the composition of these structural and behavioral elements into progressively larger subsystems, and the architectural style that guides this organization - these elements and their interfaces, their collaborations, and their composition. ”*



Software Architecture Design

through a set of Views
(Structure, Behavior, Deployment, etc.)

- **The process of creating a specification** of software elements, intended to accomplish goals, subject to constraints.

ASR - Architectural Drivers
→ Constraints

AD (Architecture Description)

ASR - Architectural Drivers
→ Primary Functionality, QAS

Importance of Software Architecture

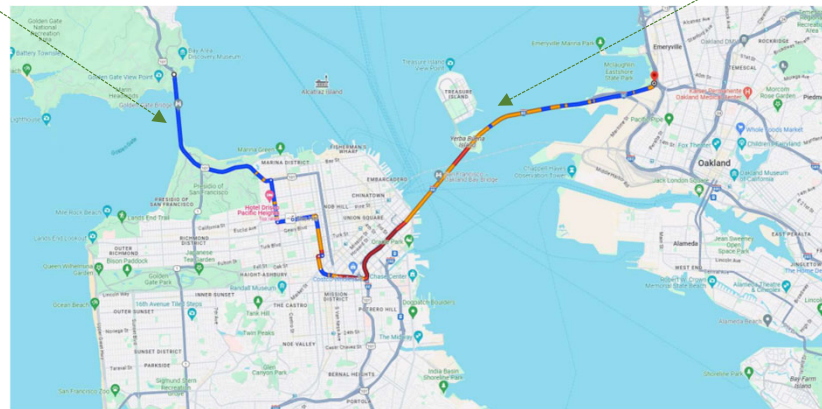


Golden Gate Bridge (1937)



San Francisco - Oakland Bay Bridge (1936)

Collapsed in 1987 on 6.9 earthquake



San Francisco, USA

Importance of Architectural Design

There is a [very high cost to a project](#) of not making certain design decisions or not making them early enough. You cannot confidently predict project cost, schedule, and quality without them.

The architecture will [influence other decisions](#) that are not in and of themselves design decisions such as selection of tools or structuring of development environment.

A well-designed, properly communicated architecture is [key to achieving agreements](#) that will guide the team.

Software Architecture Life-Cycle Activities

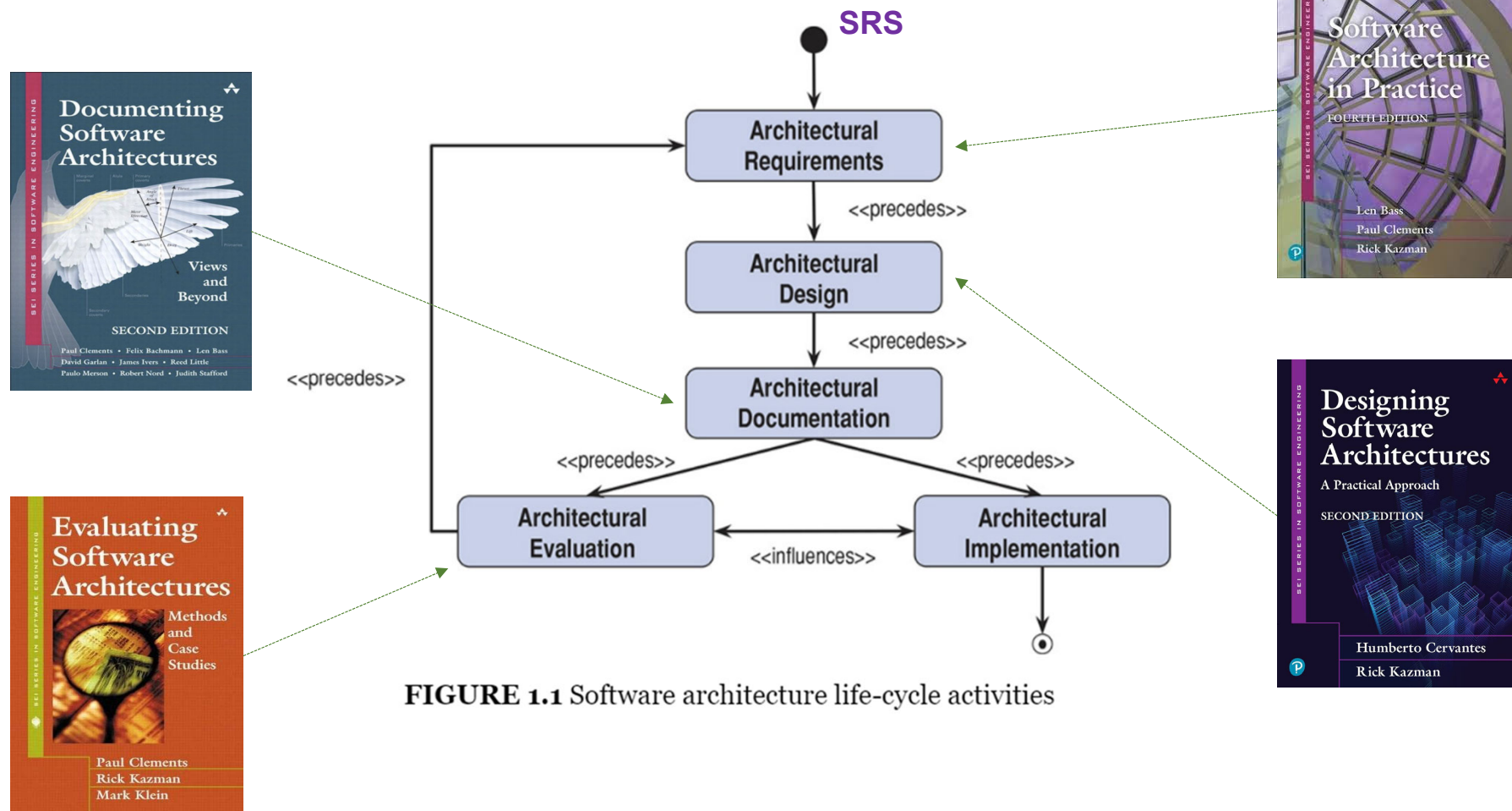


FIGURE 1.1 Software architecture life-cycle activities

Mapping to the AA Curriculum

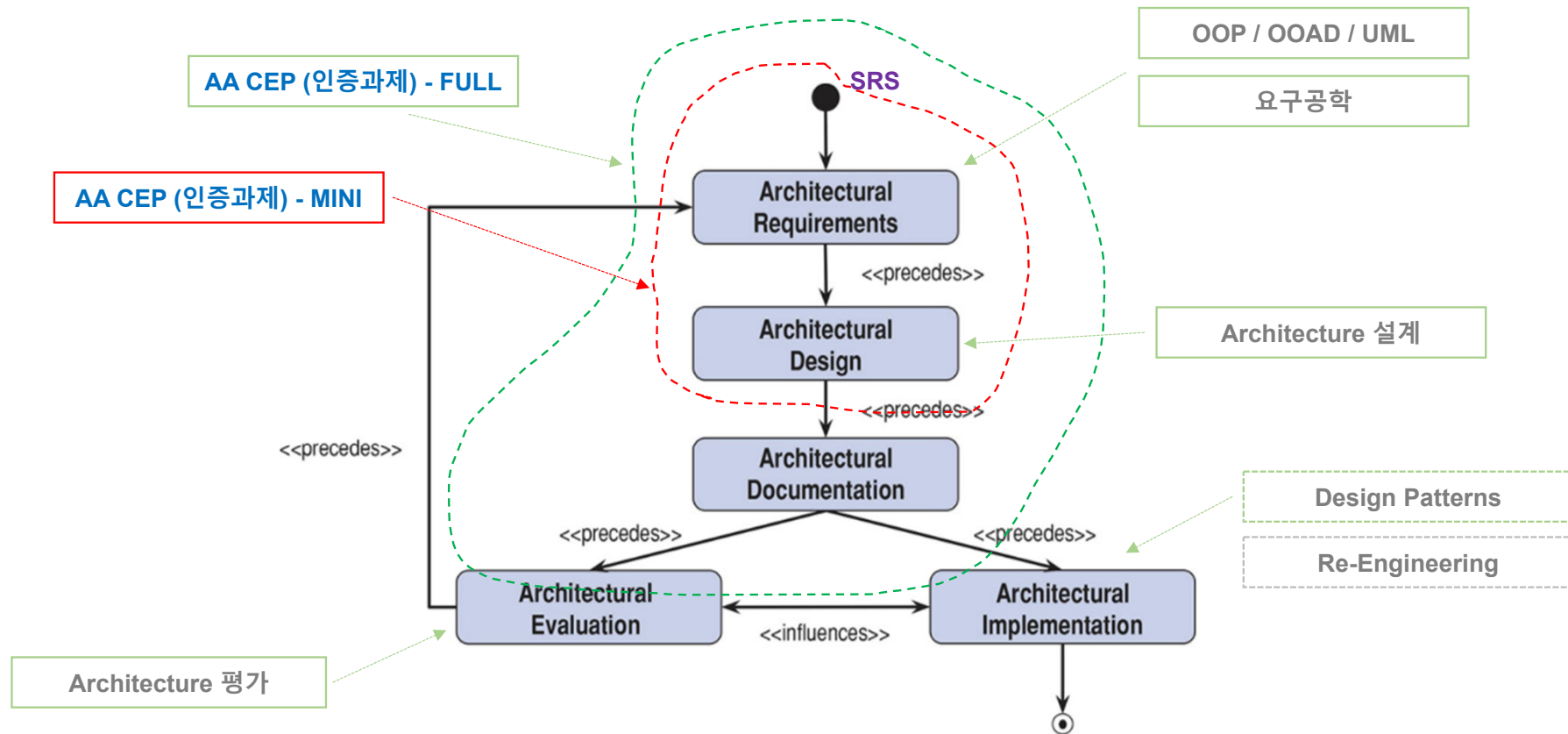


FIGURE 1.1 Software architecture life-cycle activities

Architectural Requirements

- **Architecturally Significant Requirements (ASR)**

- A small set of requirements in SRS, that have special importance for the architecture
- Examples :
 - **Primary functionality** : the most important functionality of the system
 - **QA (Quality Attribute)** : quality attributes such as high performance, high availability, or ease of maintenance
 - **Other design constraints**

- **ASR Analysis**

- Identifying all ASRs from an SRS, and then
- Transforming (or Mapping) **ASR into AD** (Architectural Drivers)

- SRS often provides very little information for architects.

- Architects need to be involved in requirements analysis.
 - Stakeholder analysis
 - User Requirements elicitation
 - Keeping traceability starting from stakeholders

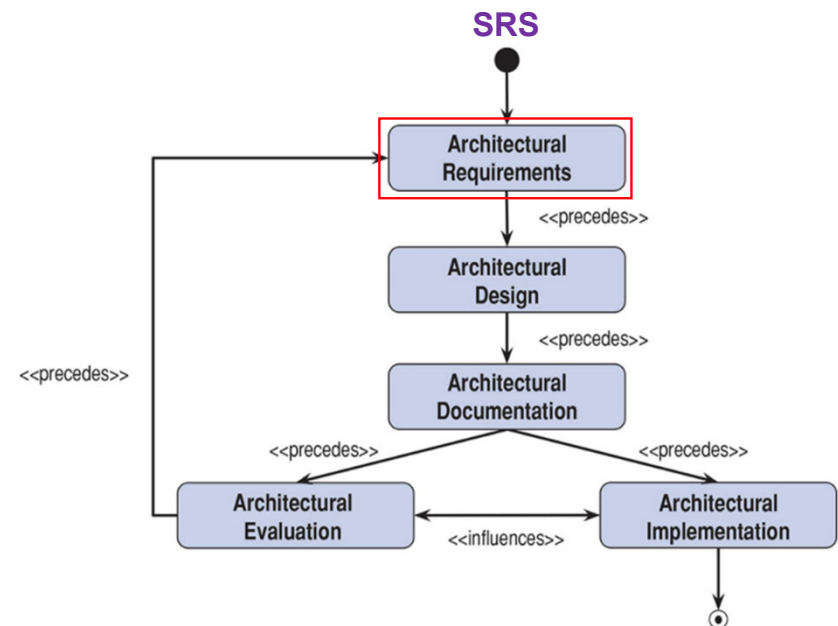


FIGURE 1.1 Software architecture life-cycle activities

Architectural Design

- **The process** of translation from the world of requirements to the world of solutions
 - Producing a set of structures composed of code, frameworks, and components
 - Example :
 - **ADD (Attribute Driven Design) 3.0** with **Design Concepts**
- A good design is one that satisfies all **AD (Architectural Drivers)**.

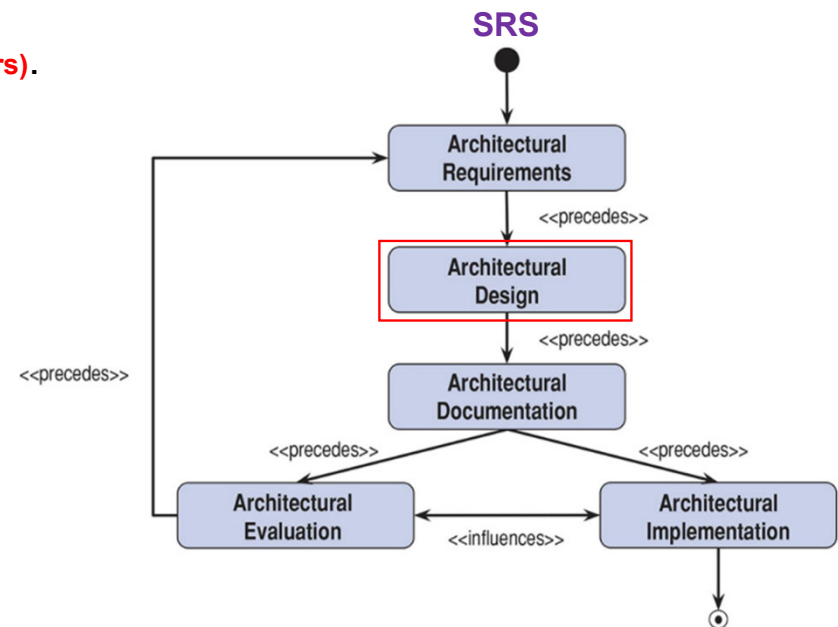


FIGURE 1.1 Software architecture life-cycle activities

Architectural Documentation

- **Preliminary documentation of the structures** should be created as part of architectural design.
 - Iterative and incremental documenting
- **Architecture Description (AD)**
 - ISO/IEC/IEEE 42010:2011 Systems and Software Engineering - Architecture Description

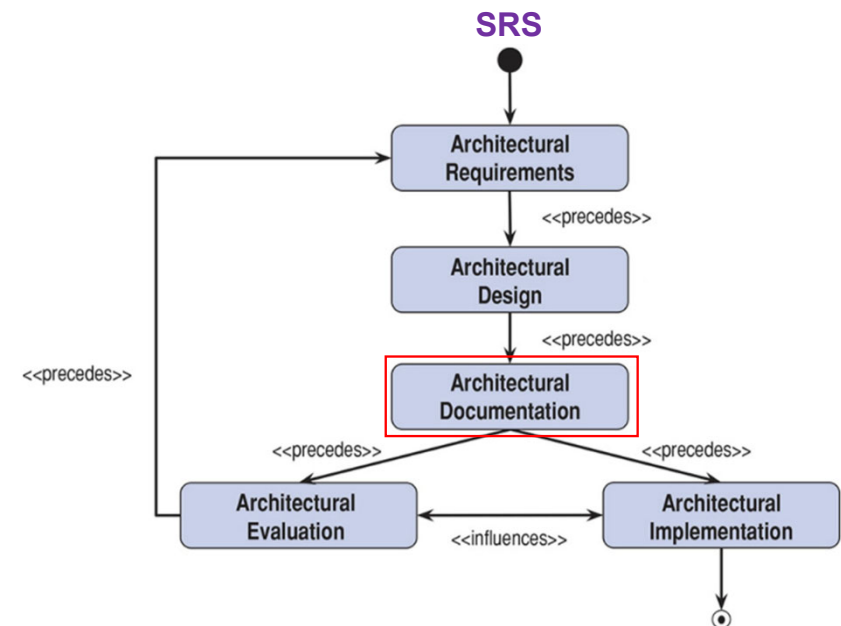


FIGURE 1.1 Software architecture life-cycle activities

Architectural Evaluation

- **Evaluating your architectural design** to ensure that the decision made are **appropriate to address all ASRs**
 - Example :
 - **ATAM** (Architecture Trade-off Analysis Method)
 - **CBAM** (Cost Benefit Analysis Method)
 - **SAAM** (Software Architecture Analysis Method)

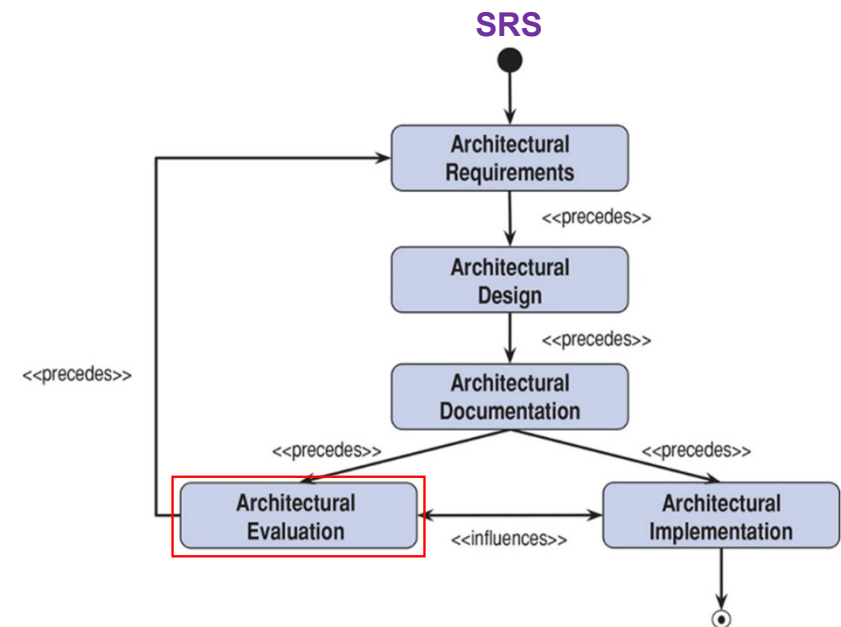


FIGURE 1.1 Software architecture life-cycle activities

Architectural Implementation

- **Implementing** the architecture that you have created and evaluated
- Low-level design and coding are often intertwined very closely.
 - **Low-level design** : Detailed component designs
 - OOD (Object-Oriented Design) from OOAD
 - SD (Structured Design) from SASD
 - **Implementation (Coding)** :
 - OOD → Object-Oriented Programming (C++ / Java)
 - SD → Procedural Programming (C / Fortran)
- **Refactoring**
 - Considering **Maintainability** of code
 - Agile
 - Code Review, TDD, CI/CD, Refactoring
 - Design Patterns

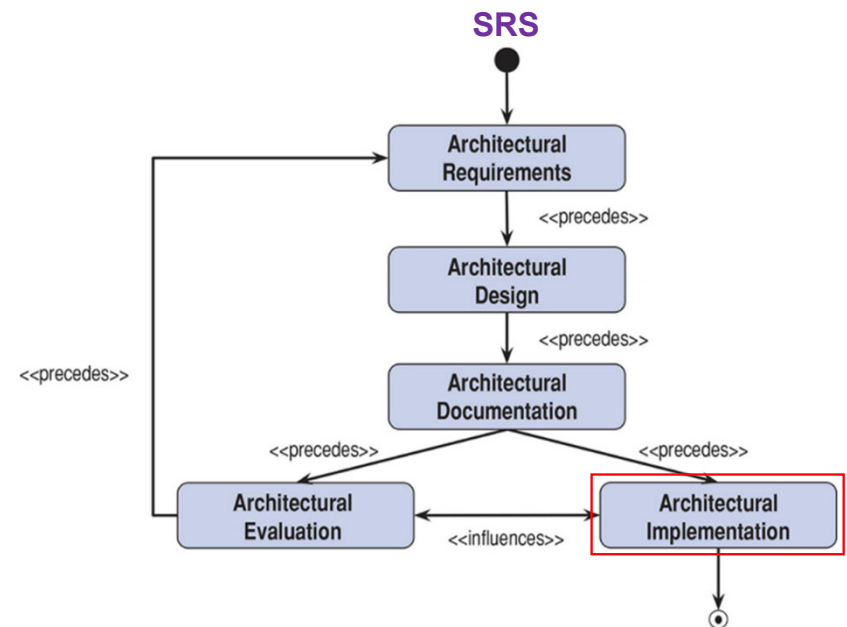
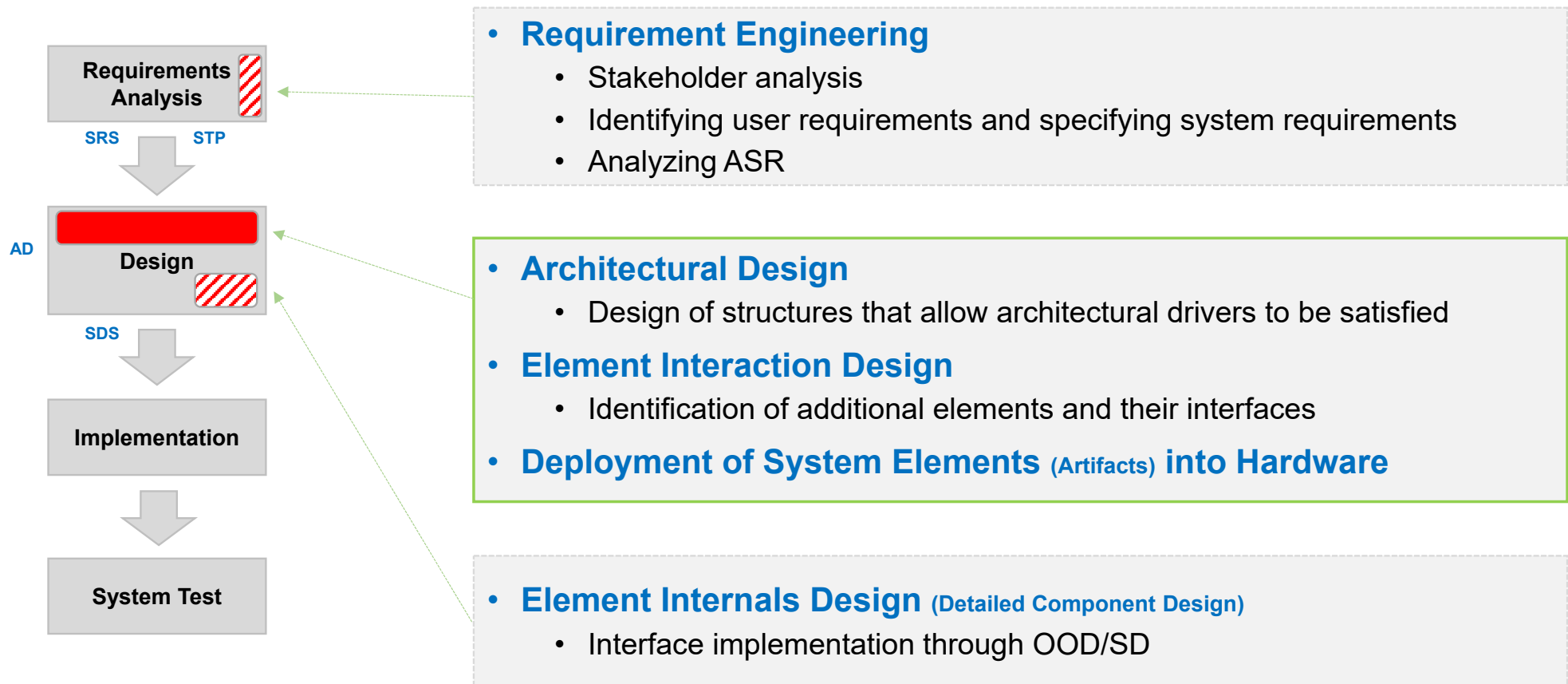


FIGURE 1.1 Software architecture life-cycle activities

Scope of Software Architecture Design



SRS : Software Requirement Specification (IEEE Std 830.1998)
STP : System Test Plan (IEEE Std 829.1998)
SDS : Software Design Specification (IEEE Std 1016.1998)
AD : Architecture Description (ISO/IEC/IEEE 42010:2011)

Software Architecture Design in a Nutshell

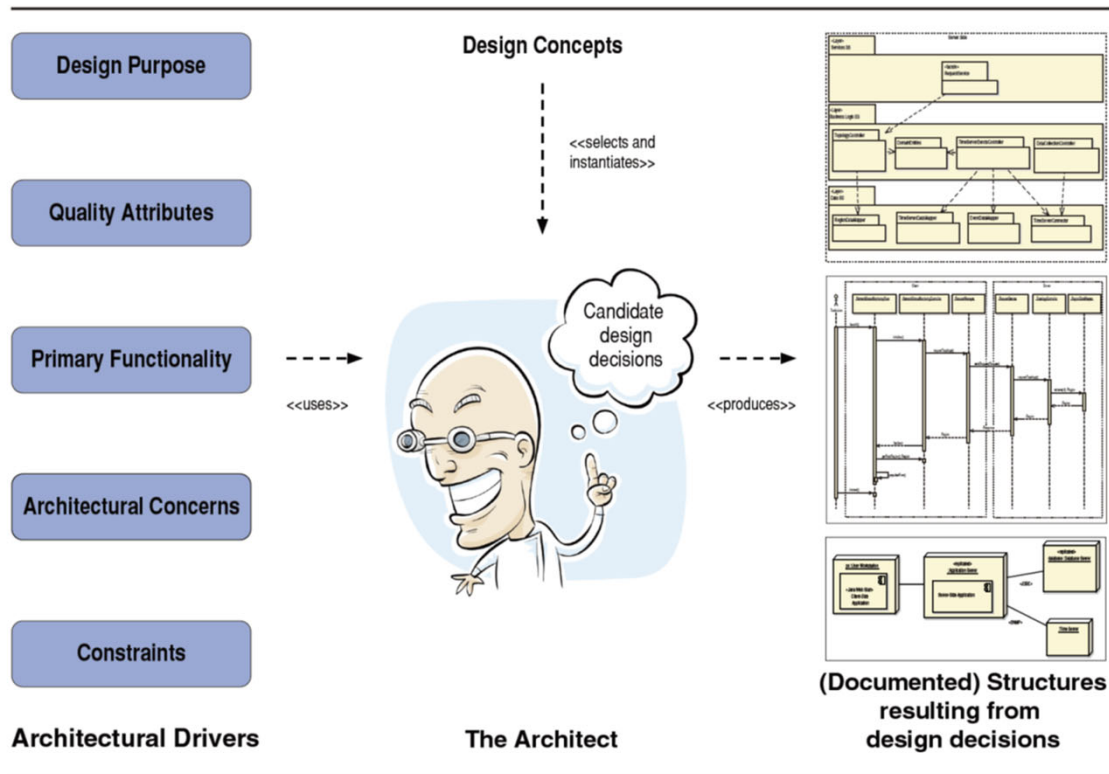
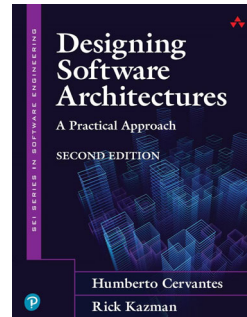


FIGURE 2.1 Overview of the architecture design activity
(Architect image © Brett Lamb | Dreamstime.com)

(1st Edition)

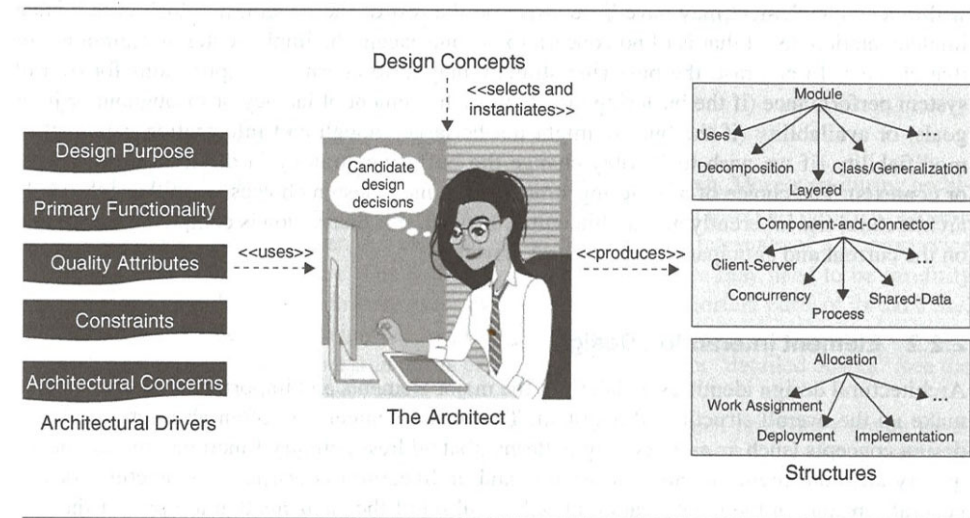
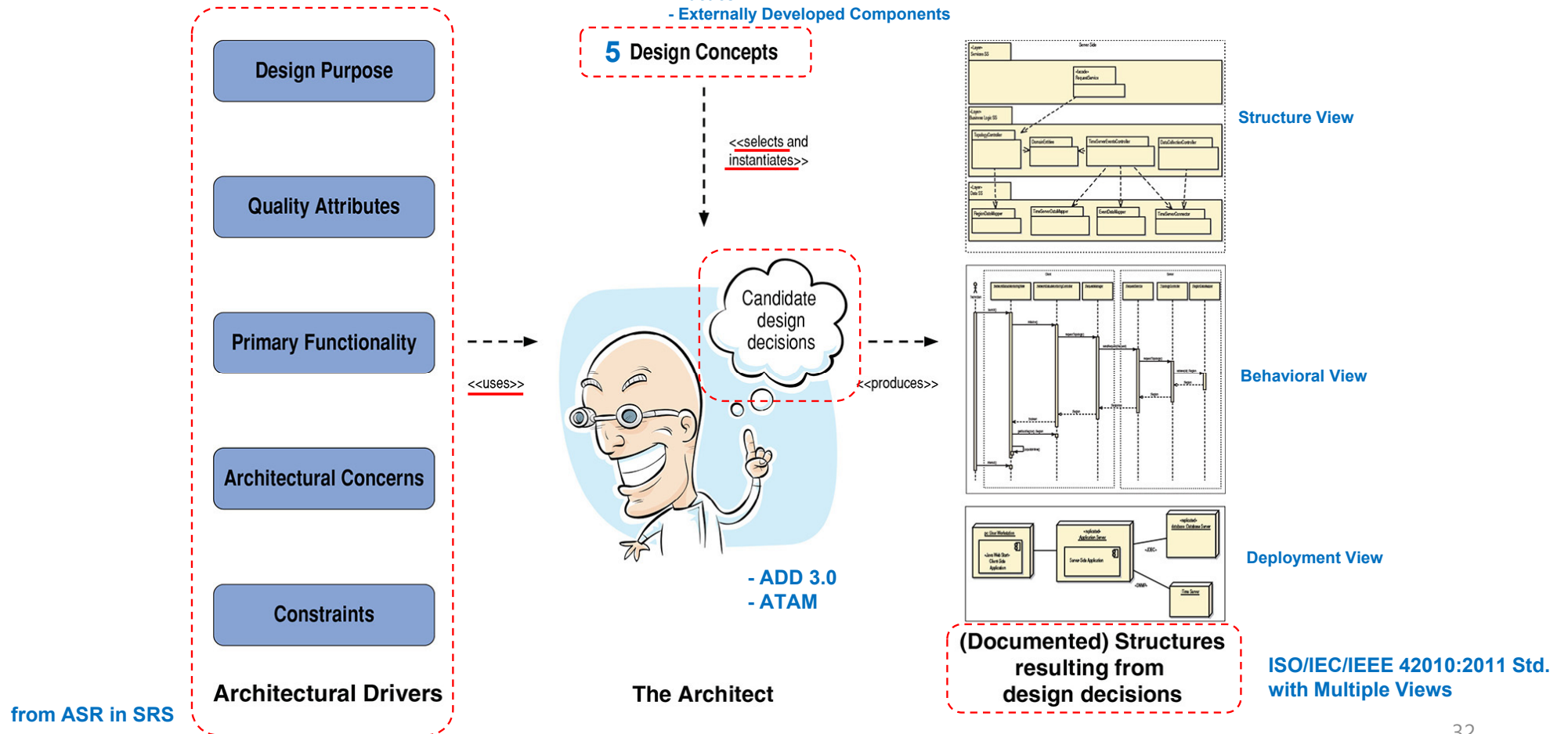


FIGURE 2.1 Overview of the architecture design activity

(2nd Edition)

Software Architecture Design in a Nutshell

- Reference Architecture
- Deployment Pattern
- Architecture Style
- Tactics
- Externally Developed Components



Software Architecture Design Approach in a Nutshell

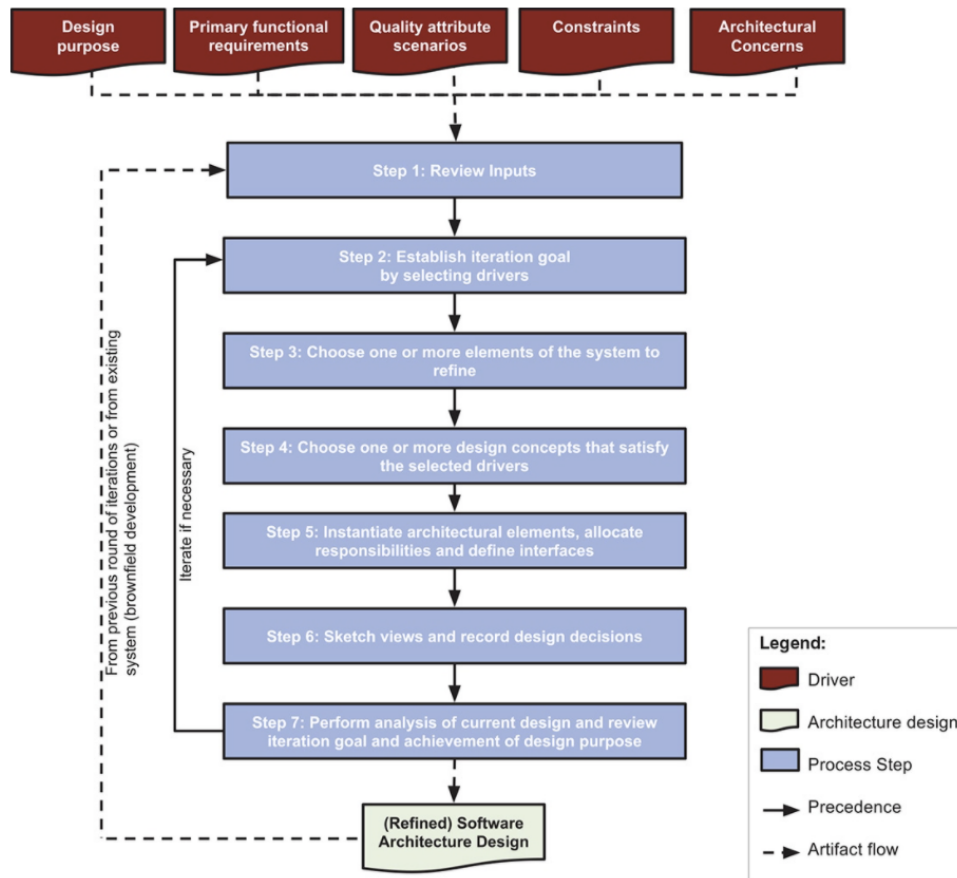


FIGURE 3.1 Steps and artifacts of ADD version 3.0

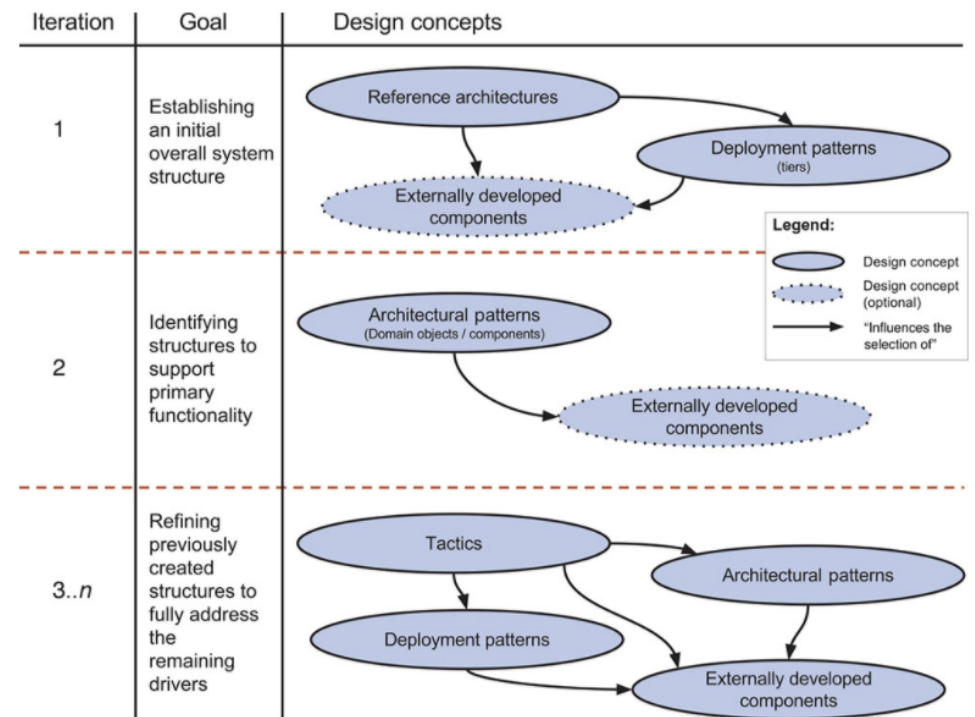


FIGURE 3.2 Design concept selection roadmap for greenfield systems

Design Concepts

- Building blocks for creating structures

Reference Architecture

- Blueprint for overall architecture
- Logical structure for specific application types
- Embodying architecture styles

Examples:

- Mobile applications
- Rich client applications
- Rich internet applications
- Service applications
- Web applications



Deployment Pattern

- How to deploy logical into physical
- Physical structure
- Essential for many QAs (Performance, Security, Availability, ...)

Examples:

- Nondistributed
- Distributed
- Performance
- Reliability
- Security



Architecture Style

- General and reusable structural layout and its properties
- Not domain-specific
- Logical >> Physical structure

Examples:

- MVC, MVP, MVVM, Layered
- Client-Server, SOA, Microservices
- Pipes and Filters, Blackboard



Tactics

- Building blocks of other patterns
- Widely-used techniques to manage QAs by architects
- Quickly evolved

Examples:

- Availability
- Interoperability
- Modifiability
- Performance
- Security
- Testability
- Usability



Externally Developed Components

- Generally called COTS software (Commercial Off-The-Shelf)

Examples:

- Technical family
- Products (COTS)
- Application framework (Hibernate, Rest, Spring, Swing, etc.)
- Platform (Java, .Net, Google Cloud, etc.)



Reference Architecture : Examples

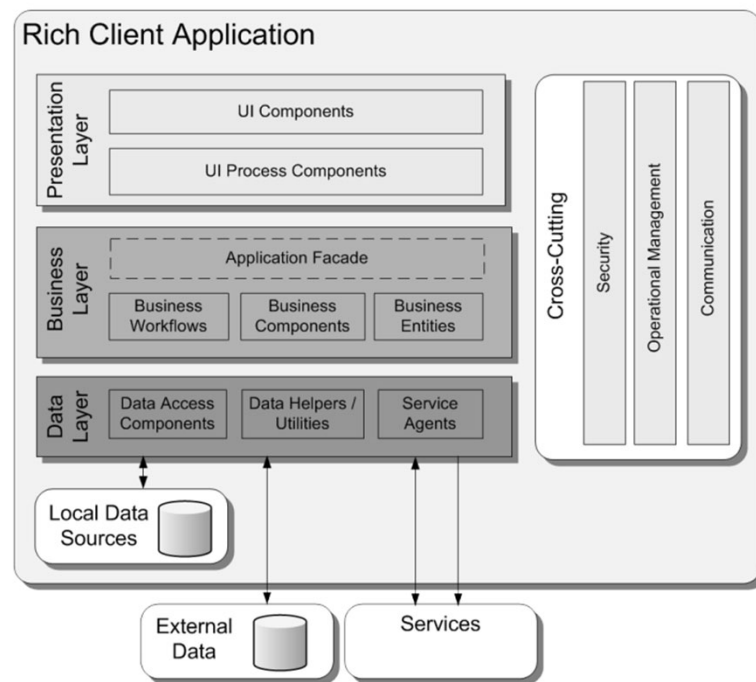
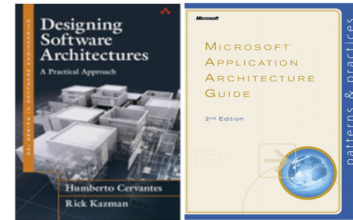


Figure 2 Rich client application

Rich Client Application

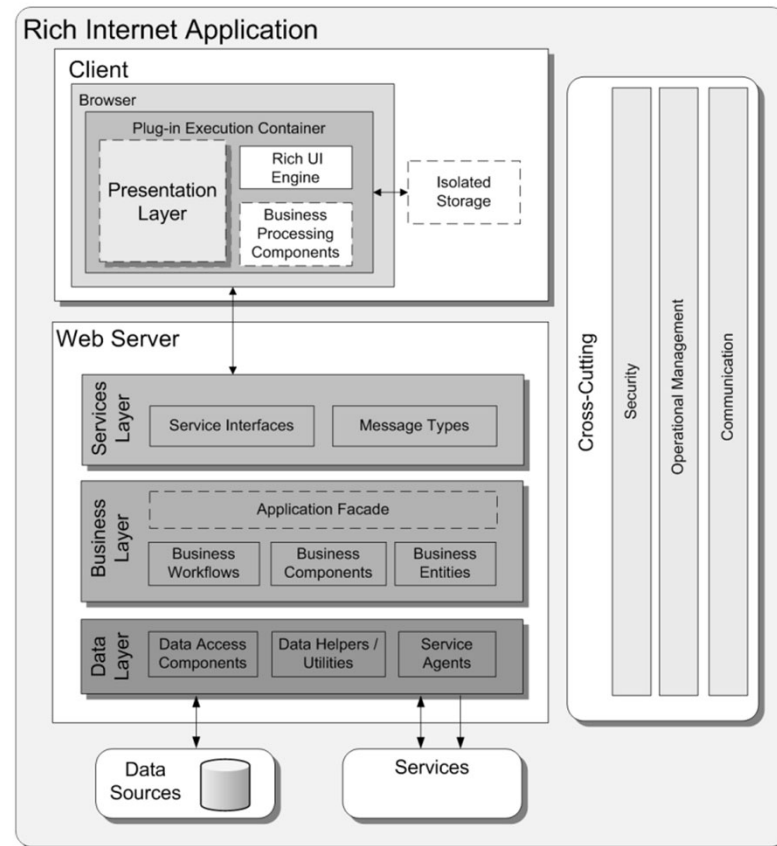


Figure 3 Rich Internet application

Rich Internet Application

Deployment Pattern : Examples

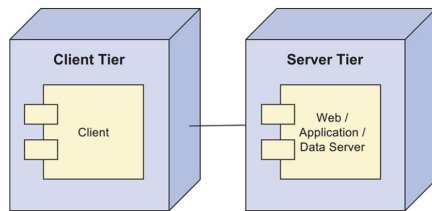


FIGURE A.8 Two-tier deployment pattern (Key: UML)

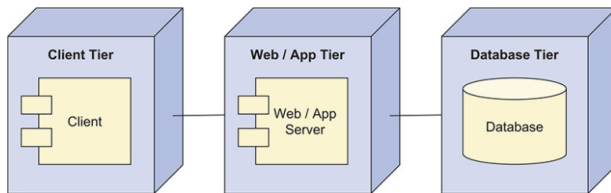


FIGURE A.9 Three-tier deployment pattern (Key: UML)

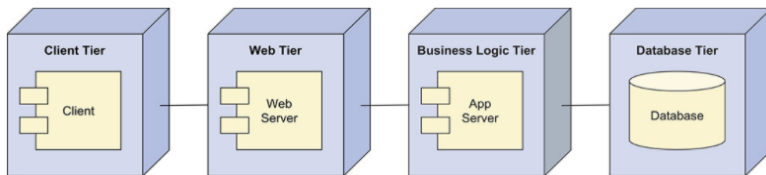


FIGURE A.10 Four-tier deployment pattern (Key: UML)

Distributed

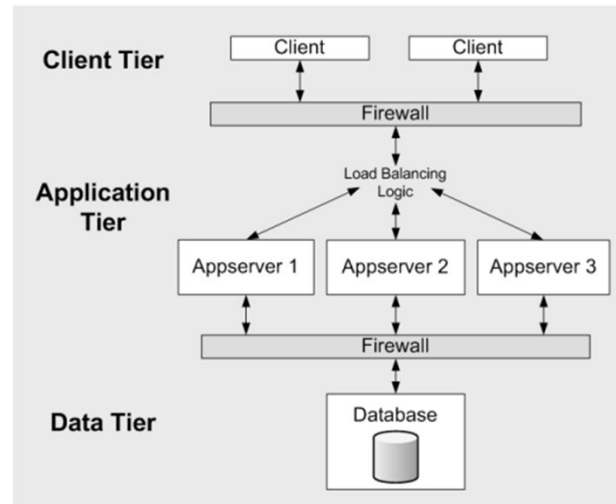


Figure 4 A load-balanced cluster

Performance

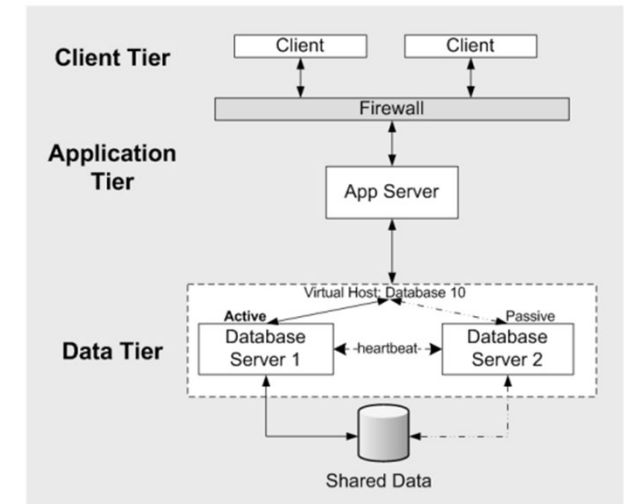
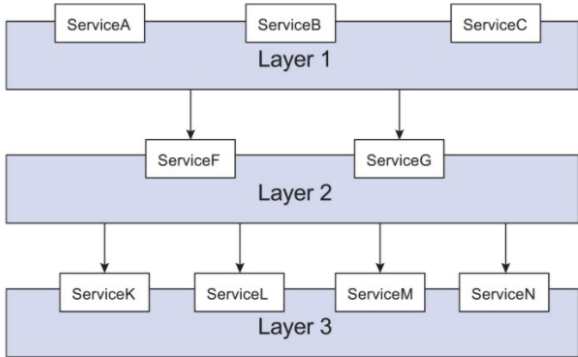


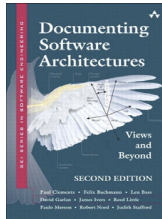
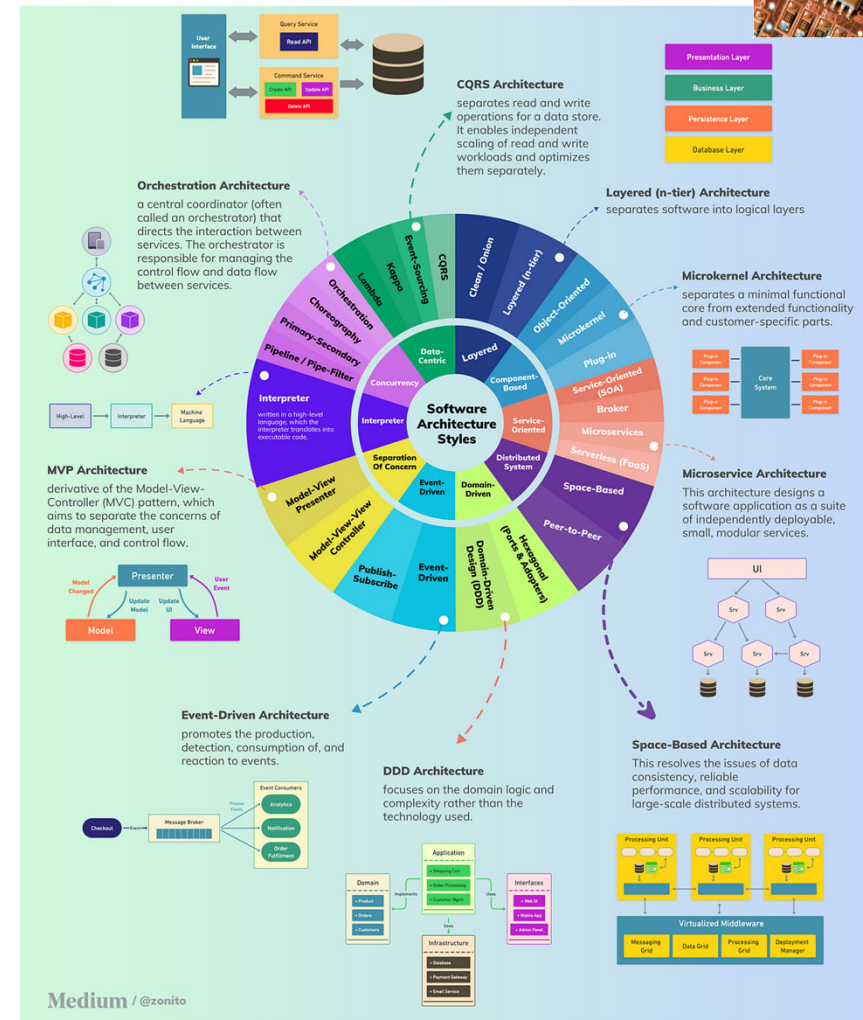
Figure 5 A failover cluster

Reliability / Availability

Architecture Style : Examples

Name	Layers
Problem and context	When transforming a Domain Model (182) into a set of modules that can be allocated to teams, [...] we need to support several concerns: the independent development of the modules, the independent evolution of the modules, the interaction among the modules.
Solution	Define two or more layers for the software under development, where each layer has a distinct and specific responsibility. To make the layering more effective, the interactions between the layers should be highly constrained. The strictest layering, as shown below, allows only unidirectional dependencies and forbids layer-bridging.
Structure	 Legend: - Layer/Service/Module - Data store - Service Interface/Method - Calls/Invokes
Consequences and related patterns	Typically, each self-contained and coherent responsibility within a layer is realized as a separate domain object. Domain objects are the containers (modules) that can be developed and evolved independently.

Layered Architecture Style



Tactics : Examples

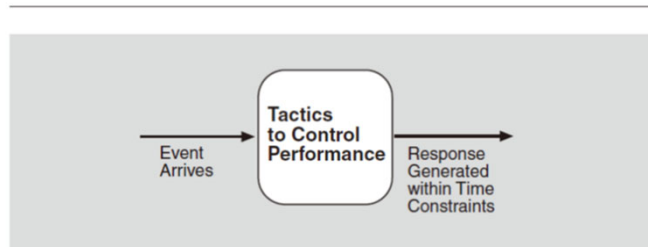
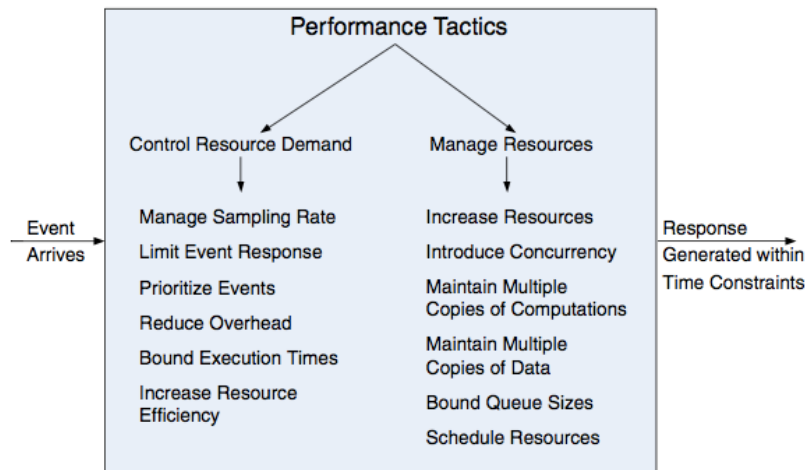


FIGURE 8.2 The goal of performance tactics



Performance

The Journal of Systems & Software 197 (2023) 111558

Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss

Architectural tactics in software architecture: A systematic mapping study[☆]

Gastón Márquez^{a,*}, Hernán Astudillo^b, Rick Kazman^c

^a Department of Electronics and Informatics, Universidad Técnica Federico Santa María, Concepción, Chile
^b Department of Informatics, Universidad Técnica Federico Santa María, Santiago, Chile
^c Department of Information Technology Management, University of Hawaii, Honolulu, HI, USA

ARTICLE INFO

Article history:
 Received 16 September 2021
 Received in revised form 8 November 2022
 Accepted 12 November 2022
 Available online 22 November 2022

Keywords:
 Architectural tactics
 Systematic mapping study
 Software architecture
 Quality attributes

ABSTRACT

Architectural tactics are a key abstraction of software architecture, and support the systematic design and analysis of software architectures to satisfy quality attributes. Since originally proposed in 2003, architectural tactics have been extended and adapted to address additional quality attributes and newer kinds of systems, making quite hard for researchers and practitioners to master this growing body of specialized knowledge. This paper presents the design, execution and results of a systematic mapping study of architectural tactics in software architecture literature. The study found 552 studies in well-known digital libraries, of which 79 were selected and 12 more were added with snowballing, giving a total of 91 primary studies. Key findings are: (i) little rigor has been used to characterize and define architectural tactics; (ii) most architectural tactics proposed in the literature do not conform to the original definition; and (iii) there is little industrial evidence about the use of architectural tactics. This study organizes and summarizes the scientific literature to date about architectural tactics, identifies research opportunities, and argues for the need of more systematic definition and description of tactics.

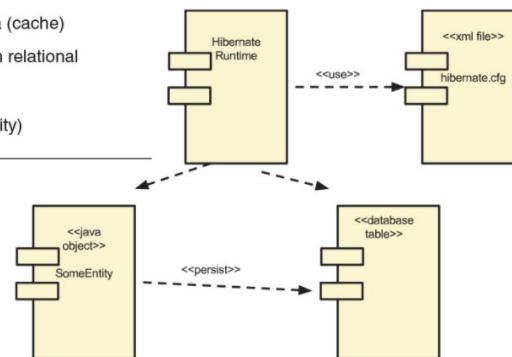
Editor's note: Open Science material was validated by the Journal of Systems and Software Open Science Board.

© 2022 Elsevier Inc. All rights reserved.

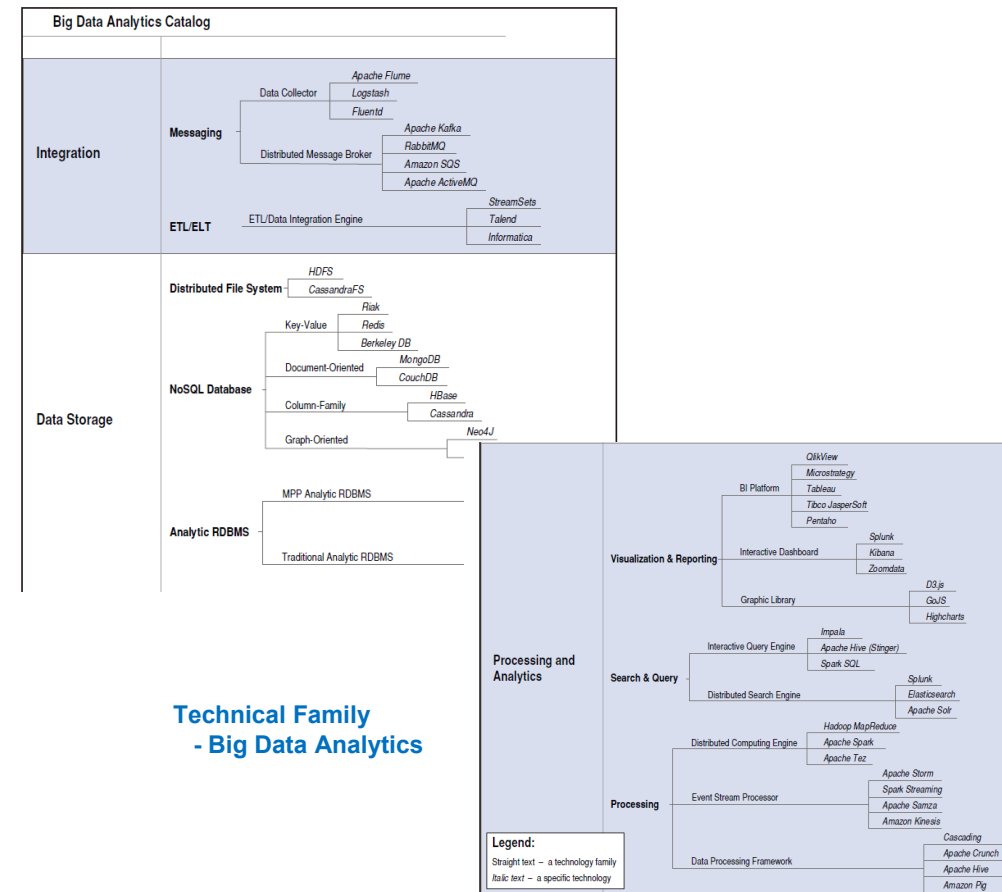
Externally Developed Components : Examples

Framework Name	Hibernate
Technology family	Object-oriented to relational mapper
Language	Java
URL	http://hibernate.org/
Purpose	Simplify persistence of objects in a relational database.
Overview	Hibernate allows objects to be easily persisted in a relational database (and it supports different database engines). Object-relational mapping rules are described declaratively in an XML file called <code>hibernate.cfg</code> or using annotations in the classes whose objects need to be persisted. Hibernate supports transactions and provides a query language called HQL (Hibernate Query Language) that is used to retrieve objects from the database. Hibernate utilizes multilevel caching schemes to improve performance. It also provides mechanisms to allow lazy acquisition of dependent objects to improve performance and reduce resource consumption. These mechanisms are configured declaratively in the configuration files.
Implemented design patterns and tactics	Patterns: - Data Mapper - Resource Cache - Lazy Acquisition Tactics: - Availability: Transactions - Performance: Maintain multiple copies of data (cache)
Benefits	Greatly simplifies the persistence of objects in relational database
Limitations	- Complex API - Slower than JDBC (Java Database Connectivity) - Difficult to map to legacy database schemas

Framework - Hibernate



This diagram represents an entity that is persisted to a database by the Hibernate runtime using the information in the configuration file (Key: UML)



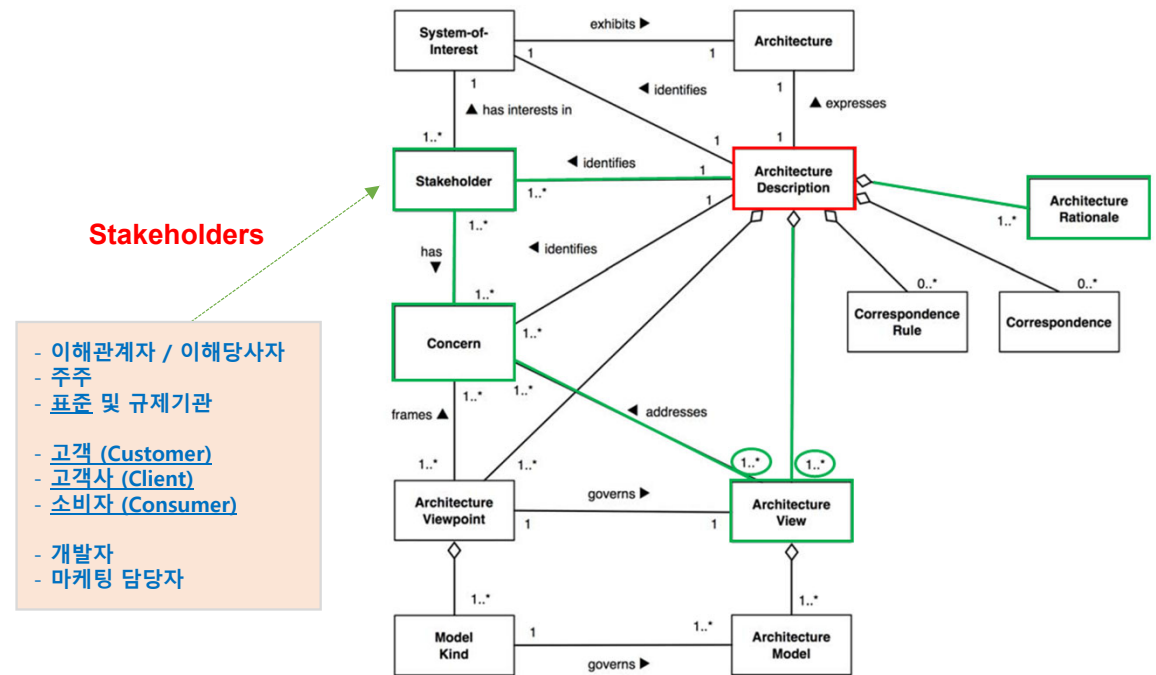
Technical Family - Big Data Analytics

FIGURE 2.10 A technology family tree for the Big Data application domain

Architecture Description (AD)

- **ISO/IEC/IEEE 42010:2011** “Systems and Software Engineering - Architecture Description”
 - Specifying requirements to be an architectural description (AD)
 - AD should demonstrate how an architecture meets the needs of the system’s diverse stakeholders.

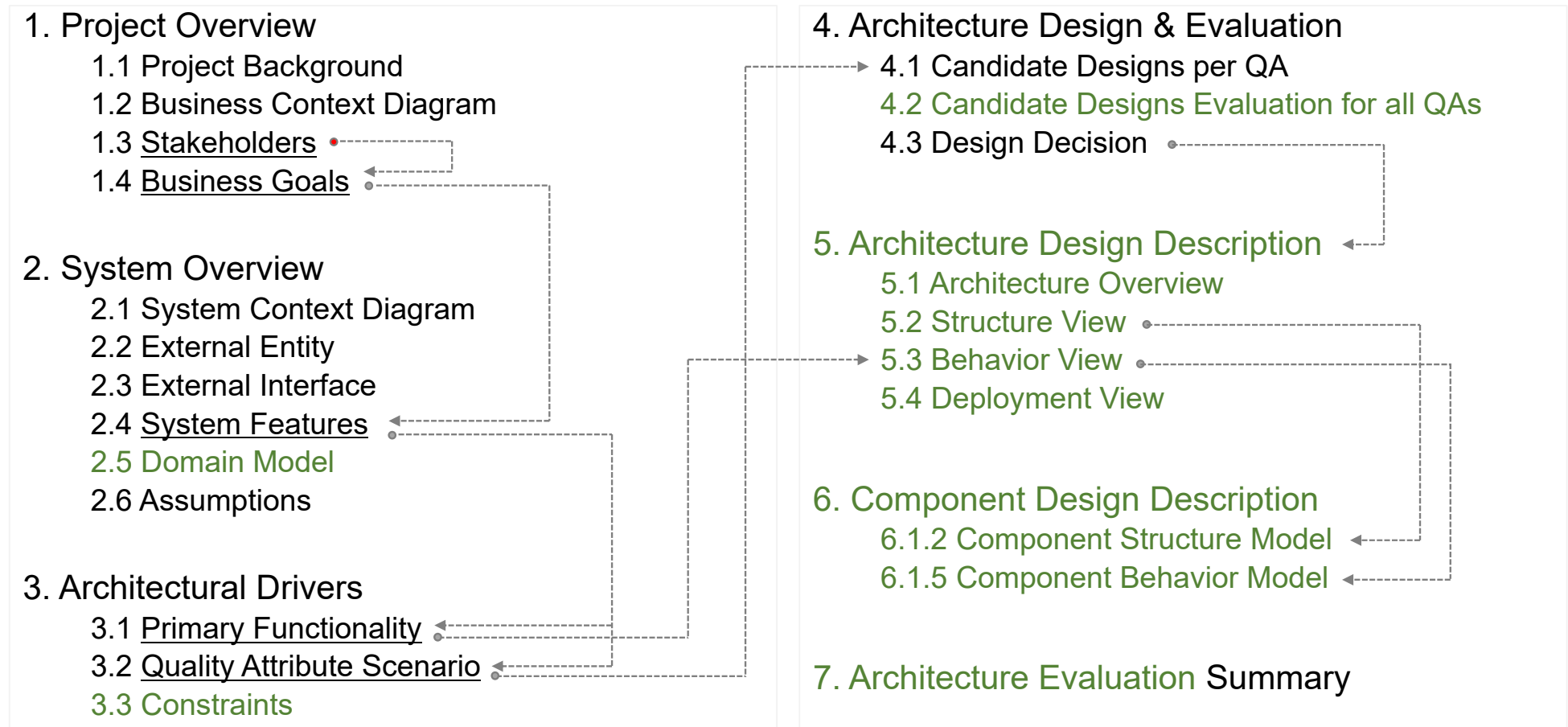
- A **Full-Traceability** is required.
 - **WHO** : Stakeholders (누가)
 - **WHAT** : Concerns (어떤 관심사를)
 - **HOW** : Architecture Views (어떻게 반영했고)
 - **WHY** : Architecture Rationale (왜)



Organizing Our Architecture Description (MINI Version)

1. Project Overview
 - 1.1 Project Background
 - 1.2 Stakeholders
 - 1.3 Business Goals
2. System Overview
 - 2.1 System Context Diagram
 - 2.2 System Features
 - 2.3 Assumptions about the System
3. Architectural Drivers
 - 3.1 Primary Functionality
 - 3.2 Quality Attribute Scenario
4. Architecture Design
 - 4.1 Candidate Designs per QA
 - 4.2 Candidate Decisions per QA
 - 4.3 Design Decision (Overview)
5. Summary

Organizing Our Architecture Description (Full Version)

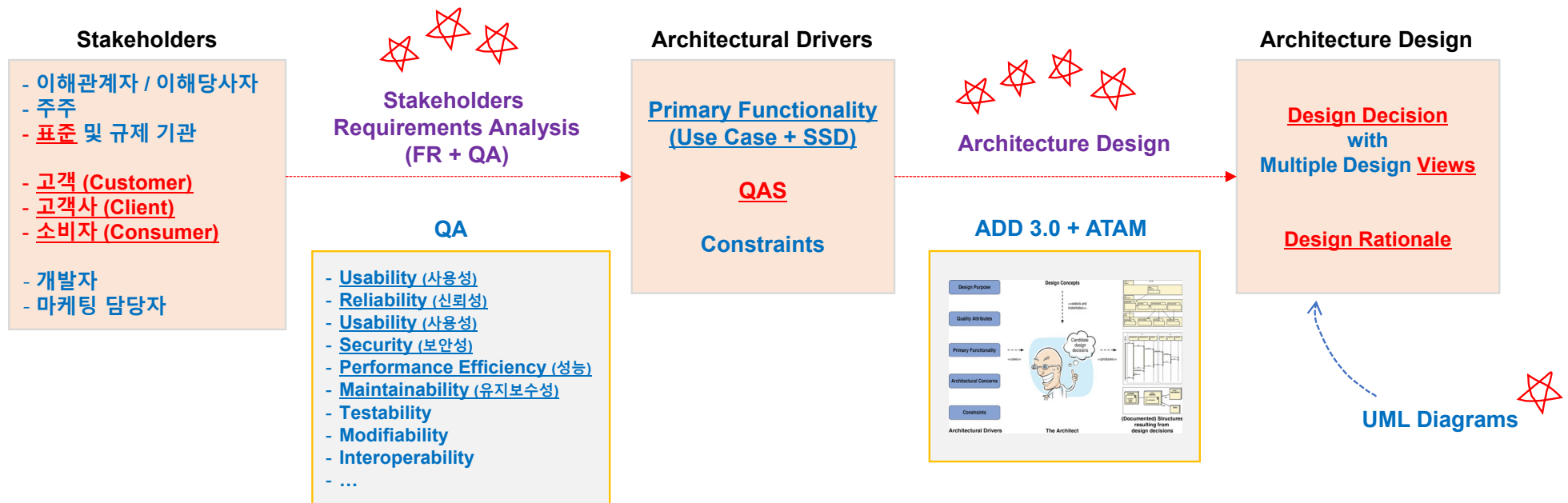


* Green chapters are skipped in MINI CEP.

What Matters Most in Architecture Design?

- We should be able to check this question with full traceability.

How well does our architecture design address all stakeholders' concerns?



Software Architecture Design in Our Architecture Description

Where We Are Now

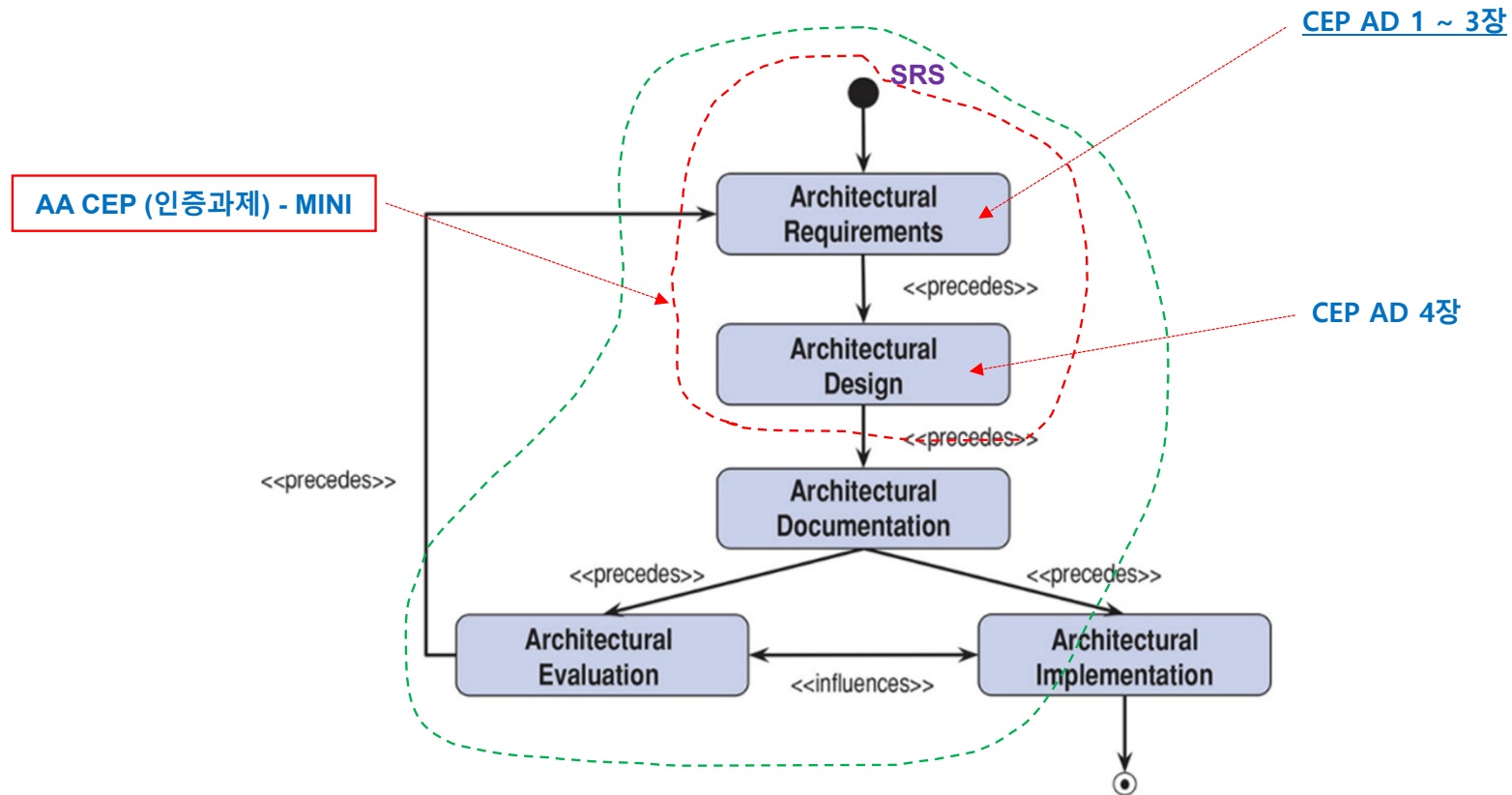


FIGURE 1.1 Software architecture life-cycle activities

1. Project Overview

Where We are Now in AD

1. Project Overview

- 1.1 Project Background
- 1.2 Stakeholders
- 1.3 Business Goals

2. System Overview

- 2.1 System Context Diagram
- 2.2 System Features
- 2.3 Assumptions about the System

3. Architectural Drivers

- 3.1 Primary Functionality
- 3.2 Quality Attribute Scenario

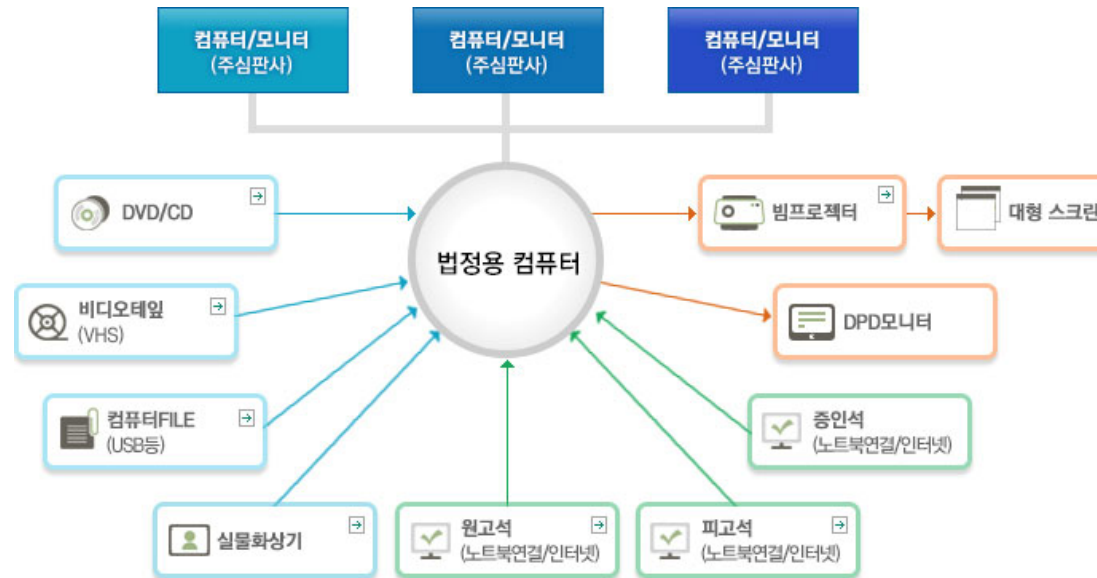
4. Architecture Design

- 4.1 Candidate Designs per QA
- 4.2 Candidate Decisions per QA
- 4.3 Design Decision (Overview)

5. Summary

1.1 Project Background

- Describes the project, its purpose, and scope
 - Background information on the system domain to help stakeholders understand the project and the system



정부 표준 전자법정



- + 영상 재판
- + 온라인 생중계
- + 방청객에게 실시간 자료 제공 (Mobile)

1.2 Stakeholders

- **Stakeholder**

- a person, group, or entity with an interest in or concerns about the realization of the architecture, or
- a party having a right, share or claim in a system or in its possession of characteristics that meet that party's needs and expectations.
- All entities that you identified in the business context diagram (Skipped in MINI CEP)

- Example :

Stakeholder	Description
고객(사용자)	설명: 심부름 플랫폼 서비스를 이용하여 심부름을 의뢰하려는 사용자 관심사: 특정한 업무를 수행하기에 시간적, 거리적 한계가 있을 때, 요금 지불을 통해 직접 수행 불가능한 업무를 대행(심부름)할 대리자(헬퍼)를 찾고 싶음. 심부름에 대하여 합리적인 요금을 지불하기 원함. 대행 요청할 업무의 특성(카테고리)에 따라 헬퍼를 쉽고 빠르게 찾기를 원하며, 대리자가 수행하는 심부름의 과정부터 결과까지 편리하게 관리/감독하기를 기대함. 심부름의 결과 수준이 높기를 기대함.

Brainstorming

1.3 Business Goals

- **Business goals** of stakeholders are the **primary influencing factors** on the architecture.
 - Business goals = Mission objectives
 - “What are our ambitions about market share for this product?”
 - “How could the architecture contribute to meet them?”
 - Should be captured explicitly because they often imply ASRs.
 - Examples :
 - Owner : “His family's stock in the company will rise by 5%.”
 - Portfolio manager : “The company will make the portfolio 30% more profitable.”
 - Project manager : “Customer satisfaction will rise by 10%.”

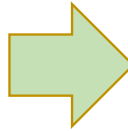


Stakeholder	Business Goal		
	ID	Statement	Importance
운전자	BG-01	운전자는 입·출차 및 주차 요금 정산 등 전반적인 주차장 이용에서 더 만족스러운 주차 경험을 기대함. - 주차장을 찾고, 입·출차 및 정산하는 <u>전체 시간을 20% 단축</u> 하기를 원함.	상

Stakeholders : Examples

Stakeholder	Description
판사	설명: 디지털 법정 시스템을 이용하여 현장/영상 동시 재판을 진행하고 판결하려는 판사 관심사: 현장/영상재판 모두 지원이 가능한 법정 시스템을 원하며, 영상재판 참여자와 현장재판을 하듯이 문제없는 재판 진행이 가능하기를 기대함
재판 참여자	설명: 디지털 법정 시스템을 이용하여 현장 혹은 영상 재판에 참여하는 재판 관계인(검사, 변호사, 원고, 피고, 증인) 관심사: - 법원 출석이 어려운 경우 화상프로그램을 통한 재판 참여가 가능하기를 원함 - 재판 관련 자료를 간편하게 제출하고 공유할 수 있기를 원하며, 편리하고 신속하게 재판이 진행되기를 원함 - 청각 혹은 발화에 문제가 있더라도 문제없이 재판 참여가 가능하기를 원함 - 재판 과정에서 증인의 신분을 보호하는 기능이 있기를 원함
방청객	설명: 공개재판을 현장에서 관찰하는 방청객 관심사: 현재 방청하는 재판 관련 멀티미디어 자료를 모바일 애플리케이션으로 실시간 확인하기를 원함
참여 사무관	설명: 법정 문서 및 기록을 관리하고 재판 과정에서 판사를 보조하는 참여 사무관 관심사: - 재판 관련 문서 및 자료를 체계적으로 관리하고, 판사에게 필요한 정보를 편리하고 빠르게 제공하여 재판과정을 지원하기를 원함 - 재판이 종료되면 현장/영상 재판 과정을 포함한 모든 재판 자료가 빠르고 쉽게 법원에 저장되는 편리한 법정 시스템을 원함

Stakeholders



Stakeholder	Business Goal		
	ID	Statement	I
판사	BG-01	현장/영상재판 모두 지원이 가능한 법정 시스템을 원하며, 영상재판 참여자와 현장재판을 하듯이 문제없는 재판 진행이 가능하기를 기대함 - 문제없이 영상재판을 진행하고, <u>영상 및 오디오 전달 지연 시간이 5초 이내</u> 이기를 기대함	상
판사	BG-02	현장/영상재판 모두 지원이 가능한 법정 시스템을 원하며, 영상재판 참여자와 현장재판을 하듯이 문제없는 재판 진행이 가능하기를 기대함 - 문제없이 영상재판을 진행하고, 시스템의 문제로 재판이 중단되더라도 <u>30분 이내에 복구되어</u> 재판이 재진행되기를 원함	상
재판 참여자	BG-03	법원 출석이 <u>어려운 경우</u> 화상프로그램을 통한 재판 참여가 가능하기를 원함 - 개인 사정으로 인한 재판 연기 혹은 불출석 횟수를 기존 대비 50% 이상 낮추기를 기대함	상
재판 참여자	BG-04	재판 관련 자료를 간편하게 제출하고 공유할 수 있기를 원하며, 편리하고 신속하게 재판이 진행되기를 원함 - 재판 업무에 소요되는 시간을 <u>기존대비 20% 감소</u> 하기를 기대함	중
재판 참여자	BG-05	재판 과정에서 증인의 신분을 보호하는 기능이 있기를 원함 - 재판에 참여한 증인은 특정인(원고 혹은 피고)의 PC 화면과 법정 스크린에서 블라인드로 표시되기를 기대함	중
재판 참여자	BG-06	청각 혹은 발화에 문제가 있더라도 문제없이 재판 참여가 가능하기를 원함 - 재판 과정에서 음성을 텍스트로, 텍스트를 음성으로 변환해주는 기능 제공을 기대함	하

Business Goals

~ BG-17

2. System Overview

Where We are Now in AD

1. Project Overview

- 1.1 Project Background
- 1.2 Stakeholders
- 1.3 Business Goals

2. System Overview

- 2.1 System Context Diagram
- 2.2 System Features
- 2.3 Assumptions about the System

3. Architectural Drivers

- 3.1 Primary Functionality
- 3.2 Quality Attribute Scenario

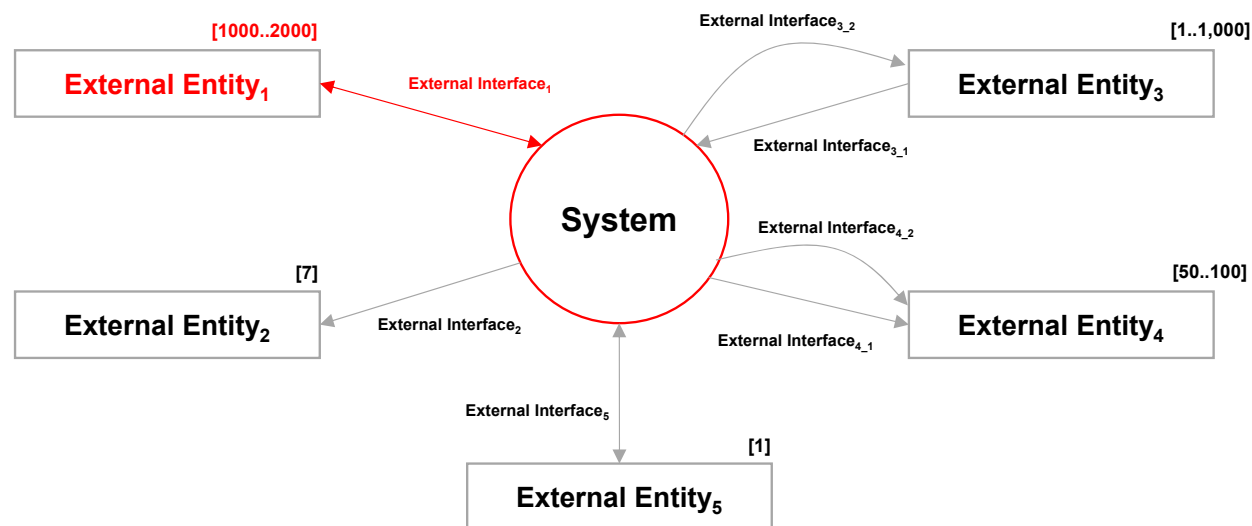
4. Architecture Design

- 4.1 Candidate Designs per QA
- 4.2 Candidate Decisions per QA
- 4.3 Design Decision (Overview)

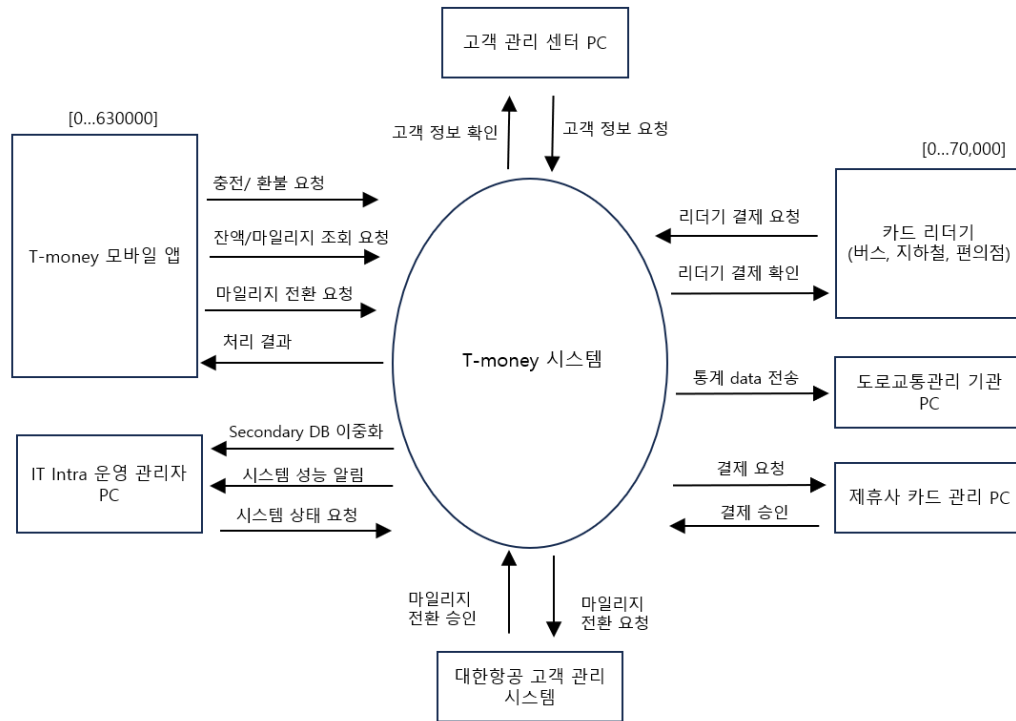
5. Summary

2.1 System Context Diagram

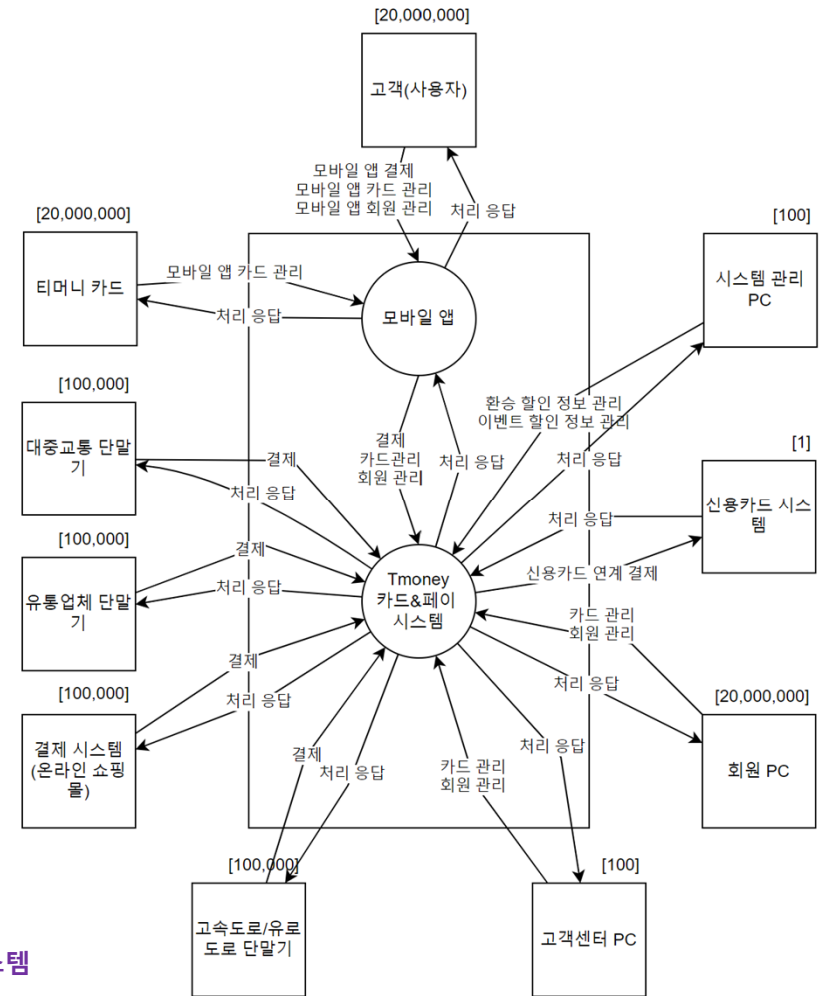
- The system tries to implement the **business goal** within **system context** (boundary).
 - A good system context model is an essential part of an effective architecture document.
- **System Context Diagram** defines
 - What is in scope? → **System Features** & Domain Model (Skipped in MINI CEP)
 - What is out of scope? → **External Entities**
 - How the system related to its environment? → **External Interfaces**



System Context Diagram : Examples



T-money 카드 & 페이 시스템

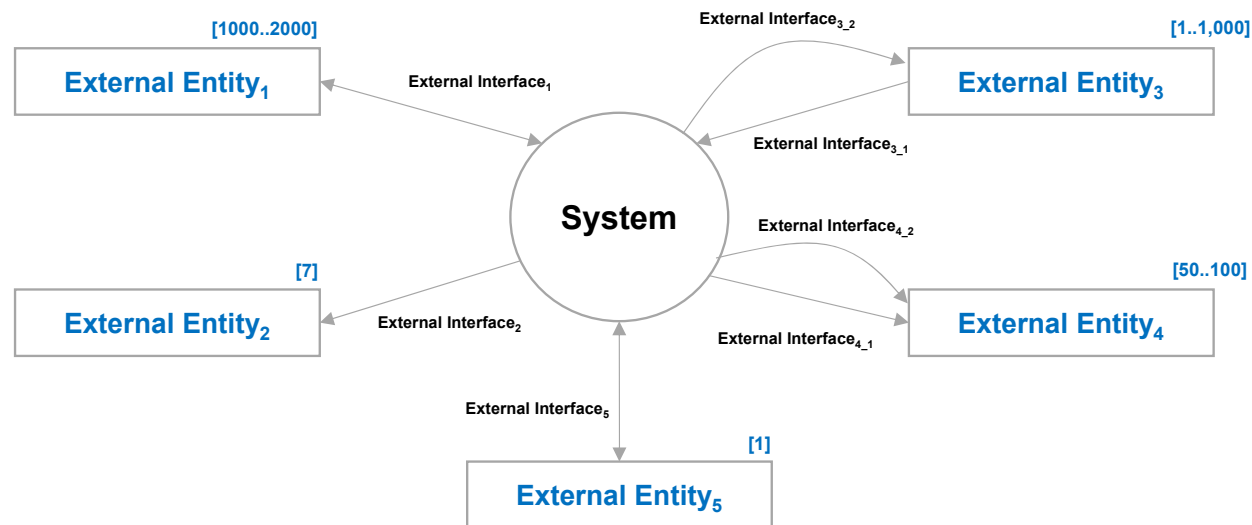


Guidelines for System Context Diagram

- NOT disclose any architecture detail about the system
 - It just appears as an **undecomposed block**.
 - In practice, it may show some internal (predefined) structure of the system being put in context.
- NOT show any temporal information
 - E.g., order of interactions, **flow of data**, **flow of control**, data flow graph
- NOT show the conditions under which data is transferred, stimuli fired, and messages transmitted
 - Each interface needs to be **just “assigned” to** one of the system’s architecture elements.

External Entity

- An **external entity** is any person, system, or device with which the system directly interacts.
 - **User** : User, a class of user, or some other person or role
 - **External system** : Another system that runs in another organization
 - **Internal system** : Another system that runs in the same organization as the system being modeled
 - **Gateway component** : Gateway or other implementation component that has the effect of hiding other systems (internal or external)
 - **Data store** : Data store that is external to the system (e.g., a shared database, data warehouse)



External Entity : Examples

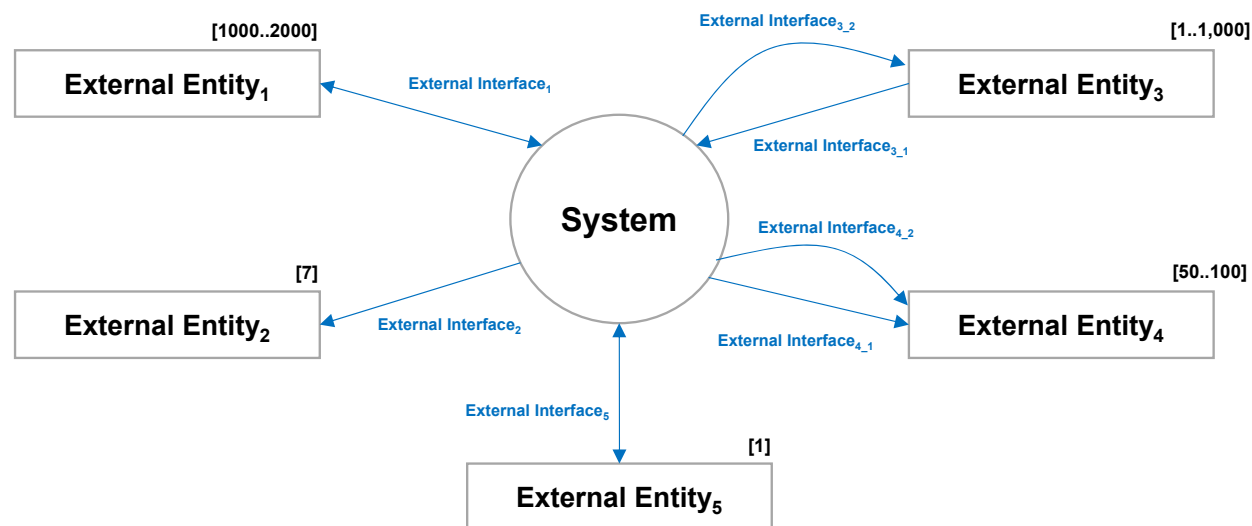
- All quality attributes that may affect the system architecture shall be described in detail.
 - All reasonable assumptions on H/W and systems

Assumptions
with Rationale

플랫폼 운영자	유형: 사용자 숙련도: 플랫폼 서비스를 통한 비즈니스 운영에 대한 전문성을 보유하고 있음. 플랫폼 서비스 운영을 위한 시스템 운용이 능숙함. 핵심 기대 사항: 시스템을 통하여 이용자들의 이용 현황을 파악하고, 관리하기를 원함. 헬퍼 중 범죄 사실이 발생하거나, 불법적인 거래가 발생할 경우 빠른 시간 내에 파악하고 조치하기를 원함.
결제 대행 시스템	유형: 시스템 역할: 시스템에서 주차 요금 결제 요청 시 각종 카드 및 간편 결제 시스템에 결제 및 지불을 대행해주는 외부 시스템 관련 Stakeholder: 결제 대행 업체 시스템 사양: Cloud 서비스 운영 가능한 서버 시스템의 품질 수준: - 가용성: 99.9% (카드 회사, 간편 결제 회사 별 시스템 점검 시간 존재) - 전자금융거래의 안전성과 신뢰성: 금융감독위원회가 정하는 기준을 준수
입구/출구 안내판	유형: 장치 역할: 주차장 출입구에 설치되어 시스템이 안내하고자 하는 내용을 LED로 출력 관련 Stakeholder: 관제센터, SW개발회사 장치 사양: - Control H/W: 아두이노 수준 - LED Display board (문자 표기: 한글/영문/숫자 출력 가능) - Connectivity: Ethernet, Wi-Fi 장치의 품질 수준: - 요청 수신 0.1초 이내에 안내판에 출력

External Interface

- **All interfaces** between the system and each external entity should be identified.
 - **Data** provider or consumer : The external entity supplies data directly to the system or receives data directly from it.
 - **Event** provider or consumer : The external entity publishes events that this system wishes to be notified of, or this system publishes events that the external entity wishes to be notified.
 - **Service** provider or consumer : The external entity is requested to perform some action by the system or requests some action of the system, and the system may return data and/or status information in response to the request.



External Interfaces : Examples

- The quality and characteristics of external interface may have a significant effect on the architecture of the system.
 - **Data** : the content, scope, and meaning of the data to be transferred
 - **Event** : events of interest, their meaning and content, and the volume and timing of their occurrence
 - **Service** : Syntax of the request(name and any parameters), the actions to be taken, any data to be returned, any ack, status, or error information that may be returned, any exception actions to be taken by either side

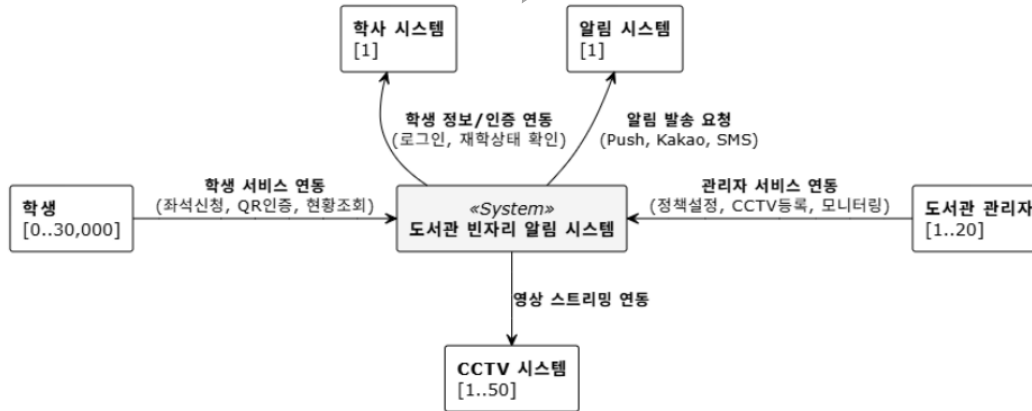
get parking status	역할: 관제 센터 직원이 주차 관리 시스템으로 주차장 운영 현황을 요청 User interface: Console (Web UI) System interface: HTTPS 특성: - 사용자 요청 빈도는 일 1회 정도로 낮을 것으로 예상 - 1회 요청 당 10만개 주차장 현황을 조회하므로 시스템 부하 고려 필요
Track errand	역할: 고객이 심부름 내역 및 결과(이미지, 동영상)를 조회 User Interface: Mobile app UI System Interface: HTTPS 특성: - 수시 호출 가능함 - 헬퍼의 DAU(Daily Active User) 22,000명이 1일 심부름 1건 수주한다고 가정하였을 때, 약 22,000건의 심부름 수행 가능 - 하루 다운로드 22,000 건 * (최대 10장 * 1MB + 영상 2건 * 10MB) = 660GB - 평균 60Mbps (약 7.65mb/s) 다운로드 부하 - 낮 피크타임을 가정하면 100Mbps 이상 다운로드 부하 가능

*Assumptions
with Rationale*

DS Examples

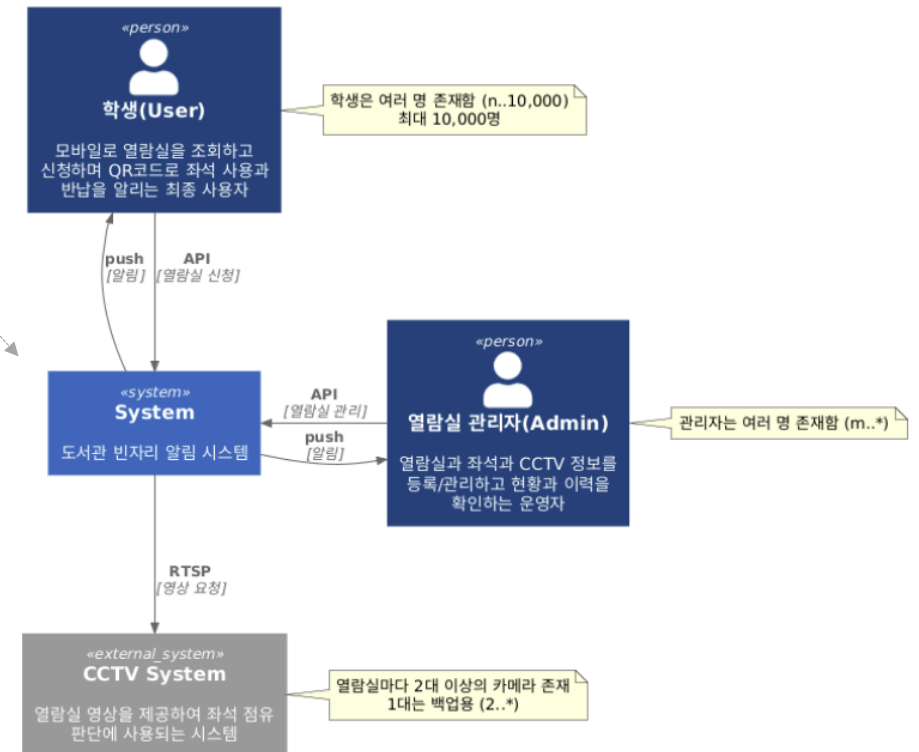
개발하려는 시스템의 범위/속성이 명확하게 드러나야 해요.
연결되는 외부의 모든 것들이 빠짐없이 포함되어야 해요.

2.1. System Context Diagram



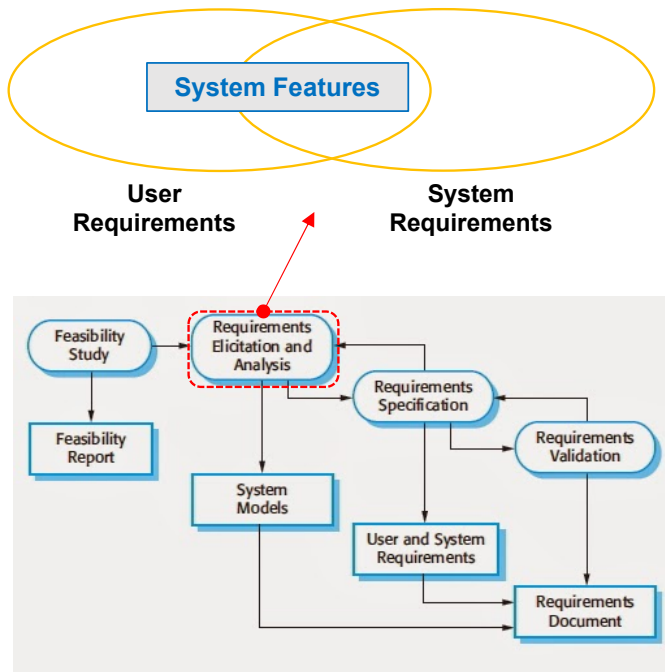
2.1. System Context Diagram

System Context Diagram - Library Vacancy Alert Backend (LBAS)



2.2 System Features

- **System features** are the high-level capabilities of the system that are required to deliver by us.
 - **System features** are refined from **Business goals** which we must realize.
 - **System requirements (FR/QAR)** are derived from system features.
 - Includes all functional and non-functional features.



Requirements Engineering Process

ID	Title	Description	I	Related Business Goal ID
SF-01	타 자판기 재고 조회	사용자가 현재 자판기에 없는 상품에 대해, 재고가 있는 다른 자판기의 위치 정보를 쉽게 확인할 수 있도록 직관적이고 간단한 UI를 제공한다.	상	BG-01, BG-24
SF-02	타 자판기 상품 예약	사용자가 원하는 상품이 있는 특정 자판기를 선택하여 해당 상품을 미리 예약할 수 있다.	상	BG-02, BG-15, BG-24
SF-03	다양한 방식의 선결제 지원	향후 모바일 간편 결제 등 다양한 결제 플랫폼을 쉽게 적용할 수 있도록 시스템을 설계한다.	상	BG-03, BG-18, BG-27
SF-04	예약 상품 간편 픽업	사용자가 선결제 후 예약된 자판기에 방문했을 때, 간단한 인증 절차를 통해 예약한 상품을 신속하고 편리하게 수령할 수 있다.	상	BG-04
SF-05	실시간 통합 자판기 모니터링 (시스템 관리자)	시스템 관리자가 중앙 관리 시스템을 통해 전체 자판기의 운영 상태(네트워크, 재고, 장애 등)를 실시간으로 파악하고 관리할 수 있다.	중	BG-05, BG-11
SF-06	신속한 장애 감지 및 알림 (시스템 관리자)	시스템 또는 네트워크 장애 발생 시, 시스템 관리자가 이를 신속하게 인지하고 대응할 수 있도록 중앙 시스템을 통한 알림 기능을 제공한다.	중	BG-06

2.3 Assumptions about the System

- **Several assumptions** that are not specified in the AA CEP Guide but are necessary for the system architecture design

- Assuming all functionalities identified in the AA CEP Guide should be implemented.

- Example :

- 1) 헬퍼가 프로필을 등록할 때 성별 외에도 수행 가능한 전문 분야가 있는 경우, 헬퍼 프로필에 기재하도록 하였다. (카테고리 기준, 배달/장보기/설치/운반/청소/돌봄 등)
- 2) 심부름 수행 도중, 고객이나 헬퍼의 스마트폰이 영원한 파괴/방전/통신불가 상태에 진입하지 않는다고 가정한다. 즉, 심부름 수행 중에 시스템과 스마트폰 사이의 복구 불가능한 통신 두절 상태는 발생하지 않는다.
- 3) 외부 시스템인 경찰청 시스템, 은행 시스템 모두 영원한 파괴/방전/통신불가 상태에 진입하지 않는다고 가정한다.
- 4) 사용자가 채팅으로 전달할 수 있는 메시지 길이는 한글 기준 1,000자(최대 3Kbyte) 이하라고 가정한다.
- 5) 헬퍼가 심부름 진행 현황 보고 용으로 업로드 할 수 있는 미디어 파일의 제한은, 이미지파일 1개당 최대 1MB, 최대 10장 / 영상 파일 최대 10MB 최대 2건으로 제한한다.

3. Architectural Drivers

Where We are Now in AD

1. Project Overview

- 1.1 Project Background
- 1.2 Stakeholders
- 1.3 Business Goals

2. System Overview

- 2.1 System Context Diagram
- 2.2 System Features
- 2.3 Assumptions about the System

3. Architectural Drivers

- 3.1 Primary Functionality
- 3.2 Quality Attribute Scenario

4. Architecture Design

- 4.1 Candidate Designs per QA
- 4.2 Candidate Decisions per QA
- 4.3 Design Decision (Overview)

5. Summary

Architectural Drivers

- **Architectural Drivers (AD)** are the key requirements that are most likely to affect the fundamental structure of the implementation.
 - AD will determine the structure (architecture) of the system.
 - Uncovering ADs as early as possible is critical to the stable architecture design.
 - ADs are a part of requirements, called by **ASR (Architecturally Significant Requirements)**.
- **5 Architectural Drivers** by CMU SEI
 - Primary Functionality
 - Quality Attribute
 - Design Purpose
 - Architectural Concerns
 - Constraints
- **Our ADs** consist of
 - **Primary Functionality**
 - **Quality Attribute Scenario (QAS)**
 - **Constraints**

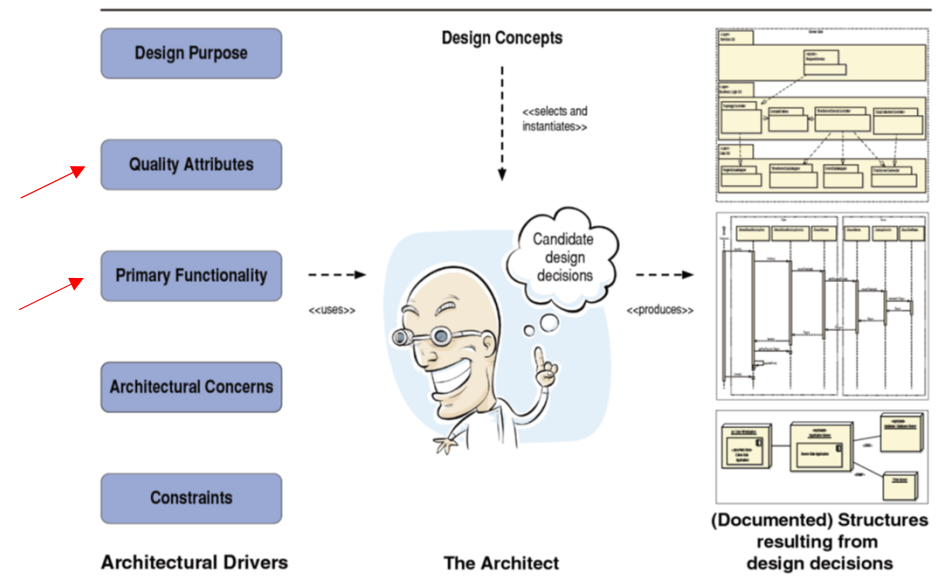
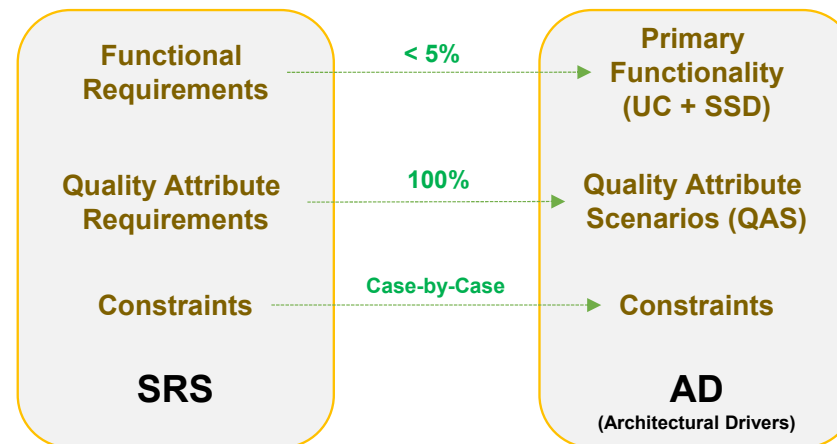


FIGURE 2.1 Overview of the architecture design activity
(Architect image © Brett Lamb | Dreamstime.com)

ASR to Architectural Drivers

- **Mapping** ASR_(Architecturally Significant Requirements) in SRS to AD_(Architectural Drivers) in AD_(Architecture Description)
 - **Functionality** → (Select & Refine) → **Primary Functionality** (Use Case + SSD_(System Sequence Diagram))
 - **Quality Attribute** → (QAW) → **QAS** (Quality Attribute Scenario)
 - Constraints → Constraints *



* It's important, but out of the scope of the AA CEP for now.

3.1 Primary Functionality

- **Functionality** is the ability of the system to do the work for which it was intended.
 - Software architecture does not normally influence functionality.
 - Functionality can often be satisfied with any predefined software architecture.
- **Primary functionality** is the **NEW** functionality that is critical to achieve the business goal.
 - Implying a high level of technical difficulty
 - Requiring the interaction of many architectural elements
 - Some quality attributes are directly connected to the primary functionality in the system.
 - **Derived attributes**
 - Approximately 10 % of use cases (user stories) in SRS
- **Functionality** → (Select & Refine) → **Primary Functionality (Use Case + SSD)**

Our Use Case Format

- **The Casual format** of use cases
 - The system is always considered as a [black box](#).
 - No design/implementation details are considered.
- **Example : LMS** (Library Management System)

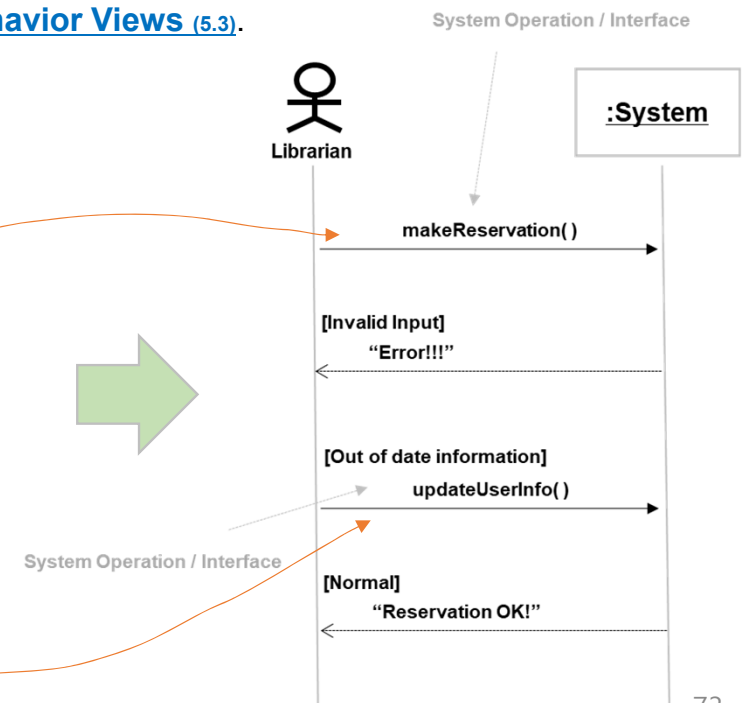


Use Case	1. Make Reservation
Actor	Librarian
Description	A librarian requests LMS to make a reservation for a book.
Stakeholders	User, Librarian, System Manager
Preconditions	Borrower should have an id_card.
Main Scenario	(A) : Actor, (S) : System 1. (A) A librarian requests the reservation of a title 2. (S) Check if a corresponding title exists 3. (S) Check if a corresponding borrower exists 4. (S) Create a reservation information
Alternative Scenario	[Out of date information] 3: (S) If the borrower's information is out of date, request for the update. (A) A librarian updates up-to-date information of the borrower. [Invalid Input] 1~3: If invalid reservation information is entered, indicate an error.

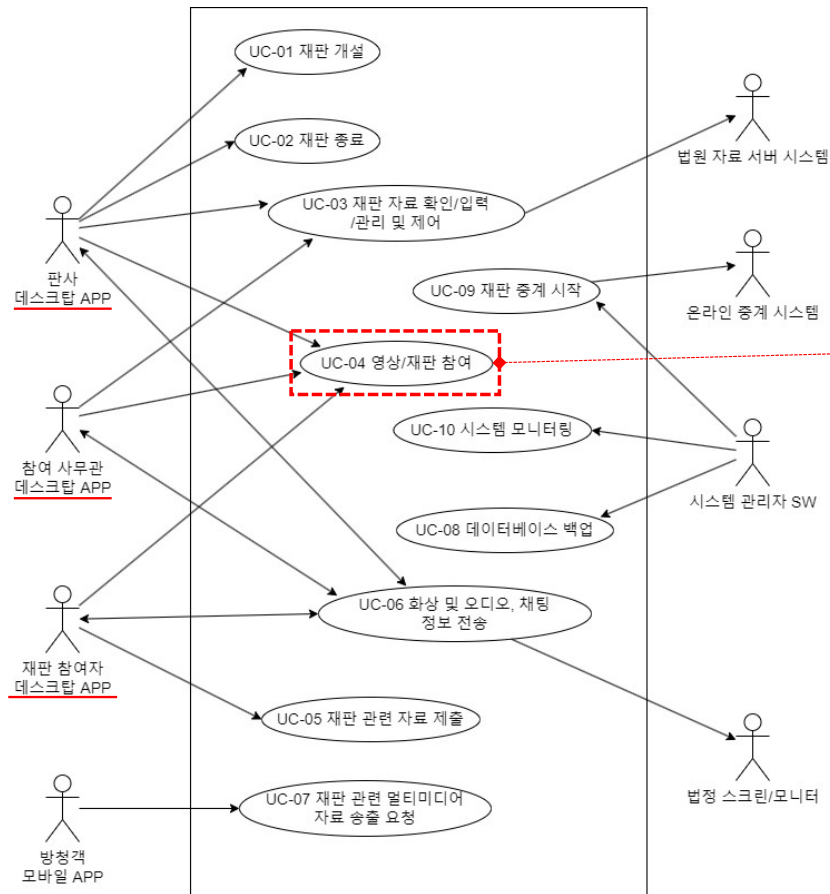
System Sequence Diagram

- A picture that shows the events that external actors generate, their order, and inter-system events, for each scenario of a use case.
 - Depicting system behavior in terms of **what the system does**, not how it does it
 - Used as an input to system design → **System operations / System interfaces**
 - One SSD for each use case
 - The identified system operations/interfaces will be linked to Behavior Views (5.3).
 - Keeping traceability is important.

Use Case	1. Make Reservation
Actor	Librarian
Description	A librarian requests LMS to make a reservation for a book.
Stakeholders	User, Librarian, System Manager
Preconditions	Borrower should have an id_card.
Main Scenario	(A) : Actor, (S) : System 1. (A) A librarian requests the reservation of a title 2. (S) Check if a corresponding title exists 3. (S) Check if a corresponding borrower exists 4. (S) Create a reservation information
Alternative Scenario	[Out of date information] 3: (S) If the borrower's information is out of date, request for the update. (A) A librarian updates up-to-date information of the borrower. [Invalid Input] 1~3: If invalid reservation information is entered, indicate an error.



UCD : Examples



Use Case Diagram

ID	Title	Summary of Description	Priority		System Feature ID	ASR?
			I	D		
UC-01	재판 개설	판사가 데스크탑 APP을 이용하여 재판 일정을 개설한다.	중	하	SF-03	X
UC-02	재판 종료	판사가 데스크탑 APP을 이용하여 재판을 종료한다.	중	하	SF-06	X
UC-03	재판 자료 확인/입력/관리 및 제어	판사 또는 참여 사무관이 데스크탑 APP을 이용하여 법원 자료 서버의 재판 자료를 확인/입력/관리 및 제어한다.	중	중	SF-07	O
UC-04	영상재판 참여	판사 또는 재판 참여자 또는 참여 사무관이 데스크탑 APP을 이용하여 사건번호에 맞는 영상재판에 참여한다.	상	중	SF-05	X
UC-05	재판 관련 자료 제출	재판 참여자가 데스크탑 APP을 이용하여 재판 관련 자료를 제출한다.	상	상	SF-09	X
UC-06	화상 및 오디오, 채팅 정보 전송	판사 또는 재판 참여자 또는 참여 사무관이 데스크탑 APP을 이용하여 영상재판을 진행한다.	상	상	SF-08	X
UC-07	재판 관련 멀티미디어 자료 송출	방청객이 모바일 APP을 이용하여 재판 관련 멀티미디어 자료를 실시간으로 확인한다.	중	중	SF-15	O
UC-08	데이터베이스 백업	시스템 관리자가 관리자 SW를 이용하여 데이터베이스를 백업한다.	상	중	SF-21	X
UC-09	재판 중계	온라인 중계 시스템을 이용하여 재판 과정을 중계한다.	중	중	SF-17	O
UC-10	시스템 모니터링	시스템 관리자가 관리자 SW를 이용하여 시스템을 모니터링 한다.	중	중	SF-20	X

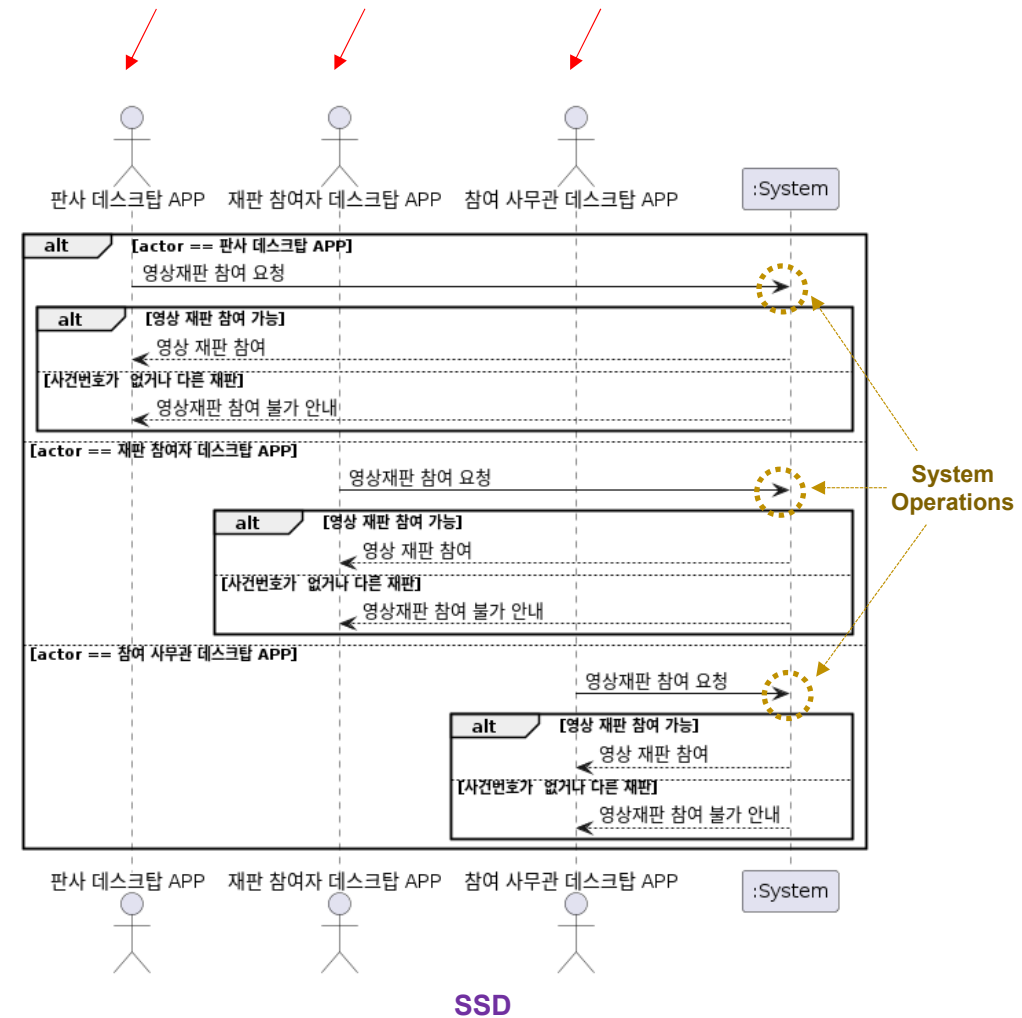
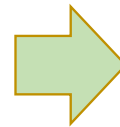
Use Case

UCM (UC+SSD) : Examples

UC-04	영상재판 참여	판사 또는 재판 참여자 또는 참여 사무관이 이 데스크탑 APP을 이용하여 사건번호에 맞는 영상재판에 참여한다.	상	중	SF-05	X
-------	---------	---	---	---	-------	---

Use Case	영상재판 참여
Actor	판사 데스크탑 APP, 재판 참여자 데스크탑 APP, 참여 사무관 데스크탑 APP
Description	판사 또는 재판 참여자 또는 참여 사무관이 데스크탑 APP 을 통해 영상재판 참여를 요청한다.
Stakeholders	판사, 재판 참여자
Preconditions	판사 또는 재판 참여자 또는 참여 사무관이 데스크탑 APP 에서 로그인한 상태
Main Scenario	(A₁) : 판사 데스크탑 APP, (A₂) 재판 참여자 데스크탑 APP, (A₃) : 참여 사무관 데스크탑 APP, (S) : System 1. (A₁) 판사 데스크탑 APP 또는 (A₂) 재판 참여자 데스크탑 APP 또는 (A₃) 참여 사무관 데스크탑 APP 이 (S) System 에게 영상 재판 참여를 요청한다. 2. (S) System 은 (A₁) 판사 데스크탑 APP 또는 (A₂) 재판 참여자 데스크탑 APP 또는 (A₃) 참여 사무관 데스크탑 APP 에게 재판 참여 결과를 전달한다.
Alternative Scenario	[사건번호가 없거나 다른 재판] 2.a. 시스템은 영상재판 참여 불가 안내를 하고, 본 유스케이스를 종료한다.

Use Case

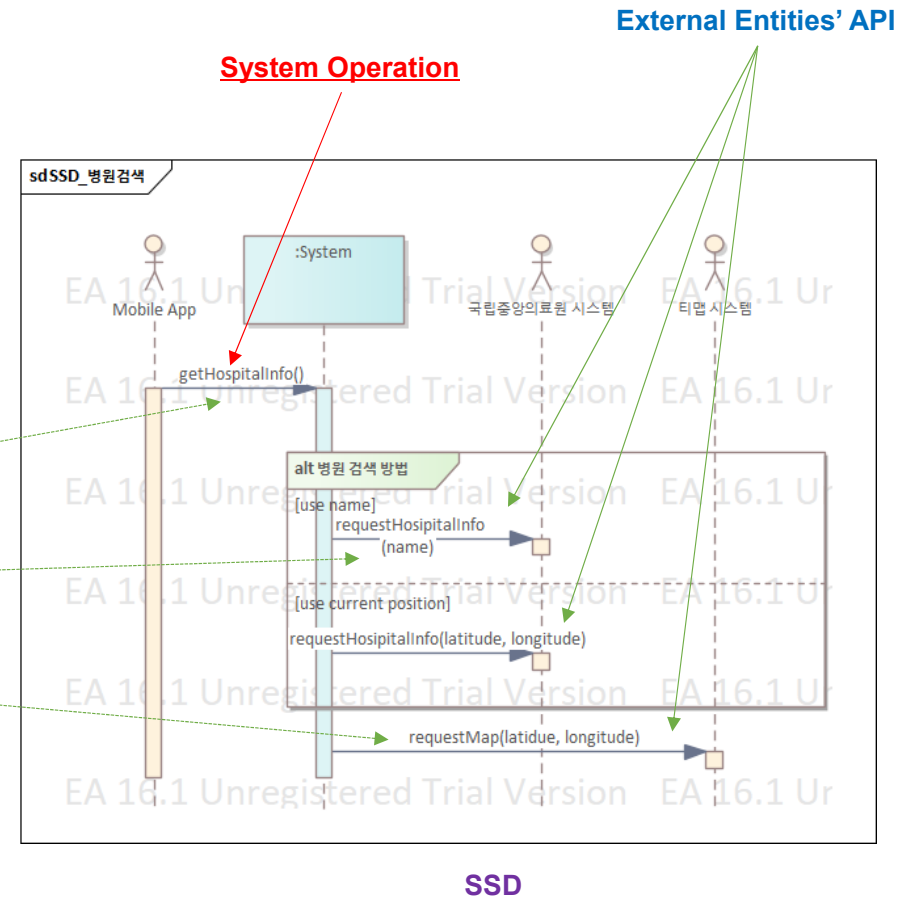


SSD

UCM (UC+SSD) : Examples

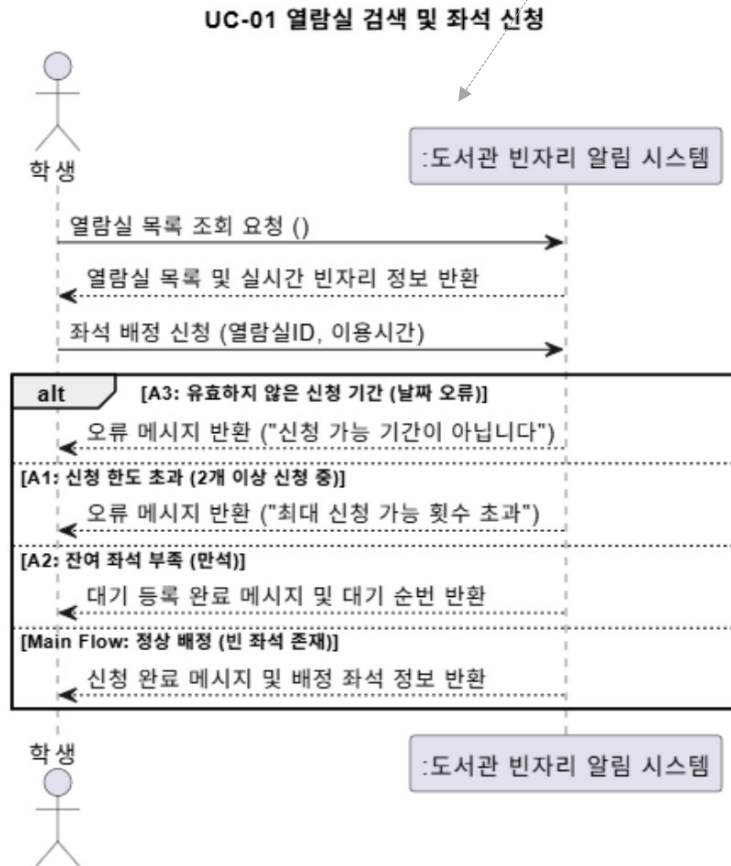
Use Case	병원 검색
Actor	Mobile App, 국립중앙의료원 시스템, 티맵 시스템
Description	Mobile App은 병원 검색을 요청하고 System은 검색 결과를 Mobile App으로 전달한다.
Stakeholders	환자, 국립중앙의료원 공공데이터 관리자, 티맵모빌리티 관리자
Preconditions	로그인 상태 위치 정보 권한을 부여한 상태 스마트폰 네트워크 사용 가능한 상태
Main Scenario	(A₁) : Mobile App, (A₂) : 국립중앙의료원 시스템, (A₃) : 티맵 시스템, (S) : System 1. (A₁) 검색하고 싶은 병원을 입력하여 검색 요청 2. (S) 검색 요청된 결과 확인 3. (S) 국립중앙의료원 시스템에 검색 요청 4. (A₂) 검색 결과 반환 5. (S) 티맵 시스템에 지도 이미지 요청 6. (A₃) 지도 이미지 반환 7. (S) Mobile App에 검색 결과 전달
Alternative Scenario	[현재 위치 기반 검색을 요청한 경우] 2. (S) 현재 위치 정보를 사용

Use Case

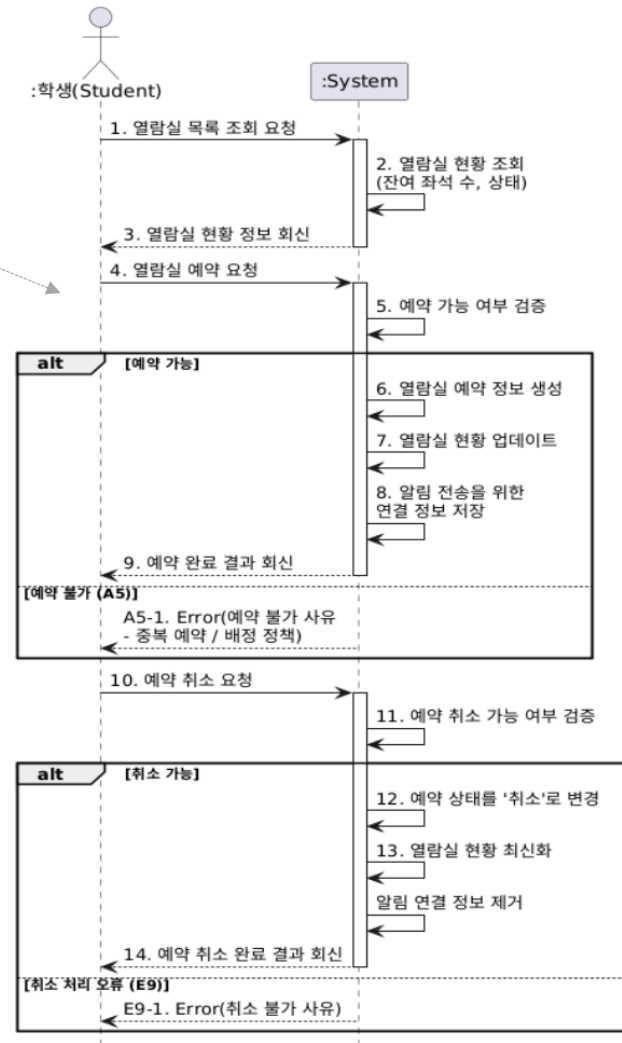


DS Examples

SSD를 그리는 목적에 맞게 그려야 해요.
UC에 있는 시나리오를 그대로 옮겨오면 안돼요.



UC-01 : 열람실 예약 관리 - System Sequence Diagram



2. Quality Attributes Scenarios (QAS)

- **Quality Attribute (Requirement)** is a measurable or testable properties of a system, that used to indicate how well the system satisfies the needs of its stakeholders.
- **QA** shapes the architecture the most significantly.
 - Functionality does not determine architecture.
 - We can design our systems as structured sets of cooperating architectural elements (layers, components, classes, databases, apps, threads, peers, tiers, and so on) to support a variety of other purposes (→ QA).
 - Systems are frequently redesigned not because they are functionally deficient, but because
 - *“They are difficult to maintain, port, or scale.” → Maintainability, Portability, Scalability, Modifiability, Extensibility*
 - *“They are too slow.” → Performance Efficiency*
 - *“They have been compromised by hackers.” → Security*

QAS (Quality Attribute Scenario)

- **QAS** is a short description of how a system is required to respond to some stimulus.
 - achieve in response to a specific stimulus
 - Describing the system's response to some stimulus
 - Specifying the response measure that you would like to
- **QAS** is a system test case that checks “*whether the system meets a required quantified level of quality.*”

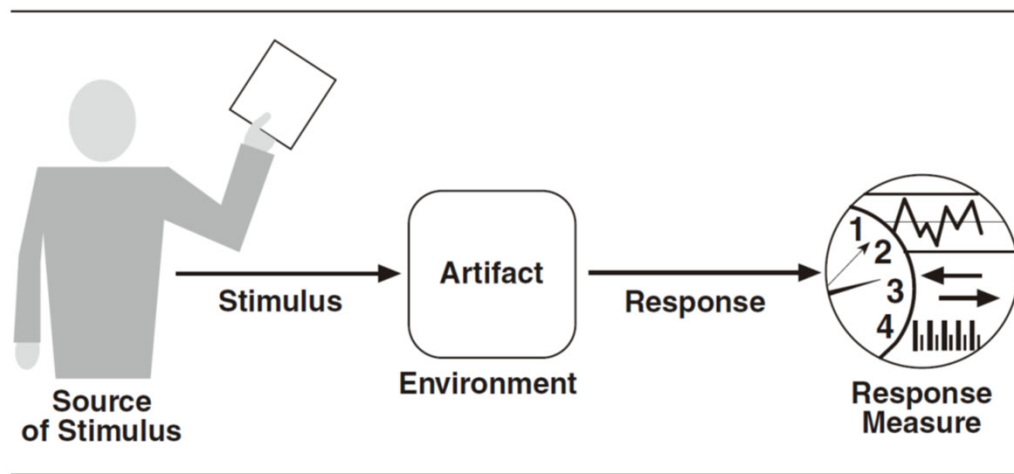


FIGURE 2.2 The six parts of a quality attribute scenario

The QAS Template

- The QAS table should have the same information with the QAS figure.

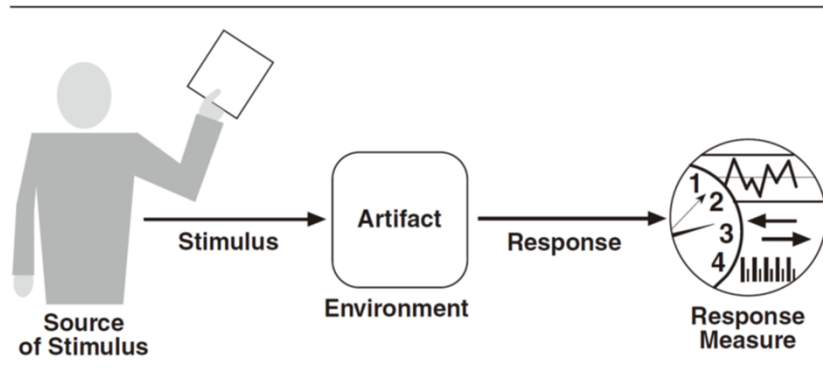
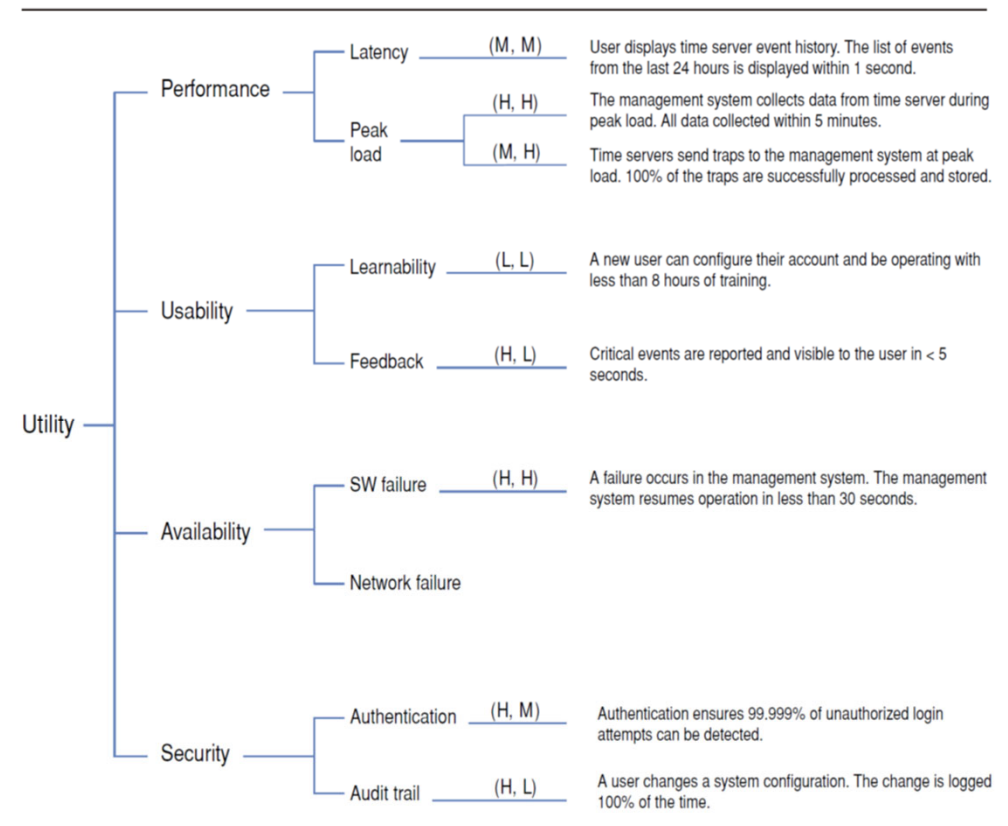


FIGURE 2.2 The six parts of a quality attribute scenario

Requirement-ID	QA_###
Category	관련된 <i>Quality Attribute</i> 가 무엇인지 기술함 (예: <i>Performance</i> , <i>Reliability</i> , <i>Security</i> 등)
Source	<i>Stimulus</i> 를 발생시키는 주체가 무엇인지 기술함
Stimulus	시스템에 입력되는 내외부 자극이 무엇인지 기술함
Artifacts	<i>Stimulus</i> 의 영향을 받는 시스템의 내부 모듈, 컴포넌트, 혹은 시스템 전체
Environment	해당 <i>Stimulus</i> 발생시 시스템의 환경(운영모드일 수도 있고, 그 외 다양한 상황 가능)
Response	기술된 <i>Environment</i> 에서 <i>Artifact</i> 이 <i>Stimulus</i> 를 받아들인 후 취하는 <i>Action</i> 이 무엇인지 설명함
Response Measure	위의 <i>Response</i> 의 정도를 측정하는 단위가 무엇인지 기술함 (예: 초당 데이터 처리량, 반응시간, 시간일 경우 <i>hour</i> , <i>minute</i> , <i>second</i> 여부 등)
Priority	<i>Quality Attribute Tree</i> 상에서의 우선순위
Description	위에 기술된 <i>Source</i> 부터 <i>Response Measure</i> 까지의 내용을 하나의 문장으로 요약해서 기술함

Quality Attribute Tree and Utility Tree

No.	Category	Response Measure	품질속성 요구사항
1	Performance	시스템 동작 Hz 수	1 Core System에서 4 Core System을 지원하는 system으로 변경될 경우, 정상적인 Operation을 수행하는 데 있어서 소모되는 전력이 1 Core System의 경우와 비교하여 35%의 Hz수를 줄여야 한다.
2		Communication 횟수, Communication 데이터	정상적인 Operation을 수행하는 데 있어서 Core 간 Communication 으로 인해 발생하는 Overhead 증가분은 10% 이내이어야 한다.
3		사용된 Memory 용량	정상적인 Operation을 수행하는 데 있어서 memory 사용량 증가 정도는 70 % 이내이어야 한다.
4		수행 중 각 Core들의 Idle Time	정상적인 Operation을 수행하는 데 있어서 전체 system의 운영 시간 중 각 Core들이 Idle 상태에 머무르는 시간은 15% 이내 이어야 한다.
5		Data Frames/Second	정상적인 Operation하에서 Video Data Decoding 성능은 기존 1 Core System의 3배 정도인 2.5 Data Frames/Second를 만족시켜야 한다.
6		화면의 가로 세로 픽셀 수	정상적인 Operation하에서 Video Decoder가 지원 가능한 출력 화면의 크기는 기존 1 Core System의 경우와 마찬가지로 1280X720 픽셀까지이다.
7		Bit rate	정상적인 Operation하에서 Video Decoder가 출력하는 화면의 화질을 측정하는 Bit rate는 기존 1 Core System의 경우와 마찬가지로 20Mbps를 지원 가능하여야 한다.
8	Modifiability	수정 컴포넌트 비율	4 Core System에서 향후 그 이상의 개수로 Core 숫자가 늘어날 경우, 4 Core System 기반의 Architecture 상에서 변경되는 Component와 Connector의 비율은 Parallel Node의 숫자에 해당하는 Instance 개수 증가를 제외하고 50% 미만이어야 한다.
9	Functionality	시스템 기능 가용률 :전체 기능 개수 대비 가용 기능 비율	1 Core System에서 4 Core System을 지원하는 system으로 변경될 경우, 정상적인 Operation하에서 Video Decoder가 제공하던 기능 중 가용한 기능은 100%를 만족시켜야 한다.
10		해당사항 없음	Video Decoder의 input data stream 중에 error가 있는 input frame이 입력될 경우, Video Decoder는 해당 frame의 decoding을 수행하지 않고 pass한 후 다음 frame을 읽어 들여야 한다.
11	Portability	시스템 기능 가용률 :전체 기능 개수 대비 가용 기능 비율	Cache memory size 가 32K인 Device에서 16K인 Device로 변경될 경우, 정상적인 Operation하에서 Video Decoder가 제공하던 기능 중 가용한 기능은 100%를 만족시켜야 한다.



A QAS Example - Availability

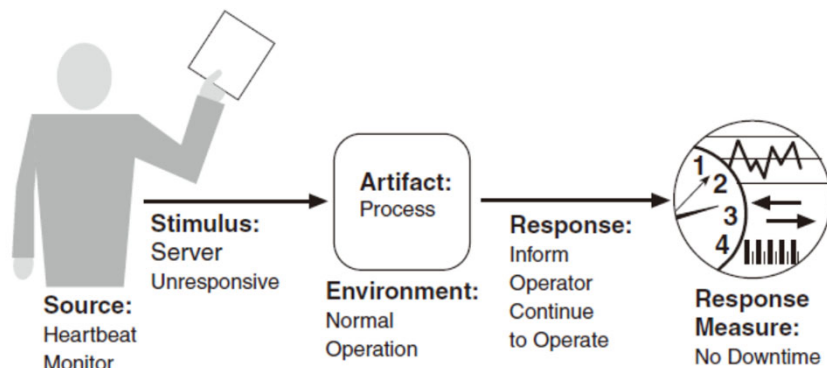


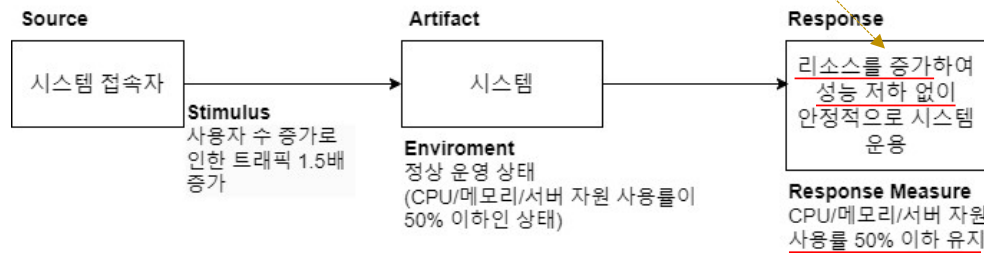
FIGURE 5.3 Sample concrete availability scenario

Portion of Scenario	Possible Values
Source	Internal/external: people, hardware, software, physical infrastructure, physical environment
Stimulus	Fault: omission, crash, incorrect timing, incorrect response
Artifact	Processors, communication channels, persistent storage, processes
Environment	Normal operation, startup, shutdown, repair mode, degraded operation, overloaded operation
Response	Prevent the fault from becoming a failure Detect the fault: - Log the fault - Notify appropriate entities (people or systems) Recover from the fault: - Disable source of events causing the fault - Be temporarily unavailable while repair is being effected - Fix or mask the fault/failure or contain the damage it causes - Operate in a degraded mode while repair is being effected
Response Measure	Time or time interval when the system must be available Availability percentage (e.g., 99.999%) Time to detect the fault Time to repair the fault Time or time interval in which system can be in degraded mode Proportion (e.g., 99%) or rate (e.g., up to 100 per second) of a certain class of faults that the system prevents, or handles without failing

QAS : Examples

QAS-03 Scalability

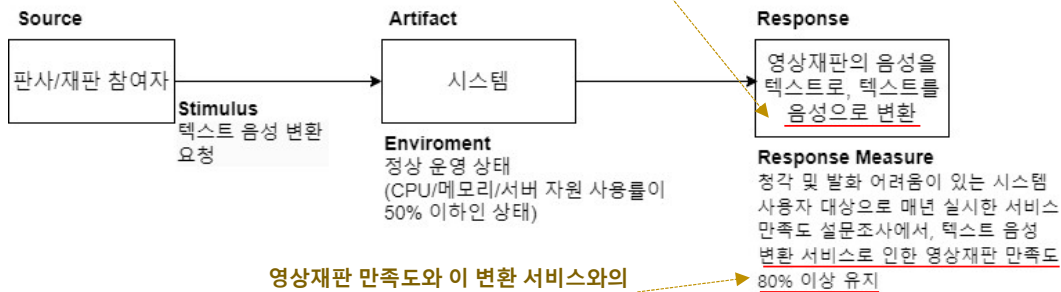
Raw Scenario: 트래픽 증가에 신속하게 대응할 수 있으면 좋겠다.



어떤 리소스?
리소스를 추가했는데, 성능이 저하될 수 있는가?
“성능” = “자원 사용률 50%이하” ??

QAS-06 Usability

Raw Scenario: 청각 혹은 발화에 문제가 있더라도 문제없이 재판에 참여할 수 있으면 좋겠다.



영상재판 만족도와 이 변환 서비스와의 직접적인 인과관계 설립 要.

Use Case

UC-04	영상재판 참여	판사 또는 재판 참여자 또는 참여 사무관이 데스크탑 APP을 이용하여 사건번호에 맞는 영상재판에 참여한다.	상	중	SF-05	X
-------	---------	---	---	---	-------	---

ID	Title	QA Type	Priority		Related Use Case	System Feature ID
			I	D		
QAS-01	영상 재판의 응답 속도	Performance	상	상	UC-06	SF-08
QAS-02	재판 자료 확인 및 입력 과정의 응답 속도	Performance	상	중	UC-03	SF-07
QAS-03	시스템 동시 사용자 수 증가로 인한 트래픽 증가에 신속하고 안정적인 대응	Scalability	중	중	UC-03, UC-04, UC-05, UC-06, UC-07, UC-10	SF-18
QAS-04	시스템 장애 발생 시 재판 자료 손실 방지	Availability	상	상	UC-03, UC-05, UC-06	SF-21
QAS-05	시스템 장애 발생 시 신속한 시스템 복구	Availability	상	중	UC-03, UC-04, UC-05, UC-06	SF-20
QAS-06	텍스트 음성 변환을 통한 사용자 접근성 향상	Usability	하	하	UC-06	SF-13
QAS-07	재판 자료 접근 권한 설정 및 재판 자료 위변조 방지	Security	상	중	UC-03, UC-04, UC-05, UC-06	SF-01, SF-02

QAS

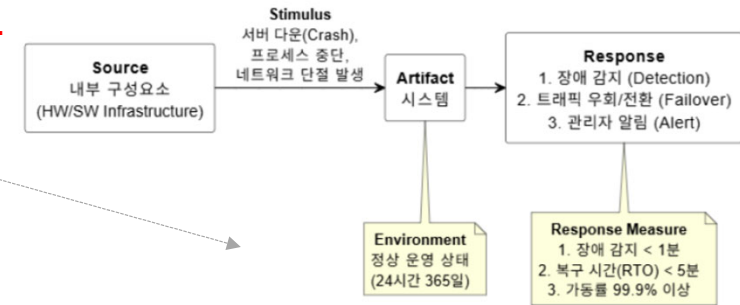
DS Examples

그림 + 표가 함께 있으면 좋아요.
System Test에서 측정 가능하도록 구체적이어야 해요.
가능하면 ISO/IEC 25023:2026을 사용해 보세요.

3.2.2. QA-01 CCTV 서비스 가용성

QA Type	Availability
Description	시스템은 주-CCTV 장비 또는 영상 수집 경로에 장애가 발생하더라도, 열람실 현황 업데이트를 5초 이내에 완료해야 한다.
Source of Stimulus	CCTV 장비
Stimulus	운영 중인 특정 열람실의 CCTV가 갑작스럽게 네트워크 단절 또는 프로세스 crash 상태가 된다.
Artifact	System
Environment	열람실 평상시 운영 환경
	다수 열람실이 동시에 사용 중인 상태
Response	시스템은 장애 CCTV를 감지한다. 백업 CCTV 또는 기존 좌석 상태 정보를 활용하여 좌석 조회 및 배정 서비스를 유지한다. 관리자 및 해당 열람실 신청 학생에게 장애 발생 알림을 전송한다.
Response Measure	서비스 중단 시간 5초 이내 (빈자리 조회/배정 기능 기준) 관리자·학생 알림 전송 ≤ 5초 이내

3.2.2. QA-01 서버 장애 시 서비스 가용성 확보



QA Type	Reliability (Availability / Fault Tolerance)
Description	핵심 서버(WAS, DB)나 프로세스에 하드웨어 고장, 네트워크 단절, 소프트웨어 오류 등 예기치 못한 장애가 발생하더라도, 이를 즉시 감지하고 예비 자원으로 전환하여 서비스 중단을 최소화해야 한다.
Source of Stimulus	내부 구성요소 (Internal Component) 또는 하드웨어 인프라
Stimulus	서버 인스턴스 다운(Crash), 응답 지연(Timeout), 또는 프로세스 중단(Omission) 발생
Artifact	시스템
Environment	정상 운영 상태
Response	1. 모니터링 시스템이 장애를 감지(Detect)하고 관리자에게 알림을 보낸다. 2. 예비 자원 할당하여 서비스 중단 없이 동작한다.
	3. 장애가 발생한 노드를 재부팅하여 복구한다.
Response Measure	1. 장애 감지 시간 < 1 분 2. 서비스 복구 시간(RTO) < 5 분 3. 서비스 가동률 99.9% 이상

Software Quality Model : ISO/IEC 9126

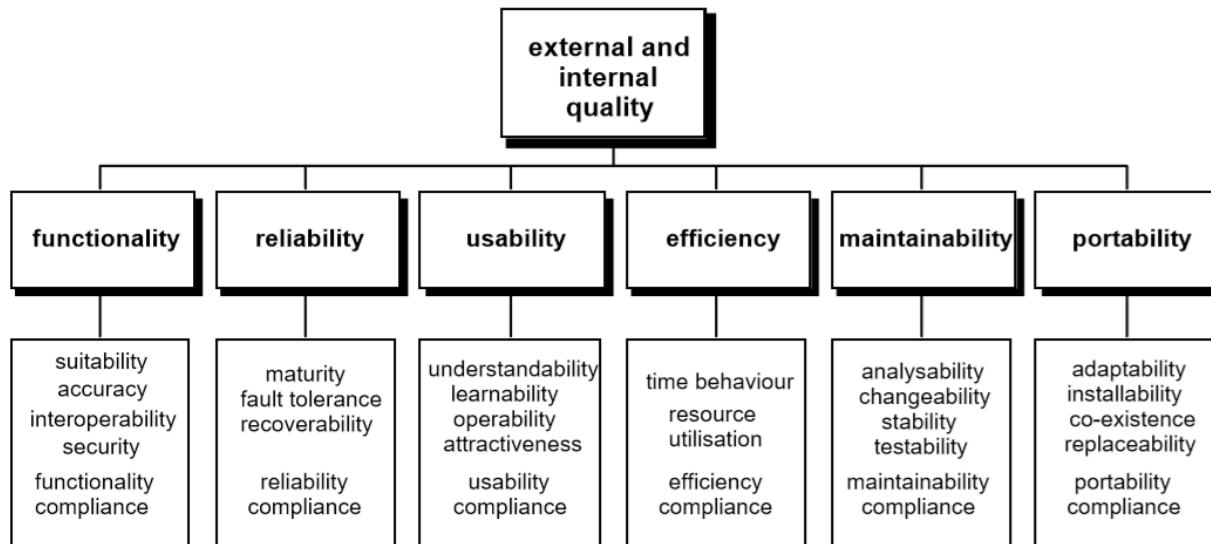


Figure 4 – Quality model for external and internal quality

FINAL
DRAFT

INTERNATIONAL
STANDARD

ISO/IEC
FDIS
9126-1

ISO/IEC JTC 1
Secretariat: ANSI
Voting begins on:
2000-01-20
Voting terminates on:
2000-03-20

Information technology — Software product quality —
Part 1:
Quality model

Technologies de l'information — Qualité des produits logiciels —
Partie 1: Modèle de qualité

RECIPIENTS OF THIS DOCUMENT ARE INVITED TO SUBMIT, WITH THEIR COMMENTS, NOTIFICATION OF ANY RELEVANT PATENT RIGHTS OF WHICH THEY ARE AWARE AND TO PROVIDE SUPPORTING DOCUMENTATION.

IN ADDITION TO THEIR EVALUATION AS BEING ACCEPTABLE FOR INDUSTRIAL, TECHNOLOGICAL, COMMERCIAL AND USER PURPOSES, DRAFT INTERNATIONAL STANDARDS MAY ON OCCASION HAVE TO BE CONSIDERED IN THE LIGHT OF THEIR POTENTIAL TO BECOME STANDARDS TO WHICH REFERENCE MAY BE MADE IN NATIONAL REGULATIONS.

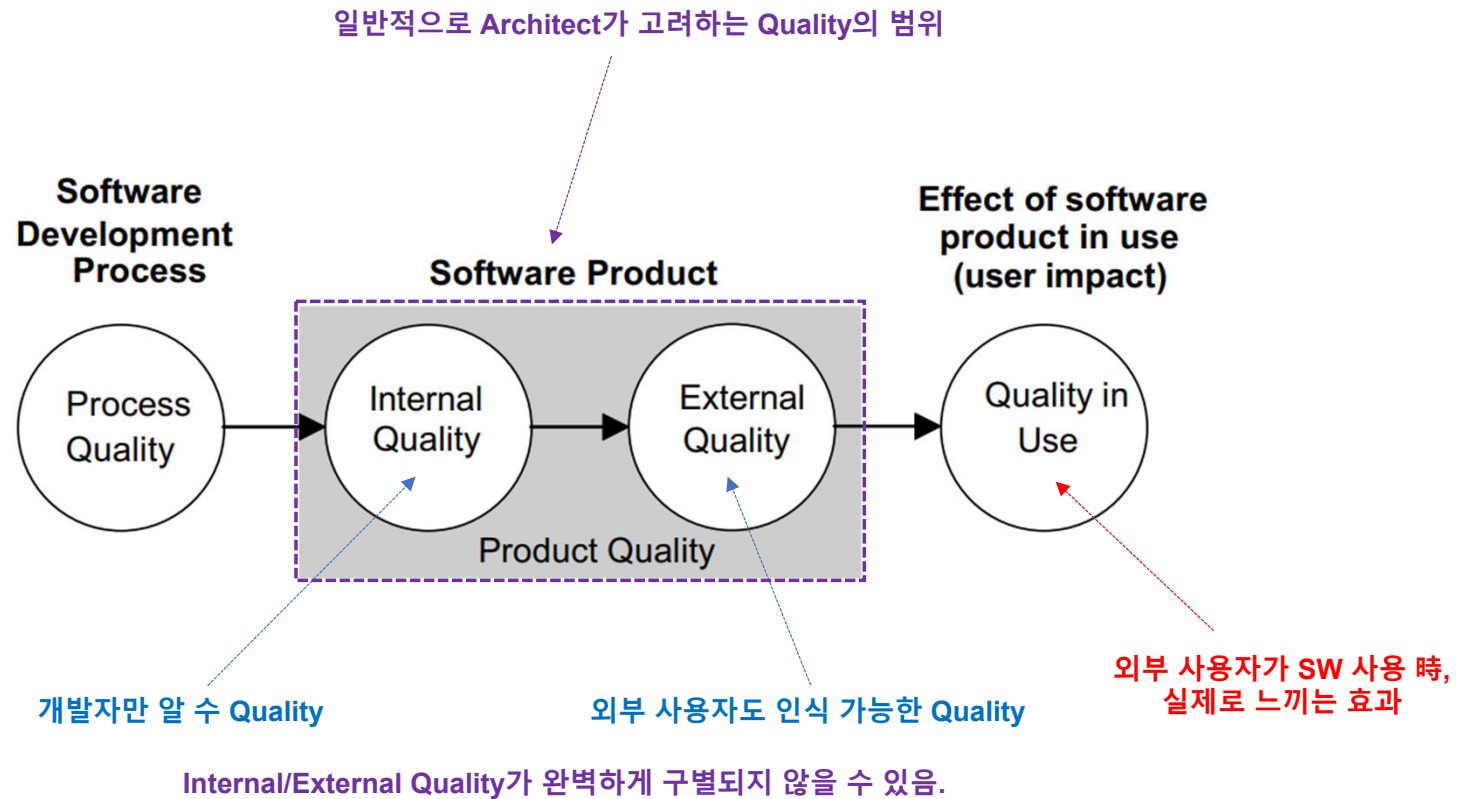
Please see the administrative notes on page ii-1

Reference number
ISO/IEC FDIS 9126-1:2000(E)

ISO IEC

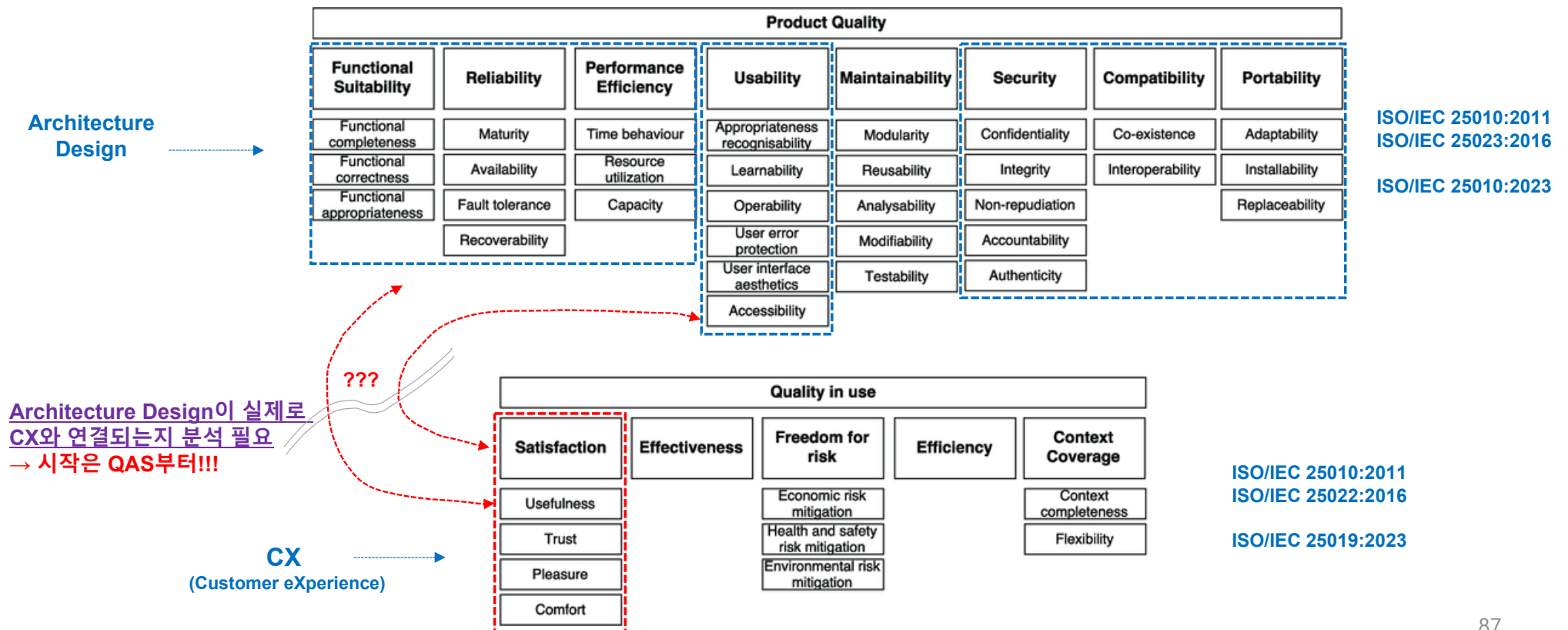
© ISO/IEC 2000

Conventional Quality Categories in ISO/IEC 9126



ISO/IEC 25010:2011 SQuaRE – System and Software Quality Model

- ISO/IEC 25010:2011 Systems and software engineering - **Systems and software Quality Requirements and Evaluation (SQuaRE)** - **System and software quality models**

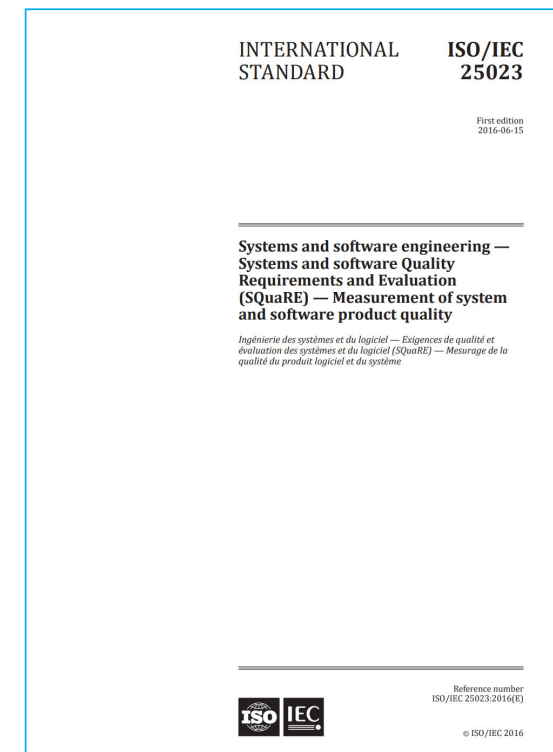


ISO/IEC 25023:2016 SQuaRE – Measurement of Product Quality

- ISO/IEC 25023:2016 Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - **Measurement of system and software product quality**
 - Based on ISO/IEC 25010:2011

Characteristics	Sub-characteristics	Measure
Performance Efficiency	Time Behavior	Mean Response Time
		Response Time Adequacy
		Mean Turnaround Time
		Turnaround Time Adequacy
		Mean Throughput
	Resource Utilization	Mean Processor Utilization
		Mean Memory Utilization
		Mean I/O Devices Utilization
		Bandwidth Utilization
	Capacity	Transaction Processing Capacity
		User Access Capacity
		User Access Increase Adequacy

ID	Name	Description	Measurement function
PTb-1-G	Mean response time	How long is the mean time taken by the system to respond to a user task or system task?	$X = \sum_{i=1 \text{ to } n} (A_i) / n$ A_i = Time taken by the system to respond to a specific user task or system task at i-th measurement n = Number of responses measured
PTb-2-G	Response time adequacy	How well does the system response time meet the specified target?	$X = A/B$ A = Mean response time measured by PTb-1-G B = Target response time specified



IOS/IEC 25023:2016 Measures for Product Qualities

Characteristics	Sub-characteristics	Measure
Functional Suitability	Functional Completeness	Functional Coverage
	Functional Correctness	Functional Correctness
	Functional Appropriateness	Functional Appropriateness of Usage Objective
		Functional Appropriateness of the Systems
Performance Efficiency	Time Behavior	Mean Response Time
		Response Time Adequacy
		Mean Turnaround Time
		Turnaround Time Adequacy
		Mean Throughput
	Resource Utilization	Mean Processor Utilization
		Mean Memory Utilization
		Mean I/O Devices Utilization
		Bandwidth Utilization
	Capacity	Transaction Processing Capacity
		User Access Capacity
		User Access Increase Adequacy
Compatibility	Co-Existence	Co-Existence with Other Products
	Interoperability	Data Formats Exchangeability
		Data Exchange Protocol Sufficiency
		External Interface Adequacy

Characteristics	Sub-characteristics	Measure
Usability	Appropriateness Recognizability	Description Completeness
		Demonstration Coverage
		Entry Point Self-Descriptiveness
	Learnability	User Guidance Completeness
		Entry Fields Defaults
		Error Messages Understandability
		Self-Explanatory User Interface
	Operability	Operational Consistency
		Message Clarity
		Functional Customizability
		User Interface Customizability
		Monitoring Capacity
		Undo Capacity
		Understandable Categorization of Information
		Appearance Consistency
		Input Device Support
	User Error Protection	Avoidance of User Operation Error
		User Entry Error Correction
		User Error Recoverability
	User Interface Aesthetics	Appearance Aesthetics of User Interfaces
	Accessibility	Accessibility for Users with Disabilities
		Supported Languages Adequacy

IOS/IEC 25023:2016 Measures for Product Qualities

Characteristics	Sub-characteristics	Measure
Reliability	Maturity	<i>Fault Correction</i>
		<i>Mean Time Between Failure (MTBF)</i>
		<i>Failure Rate</i>
		<i>Test Coverage</i>
	Availability	<i>System Availability</i>
		<i>Mean Down Time</i>
	Fault Tolerance	<i>Failure Avoidance</i>
		<i>Redundancy of Components</i>
		<i>Mean Fault Notification Time</i>
	Recoverability	<i>Mean Recovery Time</i>
		<i>Backup Data Completeness</i>
Security	Confidentiality	<i>Access Controllability</i>
		<i>Data Encryption Correctness</i>
		<i>Strength of Cryptographic Algorithms</i>
	Integrity	<i>Data Integrity</i>
		<i>Internal Data Corruption Prevention</i>
		<i>Buffer Overflow Prevention</i>
	Non-Repudiation	<i>Digital Signature Usage</i>
	Accountability	<i>User Audit Trail Completeness</i>
		<i>System Log Retention</i>
	Authenticity	<i>Authentication Mechanism Sufficiency</i>
		<i>Authentication Rules Conformity</i>

Characteristics	Sub-characteristics	Measure
Maintainability	Modularity	<i>Coupling of Components</i>
		<i>Cyclomatic Complexity Adequacy</i>
	Reusability	<i>Reusability Assets</i>
		<i>Coding Rules Conformity</i>
	Analyzability	<i>System Log Completeness</i>
		<i>Diagnosis Function Effectiveness</i>
		<i>Diagnosis Function Sufficiency</i>
	Modifiability	<i>Modification Efficiency</i>
		<i>Modification Correctness</i>
		<i>Modification Capability</i>
Portability	Testability	<i>Test Function Completeness</i>
		<i>Autonomous Testability</i>
		<i>Test Restartability</i>
	Adaptability	<i>Hardware Environmental Adaptability</i>
		<i>System Software Environmental Adaptability</i>
		<i>Operational Environmental Adaptability</i>
	Installability	<i>Installation Time Efficiency</i>
		<i>Ease of Installation</i>
	Replaceability	<i>Usage Similarity</i>
		<i>Product Quality Equivalence</i>
		<i>Functional Inclusiveness</i>
		<i>Data Reusability/Import Capability</i>

ISO/IEC 25019:2023 SQuaRE – Quality-in-Use Model

- ISO/IEC 25019:2023 Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - **Quality-in-use model**
 - Replacing the quality-in-use of ISO/IEC 25010:2011
 - Many changes, but not widely used yet
 - Corresponding std. 25022:2016 is not updated yet.

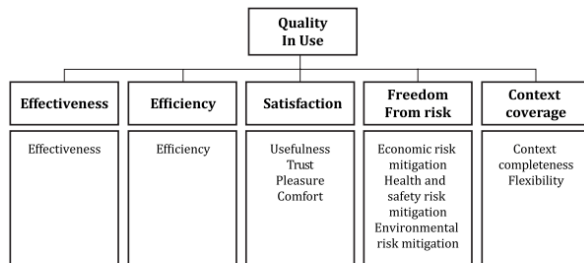


Figure A.1 — Quality-in-use model in ISO/IEC 25010:2011

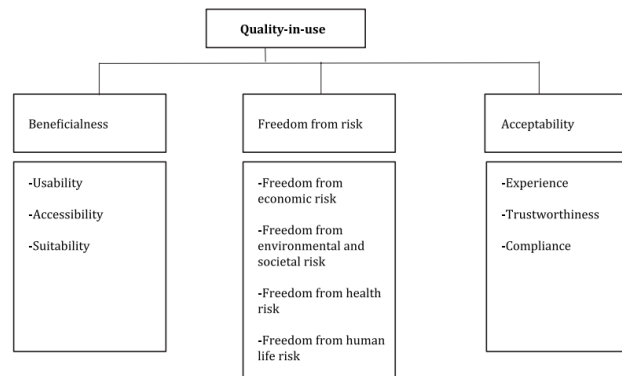
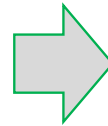
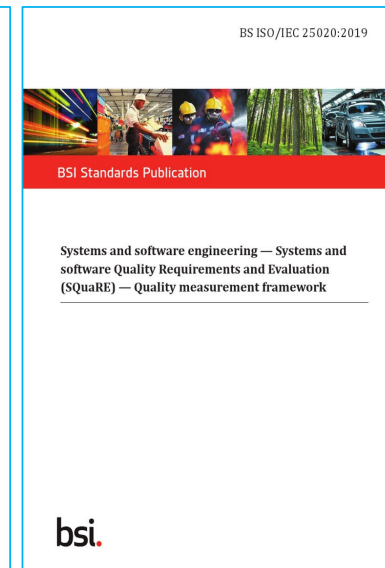
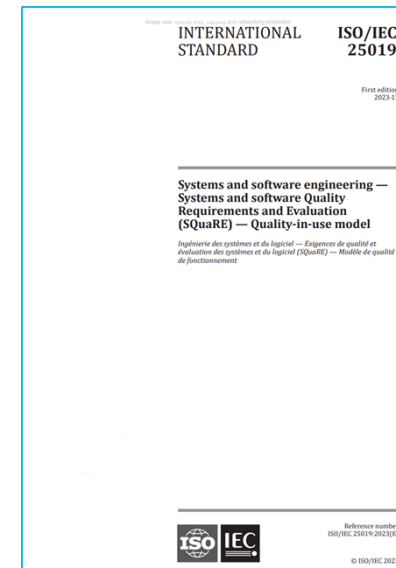


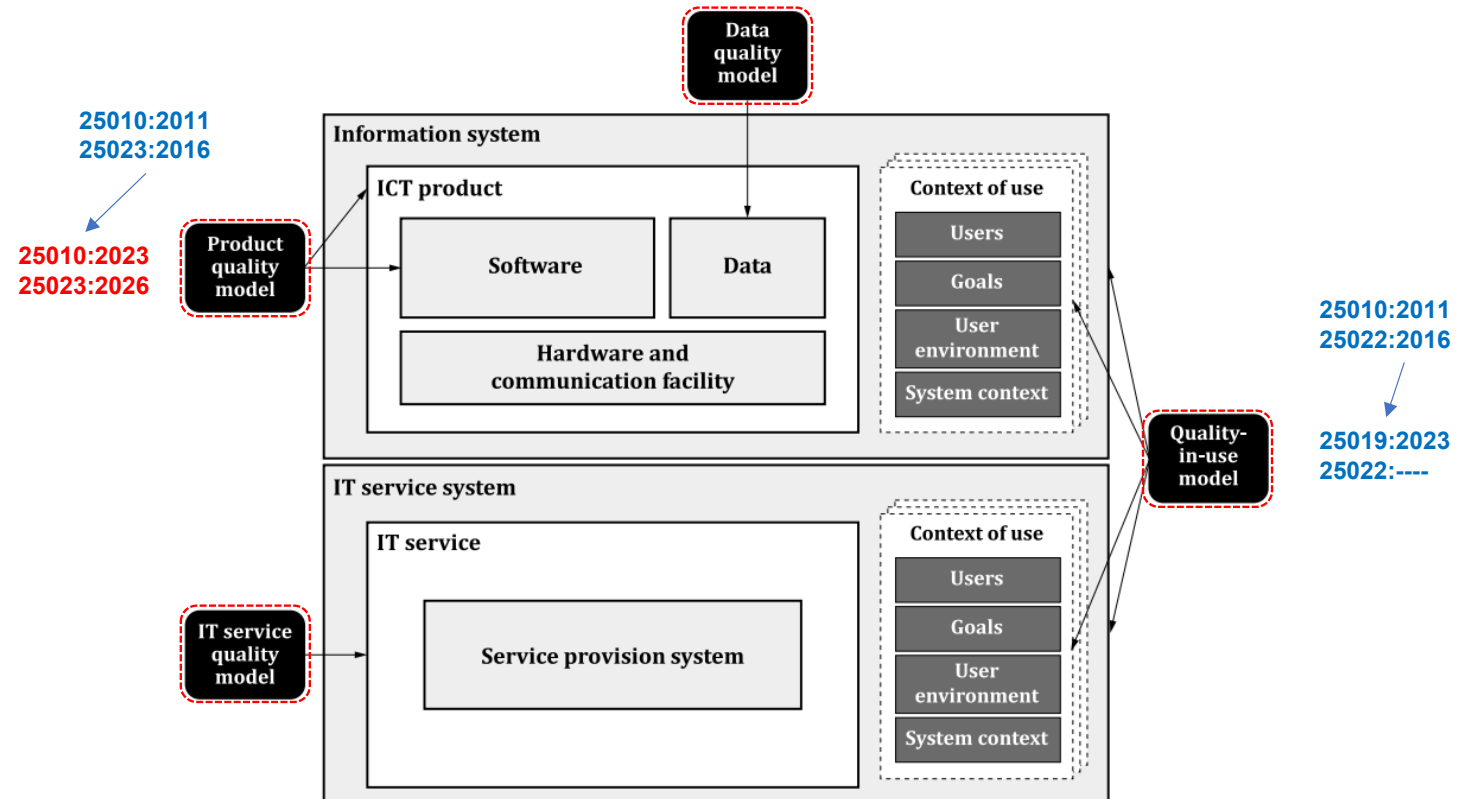
Figure 2 — The quality-in-use model



Quality Model in ISO/IEC 25010:2023

• 4 Quality Models

- Product quality
- Data quality
- IT service quality
- Quality-in-use



NOTE Arrows indicate the target entities within information systems or IT services to which 2501n quality models apply.

Figure 3 — Quality models and their target entities

ISO/IEC 25010:2023 SQuaRE – Product Quality Model

- ISO/IEC 25010:2023 Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - **Product quality model**

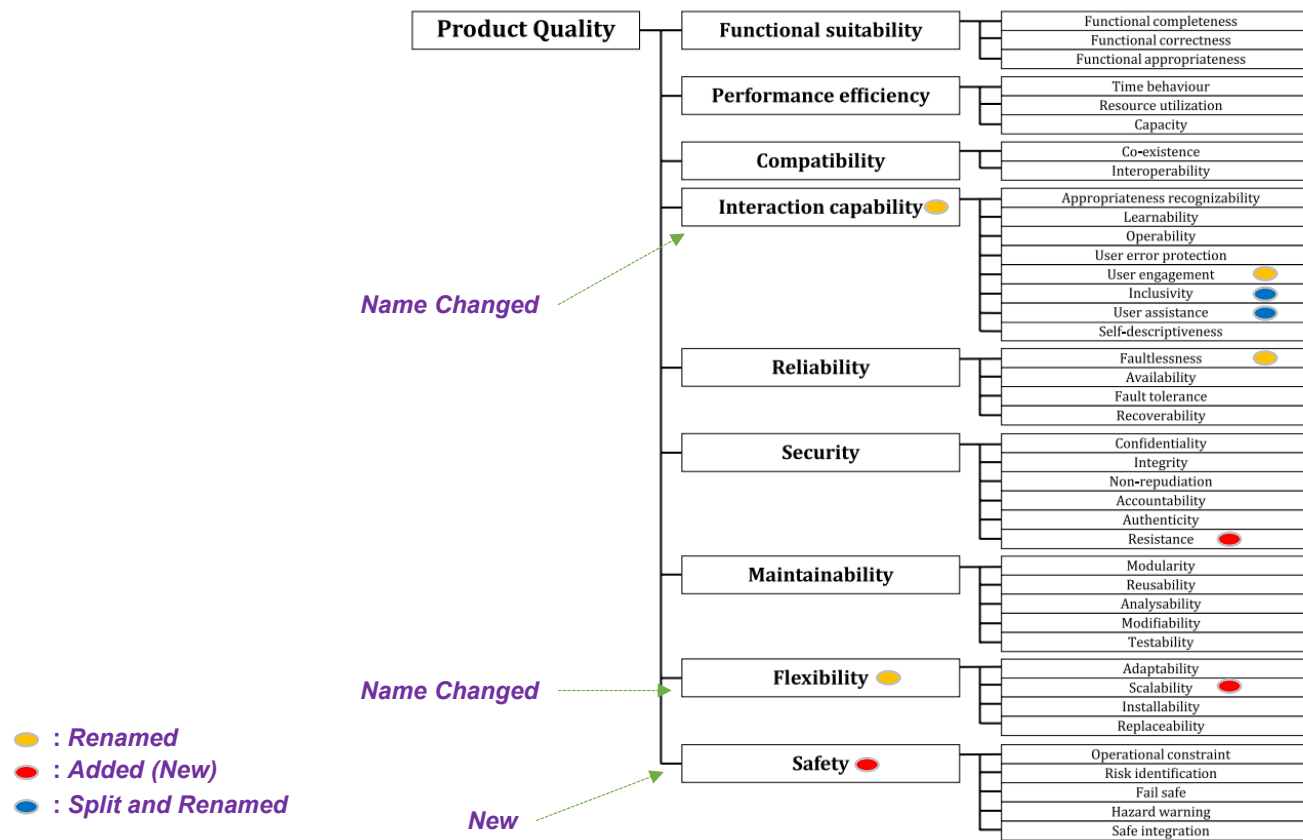
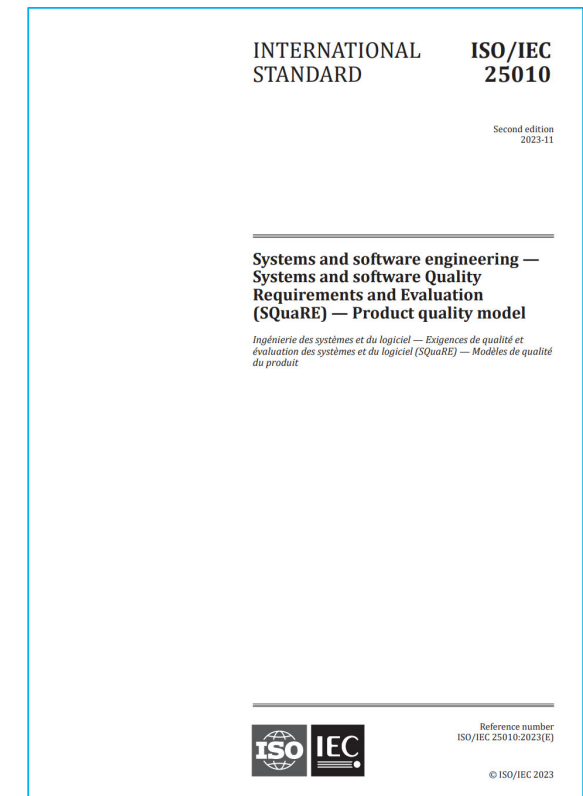


Figure 2 — Product quality model



ISO/IEC 25023:2026 SQuaRE – Measurement of Product Quality

- ISO/IEC 25023:2026 Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - **Measurement of product quality**
 - Based on ISO/IEC 25010:2023

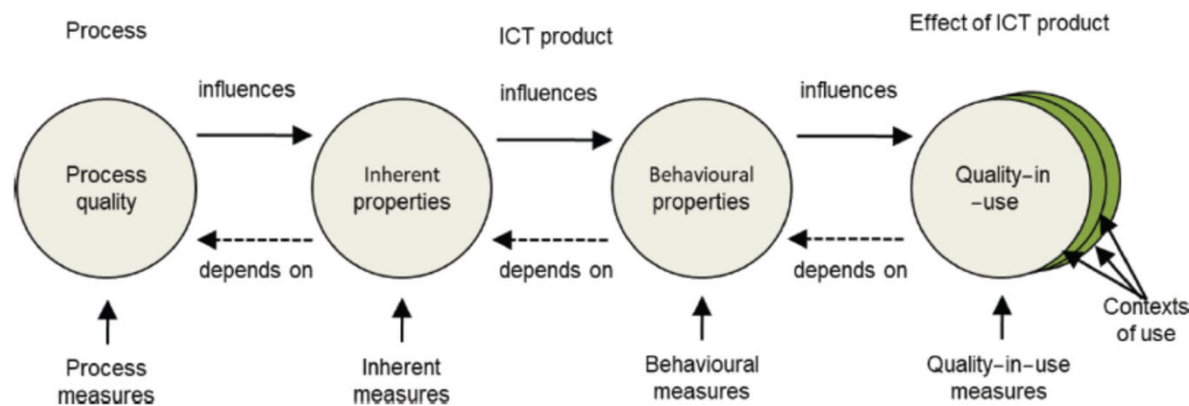


Figure 3 — Relationship between types of quality measures

	DRAFT International Standard
Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Measurement of product quality	
<i>Ingenierie des systemes et du logiciel — Exigences de qualite et Evaluation des systemes et du logiciel (SQuaRE) — Mesurage de la qualite du produit logiciel et du systeme</i>	
ICS: 35.080	ISO/IEC DIS 25023 ISO/IEC JTC 1/SC 7 Secretariat: BIS Voting begins on: 2026-03-16 Voting terminates on: 2026-06-08
Please share your feedback about the standard. Scan the QR code with your phone or click the link. Customer Feedback Form	THIS DOCUMENT IS A DRAFT CIRCULATED FOR COMMENTS AND APPROVAL. IT IS THEREFORE SUBJECT TO CHANGE AND MAY NOT BE REFERRED TO AS AN INTERNATIONAL STANDARD UNTIL PUBLISHED AS SUCH. IN ADDITION TO THEIR EVALUATION AS BEING ACCEPTABLE FOR INDUSTRIAL, TECHNOLOGICAL, COMMERCIAL AND USER PURPOSES, DRAFT INTERNATIONAL STANDARDS MAY ON OCCASION HAVE TO BE CONSIDERED IN THE LIGHT OF THEIR POTENTIAL TO BECOME STANDARDS TO WHICH REFERENCE MAY BE MADE IN NATIONAL REGULATIONS. RECORDS OF THIS DRAFT ARE INVITED TO SUBMIT WITH THEIR COMMENTS. NOTIFICATION OF ANY RELEVANT PATENT RIGHTS OF WHICH THEY ARE AWARE AND TO PROVIDE SUPPORTING DOCUMENTATION.
This document is circulated as received from the committee secretariat.	
Reference number ISO/IEC DIS 25023:2026(en)	
© ISO/IEC 2026	

IOS/IEC 25023:2026 Measures for Product Qualities

7.3.2 Time behaviour measures

Time behaviour measures are used to assess the capability of a product to perform its specified function under specified conditions so that the response time and throughput rates meet the requirements.

Table 4 — Time behaviour measures

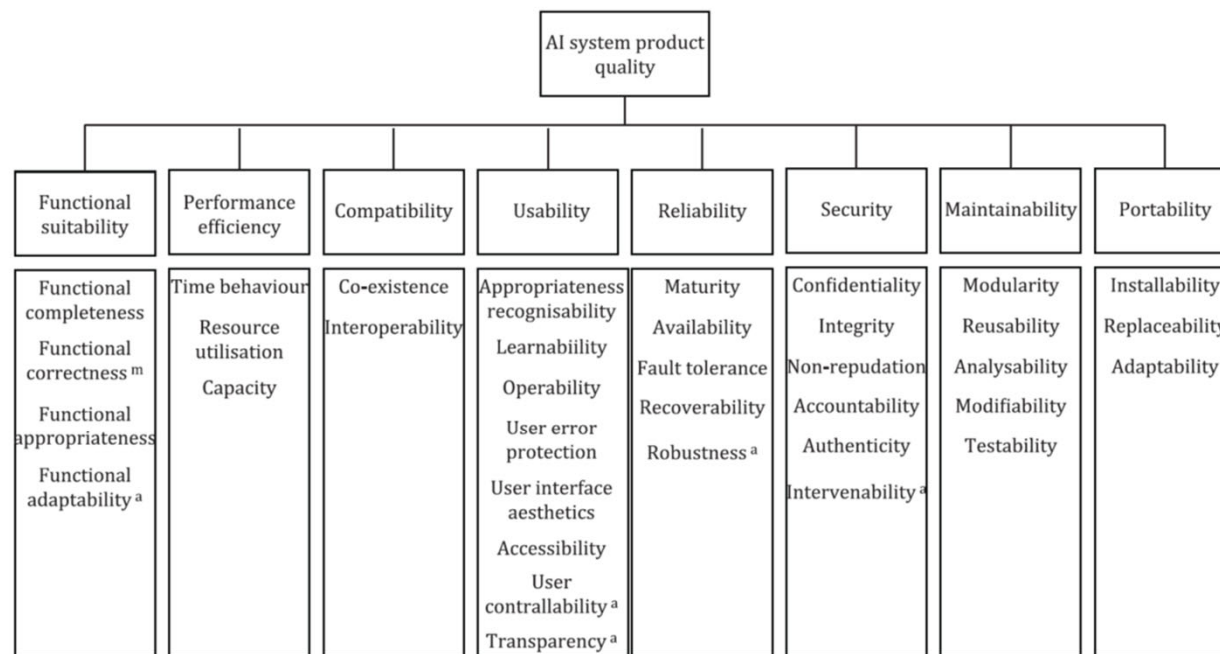
ID	Name	Description	Measurement function
PTb-1-G	Mean system wait time	What is the mean time between the arrival of a request and the point when the processing begins?	$X = \sum_{i=1}^n (A_i) / n$ A_i = Time between the arrival of an i-th request and the point when the processing begins n = Number of requests measured
PTb-2-G	Mean response time	How long is the mean time for the product to respond to a user request to perform specified tasks under specified conditions?	$X = \sum_{i=1}^n (A_i) / n$ A_i = Time taken by the product to respond to a user i-th request to perform a specific user task n = Number of responses measured
NOTE 1 In the case of a pipeline (e.g. a systems chain), the elapsed time in each stage of the pipeline has to be considered and bottlenecks in one stage can affect overall turnaround time. NOTE 2 It is a good practice to use this measure in conjunction with specified payload and/or workload. NOTE 3 See Figure 4 below for a visual distinction between response time and turnaround time.			
PTb-3-G	Response time adequacy	What proportion of the product response times meet the specified response time requirements?	$X = 1 - A/B$ A = Number of response instances that do not meet the specified response time requirements B = Number of response time requirements
NOTE 1 Response time is the duration it takes for the initial response to be received after a request has been initiated, whereas turnaround time represents the total duration required for a task to be initiated and completed. NOTE 2 An alternative to this measure is nth percentile response time under expected load conditions. It is also useful to apply it to individual functions or classes of functions.			
PTb-4-G	Mean turnaround time	What is the mean time taken for completion of a job or an asynchronous process?	$X = \sum_{i=1}^n (B_i - A_i) / n$ A_i = Time of job submission or time of starting asynchronous process i B_i = Time of completing the job or asynchronous process i n = Number of jobs or asynchronous processes

ID	Name	Description	Measurement function
PTb-5-G	Turnaround time adequacy	What proportion of the product turnaround time meets the specified turnaround time requirements?	$X = 1 - A/B$ A = Number of turnaround instances that do not meet the specified turnaround time requirements B = Number of turnaround time measured
Figure 4 — Comparison Between Response Time and Turnaround Time			
PTb-6-G	Mean throughput	What is the mean number of jobs completed per unit of time?	$X = \sum_{i=1}^n (A_i / B_i) / n$ A_i = Number of jobs completed during i-th observation time B_i = The length of time in observation period i n = Number of observations

(...)

ISO/IEC 25029:2023 SQuaRE – Quality Model for AI Systems

- Software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - **Quality model for AI systems**



^a New sub-characteristics.

^m Modified sub-characteristics.

Figure 1 — AI system product quality model

Lists of System Quality Attributes (Wikipedia)

Quality attributes [\[edit \]](#)

Notable quality attributes include:

- accessibility
- accountability
- accuracy
- adaptability
- administrability
- affordability
- agility (see Common subsets below)
- auditability
- autonomy [Erl]
- availability
- compatibility
- composability [Erl]
- confidentiality
- configurability
- correctness
- credibility
- customizability
- debuggability
- degradability
- determinability
- demonstrability
- dependability (see Common subsets below)
- deployability
- discoverability [Erl]
- distributability
- durability
- effectiveness
- efficiency
- evolvability
- extensibility
- failure transparency
- fault-tolerance
- fidelity
- flexibility
- inspectability
- installability
- integrity
- interchangeability
- interoperability [Erl]
- learnability
- localizability
- maintainability
- manageability
- mobility
- modifiability
- modularity
- observability
- operability
- orthogonality
- portability
- precision
- predictability
- process capabilities
- producibility
- provability
- recoverability
- redundancy
- relevance
- reliability
- repeatability
- reproducibility
- resilience
- responsiveness
- reusability [Erl]
- robustness
- safety
- scalability
- seamlessness
- self-sustainability
- serviceability (a.k.a. supportability)
- securability (see Common subsets below)
- simplicity
- stability
- standards compliance
- survivability
- sustainability
- tailorability
- testability
- timeliness
- traceability
- transparency
- ubiquity
- understandability
- upgradability
- usability
- vulnerability

Many of these quality attributes can also be applied to [data quality](#).

3.3 Constraints

- **Constraint**

- Restrictions on the design or implementation choices available to the developer
- Can be imposed by external stakeholders and by other systems that interact with the system
- Should be respected and generally non-negotiable.

- **Design Purpose**

- **Architectural Concerns**

- **Constraints**

- Very important, but out of the scope of our CEP.

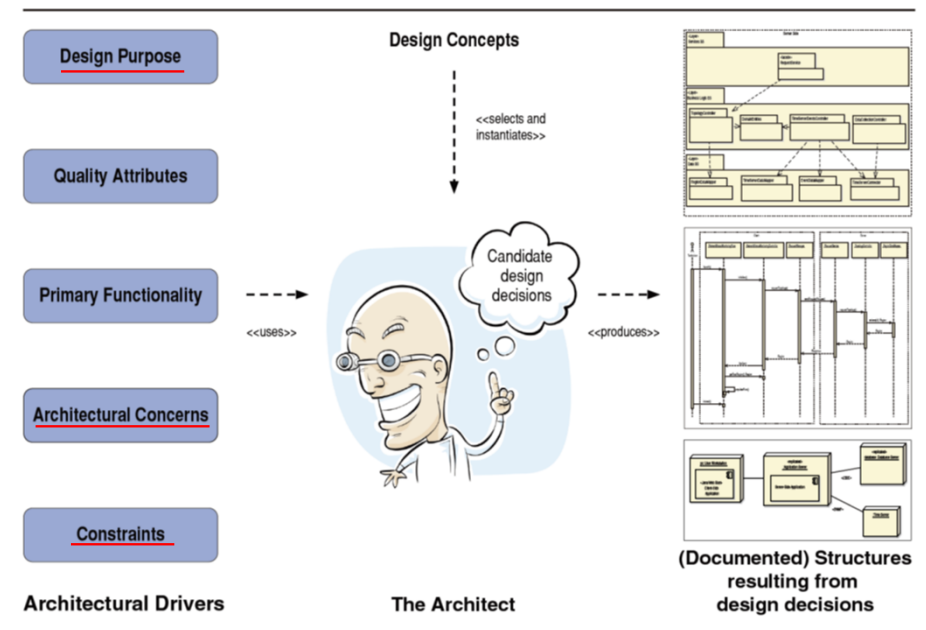


FIGURE 2.1 Overview of the architecture design activity
(Architect image © Brett Lamb | Dreamstime.com)

4. Architecture Design

Where We Are Now

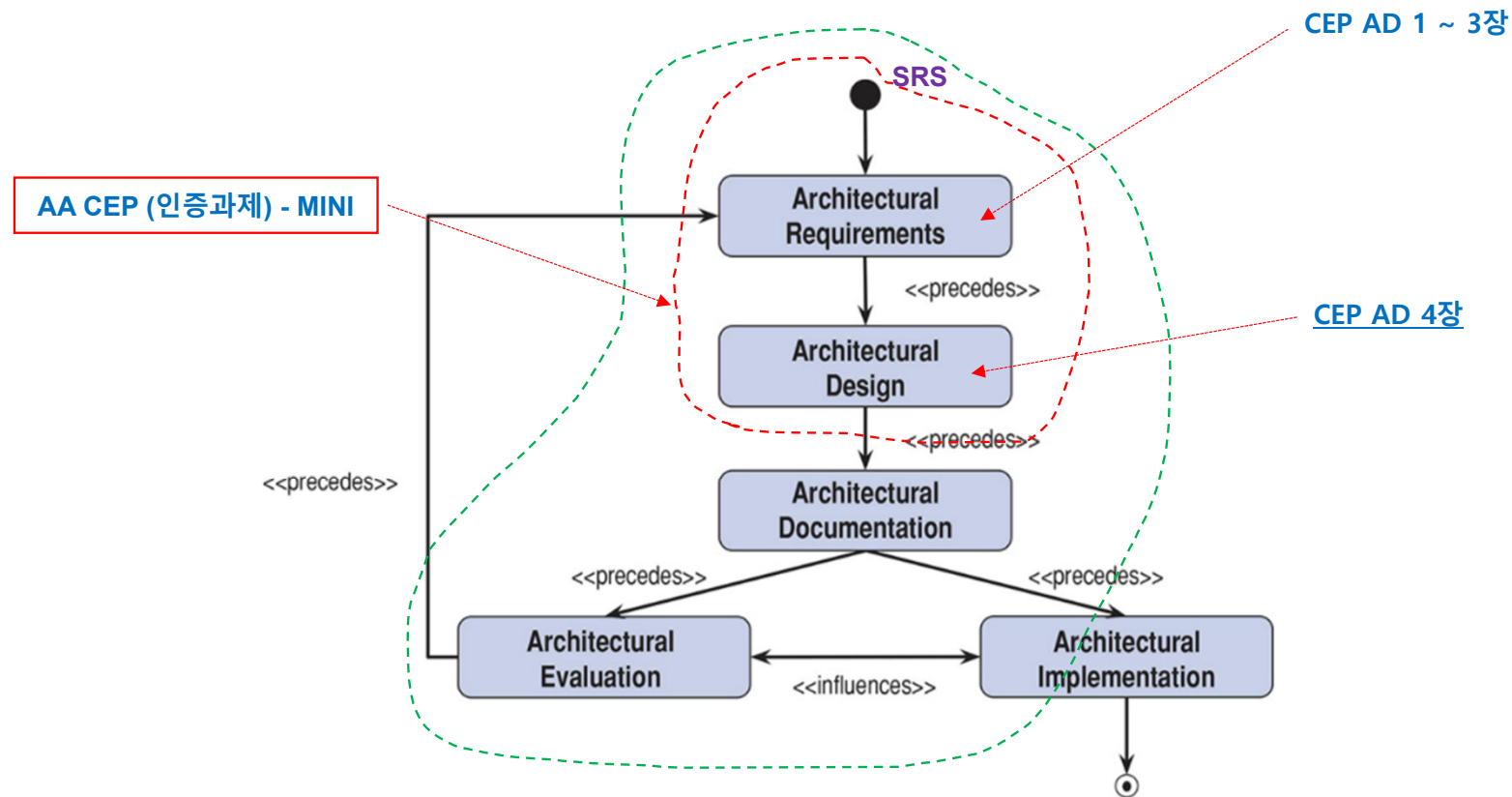


FIGURE 1.1 Software architecture life-cycle activities

Where We are Now in AD

1. Project Overview

- 1.1 Project Background
- 1.2 Stakeholders
- 1.3 Business Goals

2. System Overview

- 2.1 System Context Diagram
- 2.2 System Features
- 2.3 Assumptions about the System

3. Architectural Drivers

- 3.1 Primary Functionality
- 3.2 Quality Attribute Scenario

4. Architecture Design

- 4.1 Candidate Designs per QA
- 4.2 Candidate Decisions per QA
- 4.3 Design Decision (Overview)

5. Summary

Overview of the Architecture Design Activity

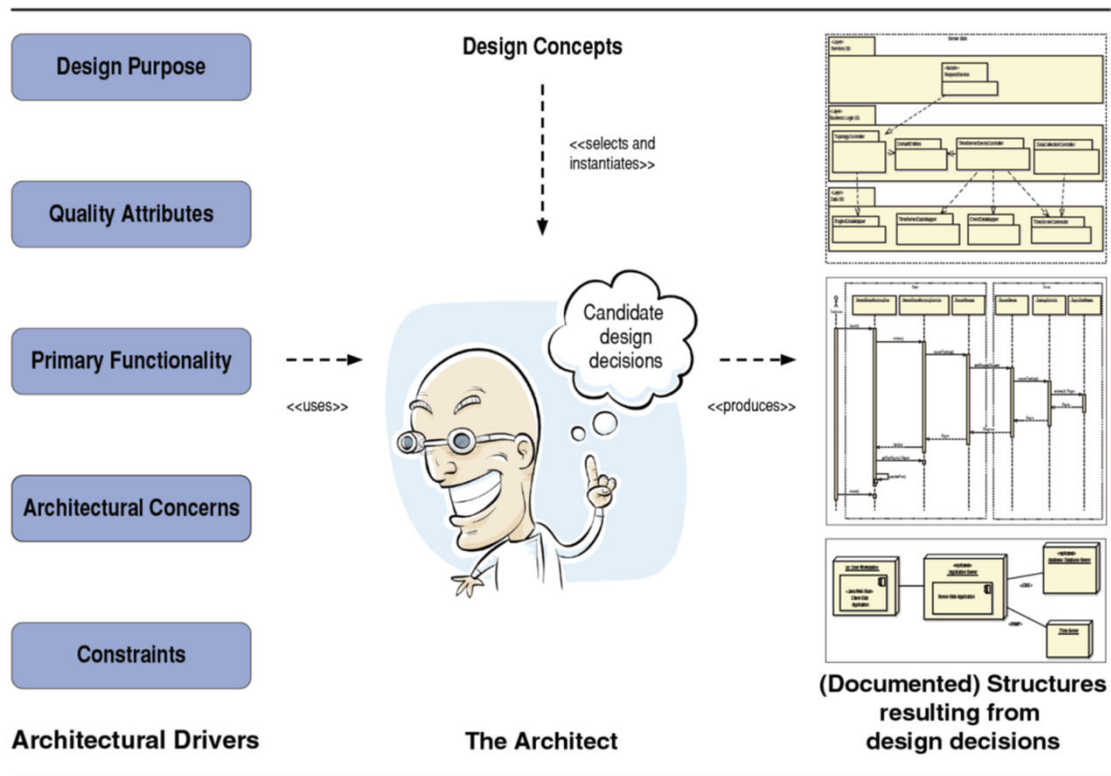


FIGURE 2.1 Overview of the architecture design activity
(Architect image © Brett Lamb | Dreamstime.com)

(1st Edition)

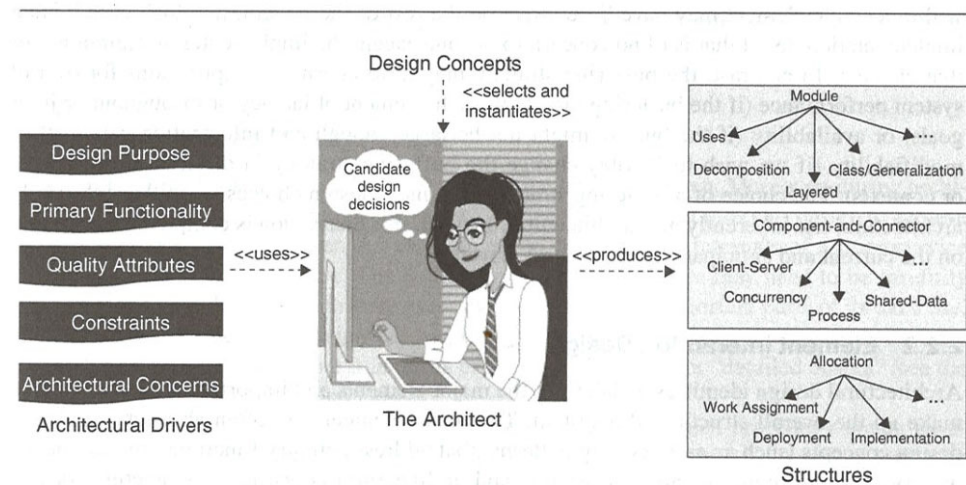


FIGURE 2.1 Overview of the architecture design activity

(2nd Edition)

Typical Architecture Design Approach in a Nutshell

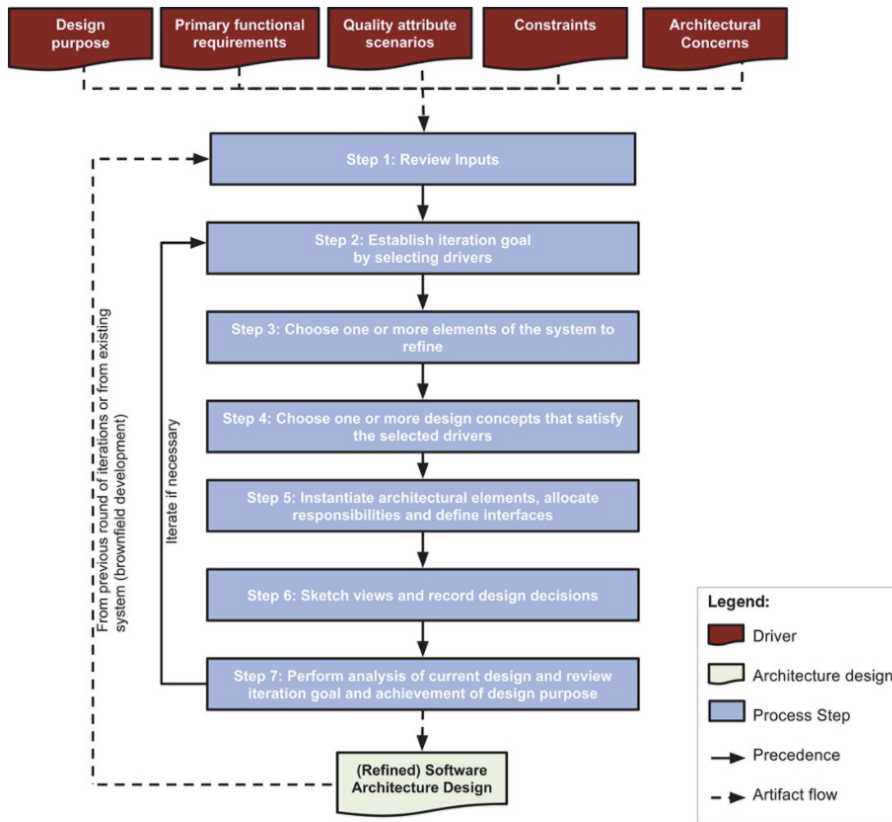


FIGURE 3.1 Steps and artifacts of ADD version 3.0

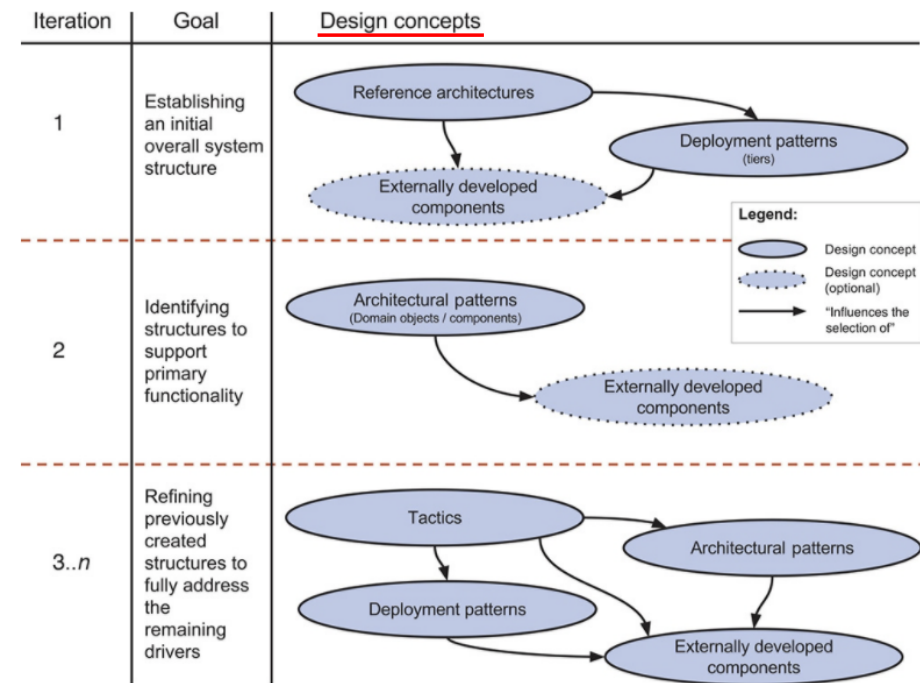


FIGURE 3.2 Design concept selection roadmap for greenfield systems

Design Concepts

- Building blocks for creating structures

Reference Architecture

- Blueprint for overall architecture
- Logical structure for specific application types
- Embodying architecture styles

Examples:

- Mobile applications
- Rich client applications
- Rich internet applications
- Service applications
- Web applications



Deployment Pattern

- How to deploy logical into physical
- Physical structure
- Essential for many QAs (Performance, Security, Availability, ...)

Examples:

- Nondistributed
- Distributed
- Performance
- Reliability
- Security



Architecture Style

- General and reusable structural layout and its properties
- Not domain-specific
- Logical >> Physical structure

Examples:

- MVC, MVP, MVVM, Layered
- Client-Server, SOA, Microservices
- Pipes and Filters, Blackboard



Tactics

- Building blocks of other patterns
- Widely-used techniques to manage QAs by architects
- Quickly evolved

Examples:

- Availability
- Interoperability
- Modifiability
- Performance
- Security
- Testability
- Usability



Externally Developed Components

- Generally called COTS software (Commercial Off-The-Shelf)

Examples:

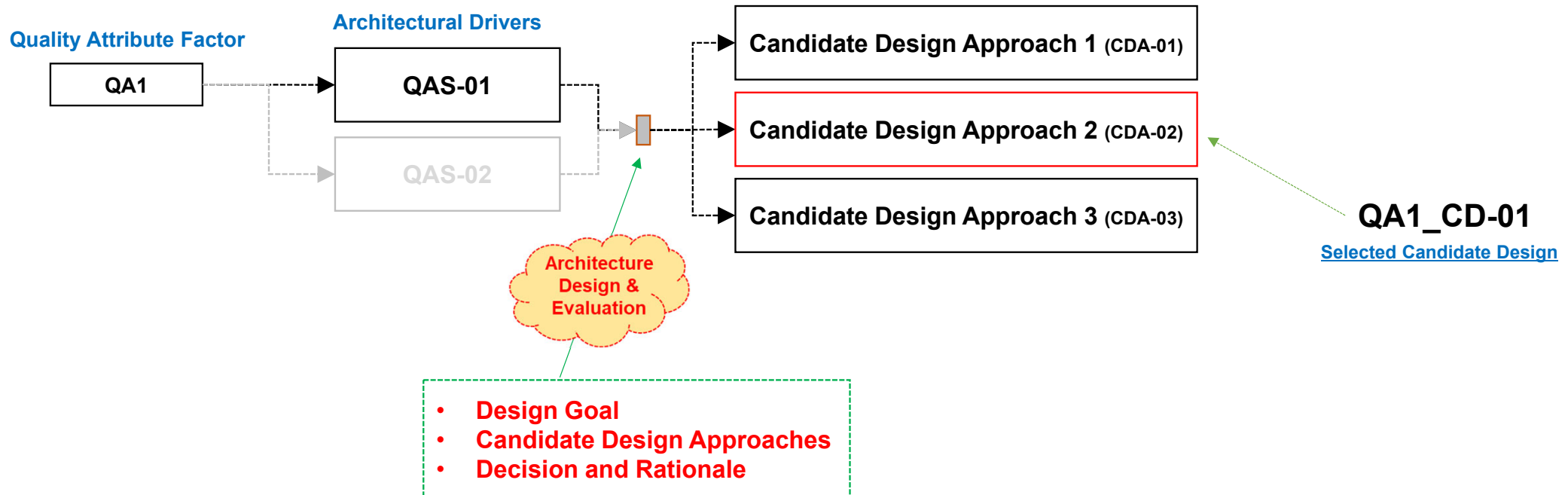
- Technical family
- Products (COTS)
- Application framework (Hibernate, Rest, Spring, Swing, etc.)
- Platform (Java, .Net, Google Cloud, etc.)



4.1 Candidate Designs per QA

- **Candidate Design Approaches (CDA)**

- Candidates of partial architecture design which satisfy with all QASs in a specific QA(Quality Attribute).
- Proposed, evaluated, and selected by architects for each QA
- Refining the domain model first is highly recommended. (in the full CEP)



* CDA (Candidate Design Approach) is not a widely used term, but only used in this class.

Candidate Desings per QA : Examples

QA	QAS	Candidate Design	Candidate Design Approach (CDA)
QA1: [Performance] Time Behavior	QAS-01	QA1_CD-01	QA1_CD-01_CDA-01: Edge-Based Processing + Dispatcher + Reduce Overhead QA1_CD-01_CDA-02: Edge-Based Processing + Load balancing + Prioritize events QA1_CD-01_CDA-03: Edge-Based Processing + Microservice + Broker
QA2: [Reliability] Availability	QAS-02	QA2_CD-01	QA2_CD-01_CDA-01: Redundant N-Tier + Passive Redundancy + state resynchronization + condition monitoring QA2_CD-01_CDA-02: Redundant N-Tier + Active Redundancy + condition monitoring + shadow operation QA2_CD-01_CDA-03: Condition monitoring + state resynchronization + rollback
QA3: [Maintainability] Modifiability	QAS-03	QA3_CD-01	QA3_CD-01_CDA-01: microservice + encapsulate QA3_CD-01_CDA-02: microservice + broker + Use an Intermediary QA3_CD-01_CDA-03: microservice + Config 기반 + Defer Binding + API Encapsulation
QA4: [Usability] Operability	QAS-04	QA4_CD-01	QA4_CD-01_CDA-01: Config 기반 + Defer Binding + MVC + Maintain Task Model QA4_CD-01_CDA-02: Config 기반 + Defer Binding + MVP + Maintain User Model QA4_CD-01_CDA-03: Config 기반 + Defer Binding + RBAS (Role-Based Access Control) + MVP + Maintain Task Model
QA5: [Security] Authenticity	QAS-05	QA5_CD-01	QA5_CD-01_CDA-01: Authenticate actors + Revoke access + API Gateway QA5_CD-01_CDA-02: Authenticate actors + Revoke access + API Gateway + Local Session Cache (with Periodic sync) QA5_CD-01_CDA-03: Authenticate actors + Revoke access + API Gateway + Pipes and Filters Architecture

Smart Home System

Design Goal

- Provides a detailed goal of the design decision to achieve a specific QA
 - Stating the architectural design issue being addressed
 - Usually, **it is a more elaborated description** than the corresponding QAS

QA	QAS	Candidate Design	Candidate Design Approach (CDA)
QA1: [Performance] Time Behavior	QAS-01	QA1_CD-01	QA1_CD-01_CDA-01: Edge-Based Processing + Dispatcher + Reduce Overhead QA1_CD-01_CDA-02: Edge-Based Processing + Load balancing + Prioritize events QA1_CD-01_CDA-03: Edge-Based Processing + Microservice + Broker
QA2: [Reliability] Availability	QAS-02	QA2_CD-01	QA2_CD-01_CDA-01: Redundant N-Tier + Passive Redundancy + state resynchronization + condition monitoring QA2_CD-01_CDA-02: Redundant N-Tier + Active Redundancy + condition monitoring + shadow operation QA2_CD-01_CDA-03: Condition monitoring + state resynchronization + rollback

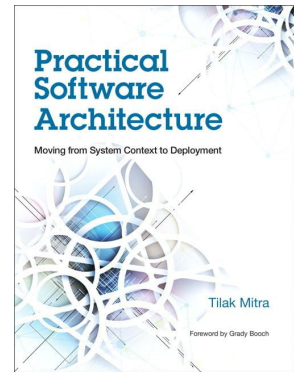
Design Goal:

입주민, 정기 방문자 등 스마트 홈 시스템의 사용자들은 모바일 앱 또는 월패드를 통해 조명, 난방 등 다양한 IoT 기기들을 제어하는 기능을 주로 사용한다. 이러한 제어는 대부분 실내 환경에서 수행되며, 사용자는 요청한 IoT 기기 조작이 즉시 발생하기를 기대한다. 특히 월패드의 경우 IoT Device 로 바로 요청을 전송하여 빠르게 처리가 가능하나, 모바일 기기의 경우는 크게 Application → 중앙 서버 → 단지 서버 → 월 패드 → IoT Device 를 거치는 다단계 경로를 통해 처리된다. 이러한 경로는 구조적으로 네트워크 지연 및 처리 병목이 발생할 수 있는 여지가 있다. 또한 기상시간, 취침 전 등 특정 시간대에는 많은 사용자들이 동시에 기기 제어를 수행할 수 있기 때문에, 과부하나 처리 지연 없이 빠르게 요청을 분산 및 처리할 수 있는 구조 설계가 필요하다.

이를 해결하기 위해, Edge-Based Processing 을 기반으로 여러 Architecture style 및 tactic 을 결합한 CDA 3 가지를 제안한다.

Candidate Design Approach (CDA)

- Illustrates with naive figures **various design alternatives** that have been considered with the objective of **solving the problem** under consideration.
 - It is okay if some architecture problems have only one alternative and that is the one chosen as the solution (but it is rare.)
- Each design approach is described in detail along with its **pros and cons**.



CDA ID	<i>Title</i> of the approach
Description	• Present and describe the design with diagrams • <u>Use naive/UML diagrams or View models</u> • <u>Describe design concepts applied, including reference architectures, architectural styles/patterns, architectural tactics, and principles</u>
Pros	Discuss architectural drivers promoted by the design alternative
Cons	Discuss architectural drivers inhibited by the design alternative

CDA : Examples

Stakeholders' Business Goals

월패드 SW 개발 부서	BG-30	사용자가 선택한 기능을 300ms 이내에 빠르게 처리하여 시스템 지연에 대한 불만이 생기지 않으면 좋겠다..	중
Mobile Application 개발 부서	BG-25	사용자가 특정 기능을 선택하면 500ms 이내에 실제로 처리하여 사용자가 시스템 지연을 느끼지 않으면 좋겠다.	중

System Features (User Requirements)



SF-11	지연 없는 스마트 홈 시스템 기능 사용	스마트 홈 시스템의 각 기능은 사용자의 요청을 빠르게 처리해야 한다.	중	BG-25, BG-30
-------	-----------------------	--	---	--------------

QAS



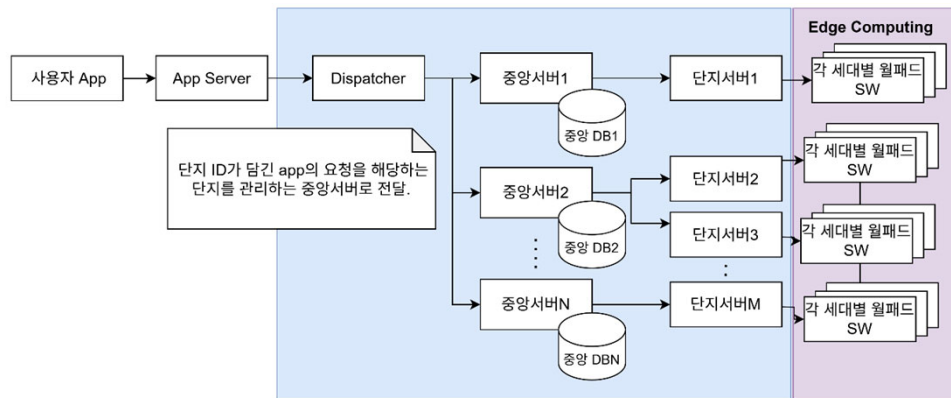
ID	Title	QA Type	Priority		Related Use Case	System Feature ID
			I	D		
QAS-01	지연 없는 스마트 홈 시스템 기능 사용	[Performance Efficiency] Time Behavior	중	중	UC-02, UC-04	SF-11

CDA



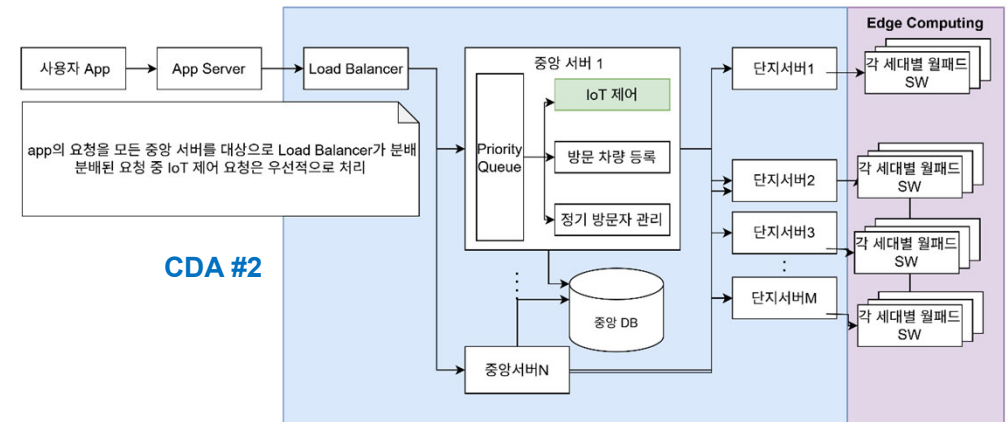
QA	QAS	Candidate Design	Candidate Design Approach (CDA)
QA1: [Performance] Time Behavior	QAS-01	QA1_CD-01	QA1_CD-01_CDA-01: Edge-Based Processing + Dispatcher + Reduce Overhead QA1_CD-01_CDA-02: Edge-Based Processing + Load balancing + Prioritize events QA1_CD-01_CDA-03: Edge-Based Processing + Microservice + Broker

QA1_CD-01_CDA-01	Edge-Based Processing + Dispatcher + Reduce Overhead
Description	이 구조는 모바일 앱의 IoT 제어 요청을, 사전에 각 주거단지별로 지정된 중앙 서버를 통해 해당 단지 서버로 빠르게 전달하는 것을 목표로 한다. - Dispatcher 는 단지 ID 를 기반으로, 요청을 미리 지정된 중앙 서버로 직접 라우팅함으로써, 단지 서버를 탐색하기 위한 중간 처리 과정을 제거하고 오버헤드를 최소화한다. - 중앙 서버는 자신이 담당하는 단지 서버 그룹에 대한 정보를 보유하고 있어 바로 단지 서버로 요청 data 전송이 가능하다. - 단지 서버는 해당 요청을 Edge 역할을 수행하는 세대 내 월패드로 전달하여, 실제 IoT 기기 제어가 수행된다.
Pros	(+) 전체 단지 중 목표 단지를 찾는 overhead 를 줄일 수 있다. (+) 새로운 단지가 추가되는 경우, Dispatcher 로직과 단지가 추가될 중앙 서버만 수정하면 된다. (+) 특정 중앙 서버에 장애가 생겨도 해당 서버를 제외한 서버에 연결된 단지들은 중단 없이 사용이 가능하다. (++) 단지별 정보, 설정 등이 담긴 DB 를 각 중앙서버 별로 나누어 관리할 수 있어 DB 접근에 대한 부하를 줄일 수 있다. (++) 실질적인 IoT 제어는 월패드에서 모두 수행하므로, 중간 서버 및 단지 서버는 단순 요청 전달 기능만 수행하면 된다.
Cons	(-) Dispatcher 가 단일 실패 지점이 될 수 있다. (--) 중앙서버와 단지서버 간의 관계가 고정되어 있어 특정 단지에 예상 못한 수준의 요청이 몰리는 경우 바로 서버 증설은 불가하다. (-) 특정 중앙 서버에 장애가 생기는 경우 해당 서버에 연결된 단지들은 모두 서비스 중단이 발생한다.



CDA #1

QA1_CD-01_CDA-02	Edge-Based Processing + Load balancing + Prioritize events
Description	이 구조는 IoT 제어 요청의 우선순위 보장과 중앙 서버 자원의 효율적 활용을 목표로 한다. - App Server 는 Load Balancer 를 통해 모든 중앙 서버를 대상으로 요청을 분산하고, 분산된 요청은 각 중앙 서버 내의 Priority Queue 로 들어간다. 이를 통해 IoT 제어 요청이 우선적으로 처리되도록 보장된다. - 각 중앙 서버는 중앙 DB 를 참조하여 모든 단지 서버를 파악하고 요청을 전달한다. - 단지 서버는 해당 요청을 Edge 역할을 수행하는 세대 내 월패드로 전달하여, 실제 IoT 기기 제어가 수행된다.
Pros	(+) IoT 요청에 대한 우선순위 보장으로, 시간에 민감한 요청을 신속하게 처리할 수 있다. (++) 중앙 서버 자원 전체를 활용할 수 있어 부하 분산 및 탄력적 확장이 가능하다. (++) 특정 중앙 서버에 장애가 발생하는 경우에도 서비스 중단 없이 운영될 수 있다. (++) 실질적인 IoT 제어는 월패드에서 모두 수행하므로, 중간 서버 및 단지 서버는 단순 요청 전달 기능만 수행하면 된다.
Cons	(-) 모든 중앙 서버가 모든 단지서버에 접근해야 하므로 연결 관리 복잡도가 증가한다. (--) 단지 정보가 중앙 DB 에 집중되므로, 단일 실패 지점이 될 우려가 있고, DB 부하/병목 우려가 있다.

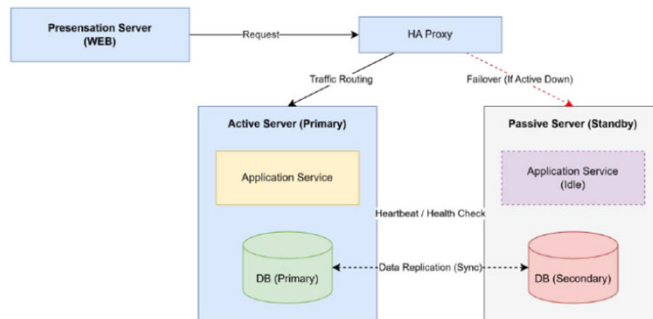


CDA #2

DS Examples

4.1.1.2. Design Approach List

4.1.1.2.1. Design Approach #1: Failover Standby Strategy (Passive Redundancy)



Description: 두 대의 서버 중 하나(Primary)만 트래픽을 처리하고, 다른 하나(Secondary)는 유휴 상태(Ide)로 대기하며 데이터 동기화만 수행한다. 장애 발생 시 고가용성(HA Proxy) 솔루션이 이를 감지하고, 사용자가 접속을 대기 중인 서버로 넘겨주어 서비스를 재개한다.

Pros:

- (+) Simplicity: 구성이 직관적이며, 운영 및 문제 해결(Troubleshooting) 과정이 단순하다.
- (++) Data Consistency: 한 시점에 하나의 서버만 쓰기(Write)를 수행하므로 데이터 충돌이나 정합성 문제가 거의 없다.
- (+) Isolation: 활성 서버의 소프트웨어적 오류(메모리 누수 등)가 대기 서버로 즉시 전파되지 않아 안전한 복구가 가능하다.

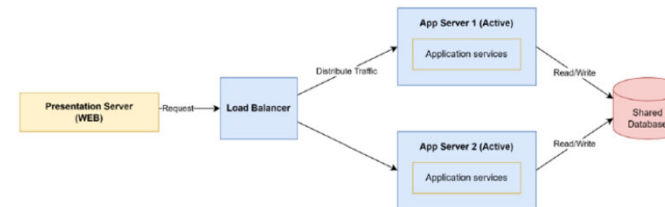
Cons:

- (-) Resource Waste: 멸형한 예비 서버 자원의 50%가 평상시 활용되지 못하고 낭비된다.
- (-) Failover Latency: 장애 감지 후 대기 서버가 활성화되는 과정에서 수 초~수 분간 서비스

36

단절이 발생한다.

4.1.1.2.2. Design Approach #1: Traffic Distribution Strategy (Load Balancer)



Description: 스케일 업이 가능한 두 대 이상의 서버 앞에 로드 밸런서(Load Balancer)를 배치하여 트래픽을 분산 처리한다. 모든 서버가 활성(Active) 상태로 동작하며, 특정 노드 장애 시 로드 밸런서가 해당 노드로 가는 트래픽을 차단함으로써 무중단 서비스를 지속한다.

Pros:

- (++) High Throughput: 모든 서버 자원을 100% 활용하므로 시스템의 전체 처리 용량이 증가한다. (QA-03 기여)
- (++) Zero Downtime: 한 노드가 죽어도 다른 노드가 이미 트래픽을 받고 있으므로, 사용자 관점에서 서비스 중단이 없다.
- (+) Scalability: 로드 밸런서 하위에 노드만 추가하면 되는 수평적 확장 구조를 가진다.

Cons:

- (-) Complexity: 분산 환경에서의 세션 관리 및 데이터 동기화 구현 난이도가 높다.
- (-) Consistency Risks: 여러 서버에서 동시에 데이터를 변경할 경우, 동시성 제어(Concurrency Control) 실패 시 데이터 불일치 위험이 있다.

4.2 Candidate Decisions per QA

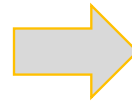
- Describe any **design trade-offs** relevant to the design decisions in terms of ADs.
 - Describes the **rationale** behind choosing the solution among the various alternatives, substantiated by a list of **architecture design principles** that the solution complies with, along with a potential list of principles that may be in noncompliance (substantiated by an explanation for the deviations).
- Select **one candidate design approach** for the QA.

QA Name		Analysis	Candidate Design Approach (CDA) #1 (Selected)	Candidate Design Approach #2	Candidate Design Approach #3
ID	Title				
QAS-01		Pros	(+) Description	(++) (+)	
		Cons	(-)	(--) (-)	
QAS-02					

QA	QAS	CD	Description
QA1: Performance	QAS-01 QAS-02	QA1_CD-01 (+ Title)	

Candidate Design :

[Performance] Time Behavior		Analysis	CDA #1 Edge-Based Processing + Dispatcher + Reduce Overhead	CDA #2 Edge-Based Processing + Load balancing + Prioritize events (Selected)	CDA #3 Edge-Based Processing + Microservice + Broker
ID	Title				
QAS-01	자연 없는 스마트 홈 시스템 기능 사용	Pros	(+) 전체 단지 중 목표 단지를 찾는 overhead 를 줄일 수 있다. (+) 새로운 단지가 추가되는 경우, Dispatcher 로직과 단지가 추가될 중앙 서버만 수정하면 된다. (+) 특정 중앙 서버에 장애가 생겨도 해당 서버를 제외한 서버에 연결된 단지 들은 중단 없이 사용이 가능하다. (++) 단지별 정보, 설정 등이 담긴 DB 를 각 중앙서버 별로 나누어 관리할 수 있어 DB 접근에 대한 부하를 줄일 수 있다. (++) 실질적인 IoT 제어는 월패드에서 모두 수행하므로, 중간 서버 및 단지 서버는 단순 요청 전달 기능만 수행하면 된다.	(+) IoT 요청에 대한 우선순위 보장으로, 시간에 민감한 요청을 신속하게 처리할 수 있다. (++) 중앙 서버 자원 전체를 활용할 수 있어 부하 분산 및 탄력적 확장이 가능하다. (++) 특정 중앙 서버에 장애가 발생하는 경우에도 서비스 중단 없이 운영될 수 있다. (++) 실질적인 IoT 제어는 월패드에서 모두 수행하므로, 중간 서버 및 단지 서버는 단순 요청 전달 기능만 수행하면 된다.	(+) 기능 분리에 따른 자원 격리로, IoT 요청이 타 기능과의 간섭 없이 처리될 수 있다. (+) 각 기능별로, 할당된 중앙 서버 자원 전체를 활용할 수 있어 Broker 로직 수정 만으로도 부하 분산 및 탄력적 확장이 가능하다. (+) 특정 기능 담당 서버에 문제가 있더라도 나머지 기능은 중단 없이 운영될 수 있다. (+) Microservice 구조를 통해 각 기능별 서비스 간의 성능 영향을 최소화한다. (++) 실질적인 IoT 제어는 월패드에서 모두 수행하므로, 중간 서버 및 단지 서버는 단순 요청 전달 기능만 수행하면 된다.
		Cons	(-) Dispatcher 가 단일 실패 지점이 될 수 있다. (--) 중앙서버와 단지서버 간의 관계가 고정되어 있어 특정	(-) 모든 중앙 서버가 모든 단지서버에 접근해야 하므로 연결 관리 복잡도가 증가한다. (--) 단지 정보가 중앙	(-) Broker 가 단일 분기점이자 병목/장애 지점이 될 수 있다. (--) 기능별 중앙 서버 구성이므로, 동시



QA	QAS	CD	Description
QA1: Performance / Time Behavior	QAS-01	QA1_CD-01 Edge-Based Processing + Load balancing + Prioritize events	이 구조는 IoT 제어 요청의 우선순위 보장과 중앙 서버 자원의 효율적 활용을 목표로 한다. - App Server 는 Load Balancer 를 통해 모든 중앙 서버를 대상으로 요청을 분산하고, 분산된 요청은 각 중앙 서버 내의 Priority Queue 로 들어간다. 이를 통해 IoT 제어 요청이 우선적으로 처리되도록 보장된다. - 각 중앙 서버는 중앙 DB 를 참조하여 모든 단지 서버를 파악하고 요청을 전달한다. - 단지 서버는 해당 요청을 Edge 역할을 수행하는 세대 내 월패드로 전달하여, 실제 IoT 기기 제어가 수행된다. (선정 이유) 본 QAS 의 핵심인 "IoT 제어 요청 후 월패드 300 ~ 모바일 500ms 내 제어 완료" 조건을 만족시키기 위해서는, IoT 제어 동작 자체를 빠르게 처리하는 것과 모바일에서 월패드까지 요청이 전달되기 까지의 시간을 줄이는 것이 핵심이다. 세 구조 모두 각 세대 내 월패드를 Edge 로 사용하여 IoT 제어 자체의 지연은 유사하나, 모바일에서 월패드까지의 요청 전달 측면에서 QA1_CD-01_CDA-02 가 가장 효율적인 것으로 판단된다. QA1_CD-01_CDA-02 는 Priority Queue 를 통해 가장 시간에 민감한 요청인 IoT 제어를 우선적으로 처리할 수 있으며, 모든 중앙 서버가 동일한 구조로 확장 가능하여 특정 서버 장애 시에도 목표한 시간 범위 내에 대응할 수 있도록 탄력성을 확보할 수 있다. 또한 타 CDA 대비 Dispatcher 나 Broker 등 중앙 집중된 의사 결정 로직이 없기 때문에 시스템의 병목 및 단일 실패 위험이 상대적으로 낮아 안정적으로 시스템 성능을 유지할 수 있다.

Candidate Design (Decision)

Rationale (Design Evaluation)

DS Examples

더 다양한 Design Concepts이 조합되면 더 좋아요.

Full CEP에서는 QA 별로 비교분석을 했으나, Mini CEP에서는 전체 QA를 대상으로 비교분석을 수행해요. (약식 ATAM)

4.2. Design Decisions per QA

4.2.1. Decision for QA1 서버 장애 시 서비스 가용성 확보

Quality Attribute		Analysis	Design Approach (DA) #1	Design Approach (DA) #2
ID	Title		Failover Strategy (Passive Redundancy)	Traffic Distribution Strategy (Load Balancer) SELECTED
QA-01	서비스 가용성	Pros	(++) 데이터 정합성 유지가 쉬움 (+) 구성 및 운영이 비교적 단순함 (+) 장애 격리 효과 우수	(++) 장애 발생 시 전환 지연없이 즉시 서비스 지속 가능 (+) 단일 노드에 가해지는 작업 부하가 분산되어 장애 확률 자체를 낮춤 (++) 자원활용 효율성 높음
		Cons	(-) Failover 시 수 초~수 분의 서비스 지연(Glitch) 발생 (-) 자원 낭비 (50% 유휴 자원)	(-) 세션 클러스터링 및 데이터 동기화 구성 복잡도 높음 (-) 동시성 제어 실패 시 데이터 불일치 위험
QA-02	대용량 트래픽 처리	Pros	-	(++) 수평적 확장(Scale-out) 구조이므로 대용량 트래픽 분산 처리에 최적
		Cons	(-) 트래픽 폭주 시에도 한 대의 서버만 부하를 감당해야 함	-
QA-03	실시간 영상 분석 및 상태 동기화 성능	Pros	-	(+) 모든 서버가 분석에 참여하므로 처리 용량(Capacity) 2배 증가
		Cons	(-) 서버 자원의 절반이 유휴 상태이므로 최대 처리량 제한됨	-
QA-04	운영정책의 동적변경	Pros	-	-
		Cons	-	-
QA-05	이기종 CCTV 연동	Pros	-	-
		Cons	-	-

4.2. Design Decisions per QA

4.2.1. Decision for QA1 (Availability)

Quality Attribute		Analysis	Design Approach (DA) #1	DA #2	DA #3
ID	Title		Active-Active	Active-Sparse	Cold Spare
	성		생 시 즉시 대체 가능	로 5초 이내 복구 가능	용 최소
		Cons	(--) 리소스 비용 증가, 중복 영상 처리 오버헤드	(-) 전환 시 미세한 지연 가능	(--) 재-기동 시간 발생
QA-02	Request 처리 성능	Pros	영향 없음	영향 없음	영향 없음
		Cons	영향 없음	영향 없음	영향 없음
QA-03	배경 알고리즘 수정 용이성	Pros	영향 없음	영향 없음	영향 없음
		Cons	영향 없음	영향 없음	영향 없음
QA-04	CCTV 호환성	Pros	(+) 백터 이중화 가능	(+) 백업 백터 구성 가능	(+) 백터 교체 용이
		Cons	(--) 동기화 필요	영향 없음	영향 없음
QA-05	CCTV 영상 분석 확장성	Pros	다양한 입력 기반 분석 시나리오 가능	(+) 단일 파이프라인 유저로 확장 단순	(+) 분석 로직 독립적 확장 가능
		Cons	(--) 파이프라인 중복 처리로 구조 복잡	(-) 전환 시 분석 동기화 고려 필요	(--) 실시간 분석 적용에 제약
QA-06	학생정보 보호	Pros	영향 없음	영향 없음	영향 없음
		Cons	영향 없음	영향 없음	영향 없음
QA-07	배경알고리즘 확장 용이성	Pros	영향 없음	영향 없음	영향 없음
		Cons	영향 없음	영향 없음	영향 없음

최대한 자세히 분석!

4.3 Design Decision (Overview)

- Collect all the selected CDs to complete **the final design decision (DD)**.
 - All design decisions that are considered architecturally important to satisfy the business, technical, and engineering goals are captured and summarized.
 - The entire DD is described through a **naive picture**(s).
- **Details of the entire DD** will be explained through **three views in Section 5**. (Skipped in the CEP for A7)
 - **5.1 Architecture Overview**
 - **5.2 Structure View**
 - **5.3 Behavior View**
 - **5.4 Deployment View**

Design Decision : Examples

본 스마트 홈 시스템은 Performance, Reliability, Maintainability, Usability, Security 등 다양한 품질 속성 요구를 동시에 만족시키기 위해 각 속성에 따라 최적의 CDA를 조합하여 적용하였다. 모바일 앱 및 월패드는 사용성을 극대화하는 MVP 패턴을 적용하여 TASK 기반 UI를 제공하며, API Gateway와 인증 서버를 중심으로 보안성을 확보하였다. 중앙 서버는 Load Balancer와 Priority Queue를 통해 실시간 처리 성능을 보장하고, 단지 서버는 Active Backup 구조를 통해 가용성을 확보하였다. 외부 시스템 간의 연계를 위해서는 Config 기반 Defer Binding과 API Encapsulation을 적용하였다. 이러한 통합 설계를 통해 전체 시스템은 앞서 3.2 절에서 제시된 다섯 가지 주요 품질 속성 시나리오를 모두 만족하도록 구성되었다.

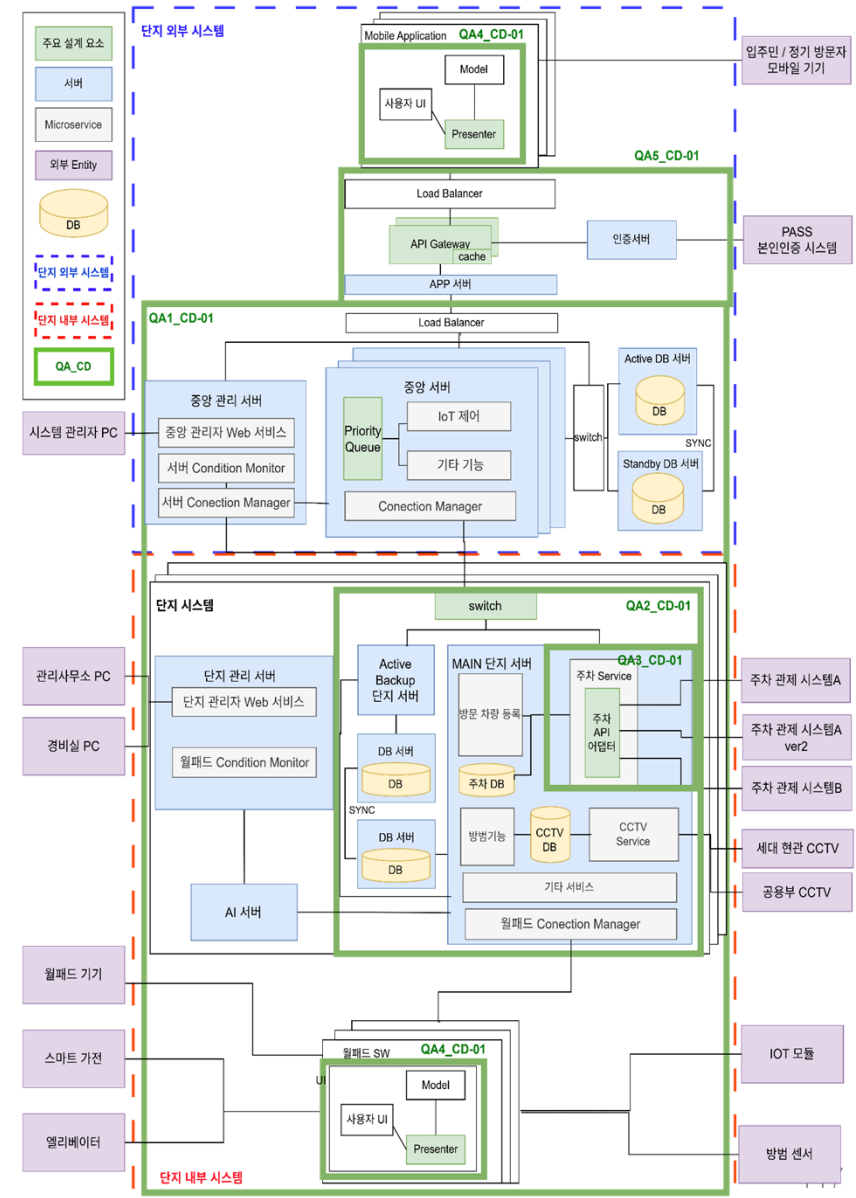
[QA1 Performance - QA1_CD-01]

중앙 서버는 Load Balancer를 통해 서버 부하를 분산 처리하고, 서버 내부에는 Priority Queue를 도입하여 IoT 제어 요청의 우선순위를 동적으로 판단한다. 이를 통해 IoT 요청을 제한 시간 내에 처리하고, 사용자 요청의 성능을 높일 수 있다. 또한 중앙 서버의 Connection Manager를 각 기능 모듈과 분리하여 병렬 처리 구조를 유지함으로써 높은 트래픽에도 유연하게 대응할 수 있다. 또한 IoT 기기 제어 등 주요 비즈니스 로직은 Edge인 월패드 SW로 전가하여, 과부하가 예상되는 중앙 시스템, 단지 서버 등은 트래픽 처리에만 집중할 수 있도록 하였다. 최초 설계에서는 중앙서버들이 동시에 접속하는 DB를 하나 두었으나, 4.3절에서 이로 인한 DB 병목 현상에 대한 우려가 확인되어 DB를 이중화 하여 Active / Standby DB 서버를 두었다.

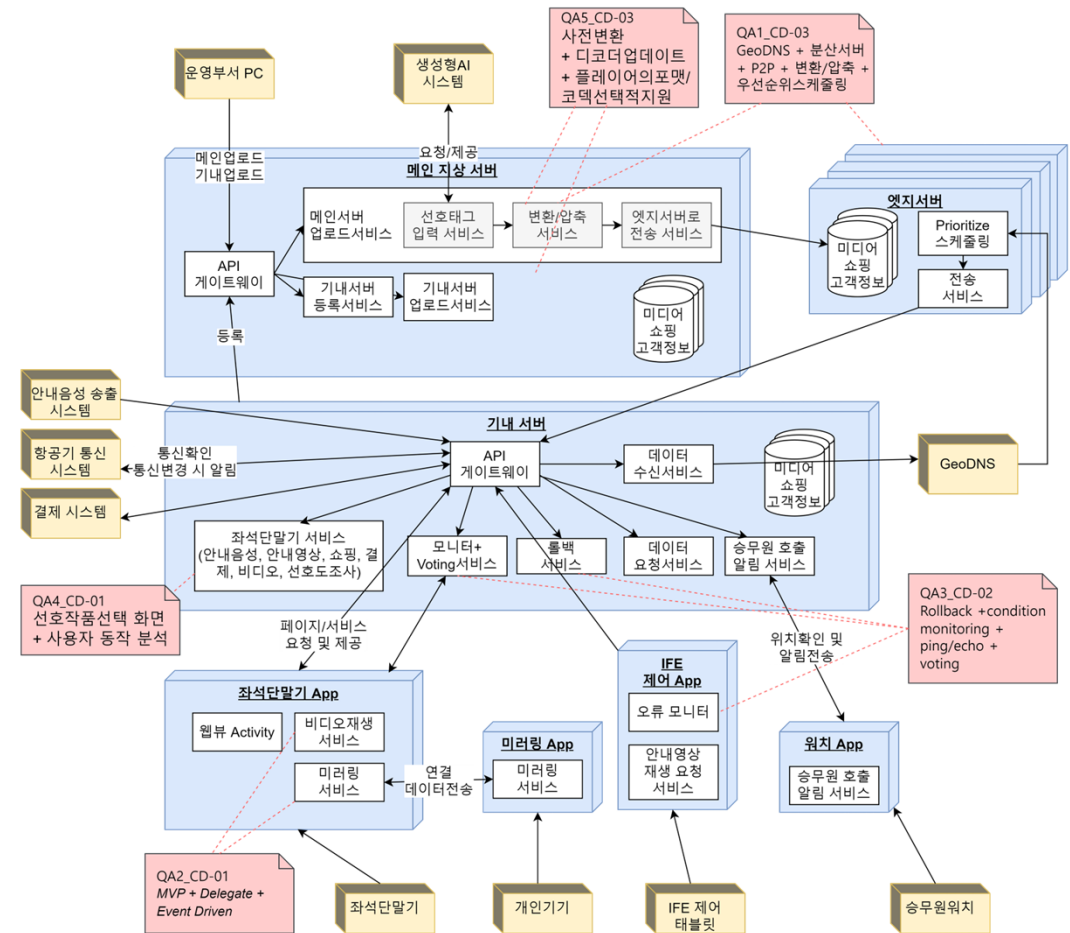
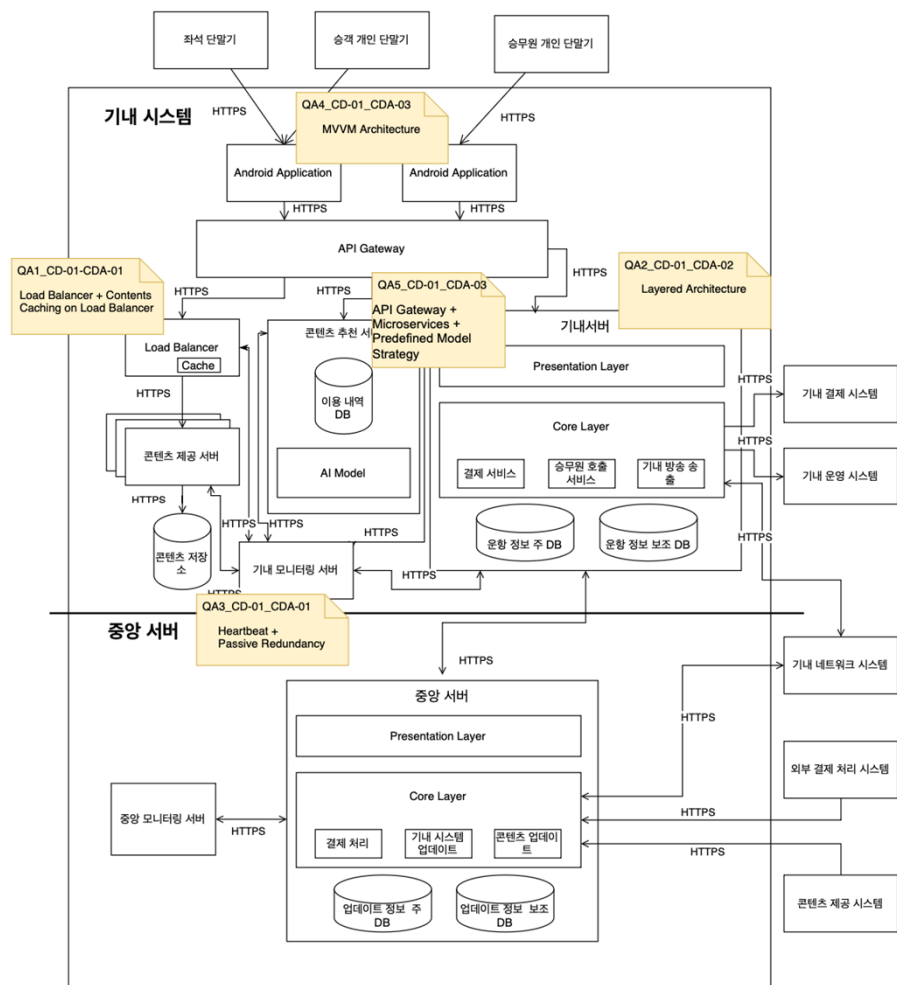
[QA2 Availability - CDA: QA2_CD-01]

...

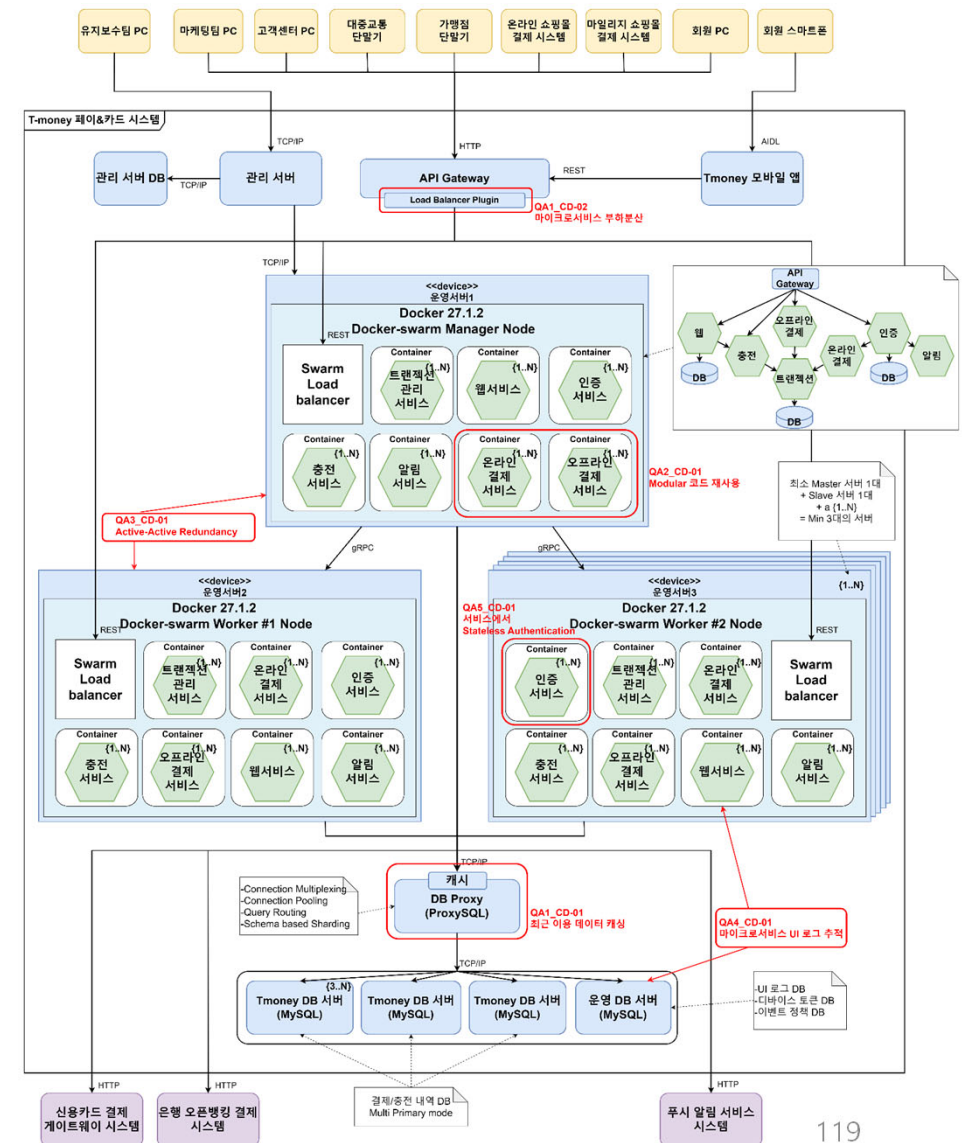
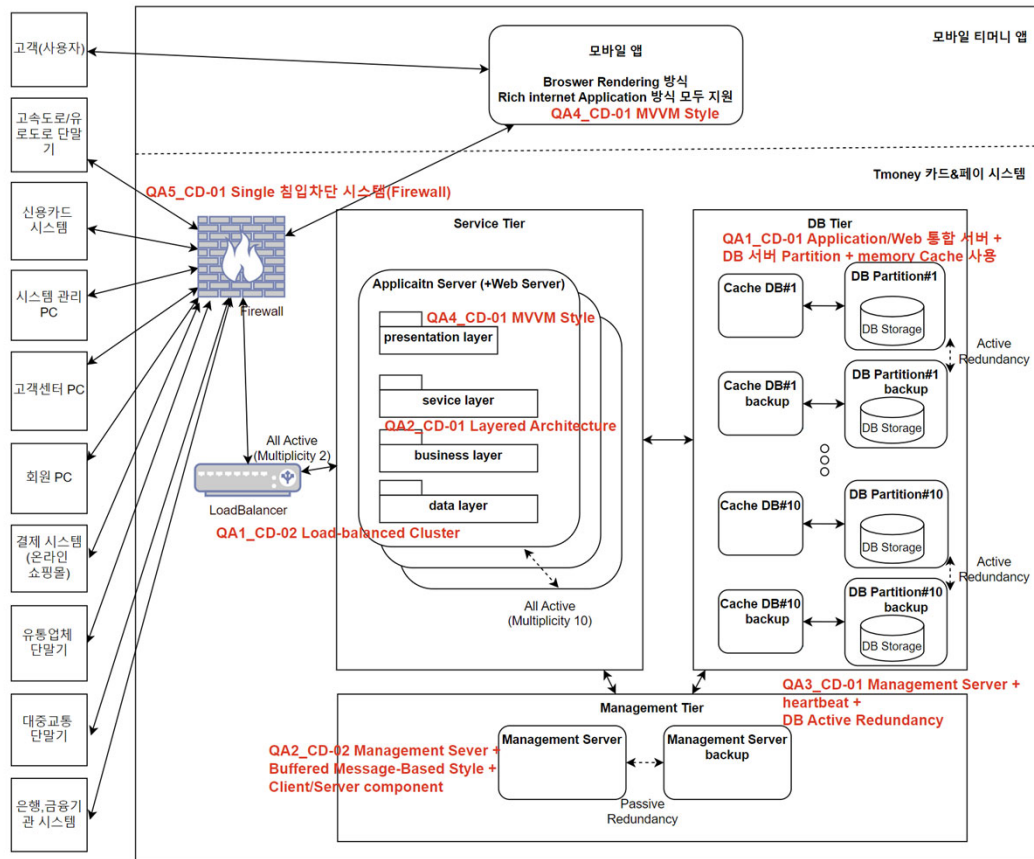
Smart Home System



Design Decision : Examples

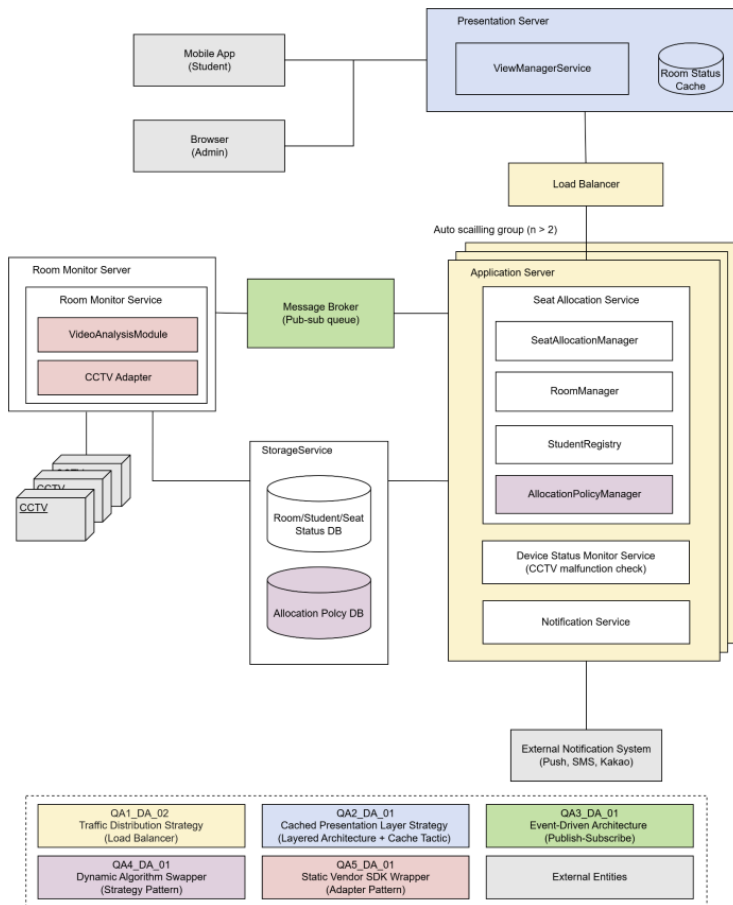


Design Decision : Examples

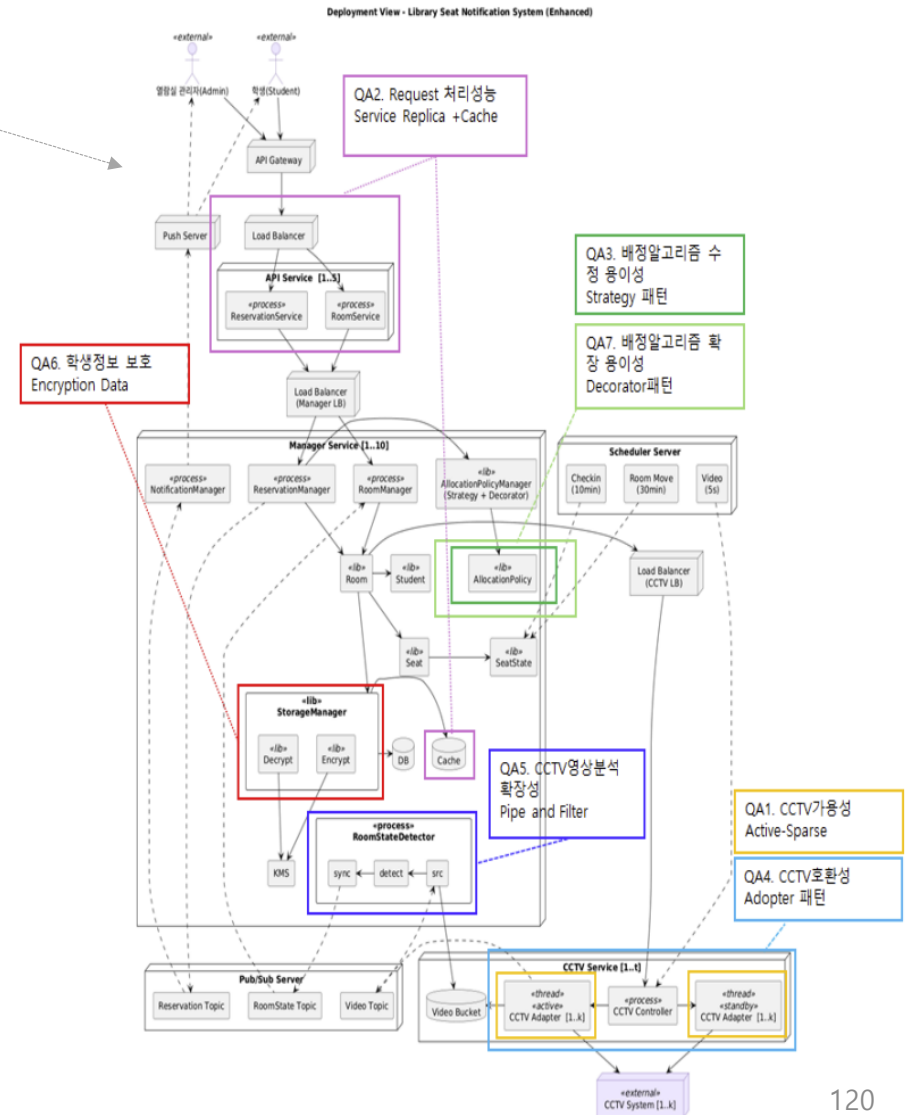


DS Examples

그림 이해 잘 안돼요.
시스템의 내·외부 범위가 구분 안돼요.



4.3. Design Overview



5. Summary

Where We are Now in AD

1. Project Overview

- 1.1 Project Background
- 1.2 Stakeholders
- 1.3 Business Goals

2. System Overview

- 2.1 System Context Diagram
- 2.2 System Features
- 2.3 Assumptions about the System

3. Architectural Drivers

- 3.1 Primary Functionality
- 3.2 Quality Attribute Scenario

4. Architecture Design

- 4.1 Candidate Designs per QA
- 4.2 Candidate Decisions per QA
- 4.3 Design Decision (Overview)

5. Summary

Summary

- Architecture Design 과정에서,
 - 미처 고려하지 못했던 내용들
 - 발생한 Candidate Design 간의 Conflicts (e.g., Security vs. Reliability) 및 해결 방안 제안
- **Appendix**
 - 사용한 모든 중요한 Design Concepts에 대한 간략한 설명 (0.3 페이지씩)

